



lecture 0

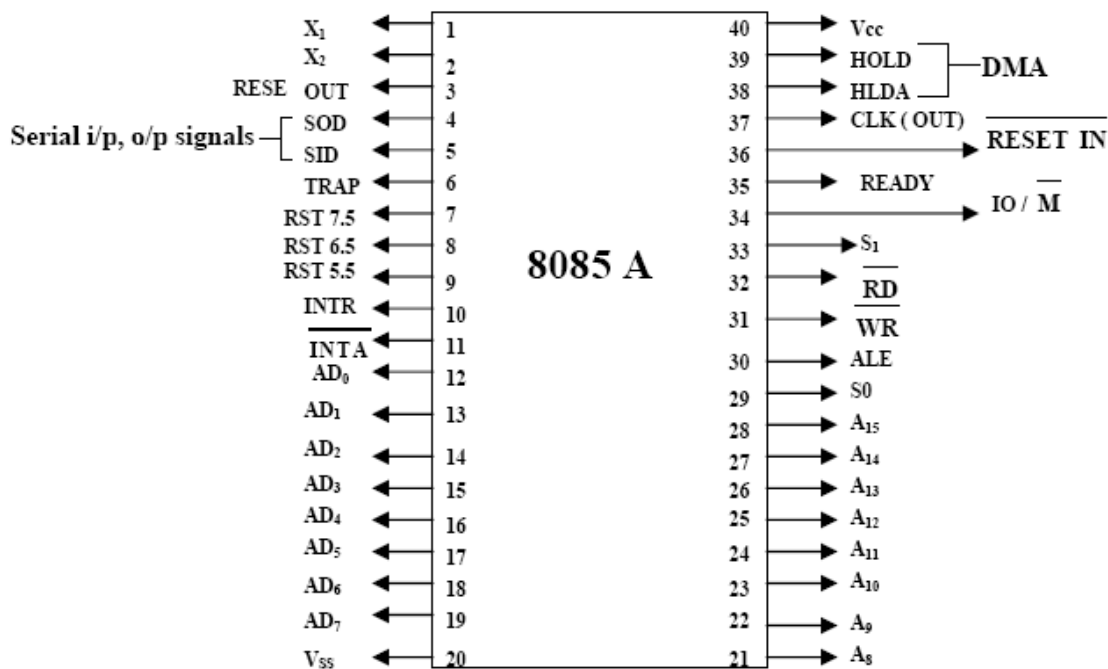
The 8085 microprocessor

- General definitions
- Overview of 8085 microprocessor

The main features of 8085 μ p are:

- It is a 8 bit microprocessor.
- It is manufactured with N-MOS technology.
- It has 16-bit address bus and hence can address up to $2^{16} = 65536$ bytes (64KB) memory locations through $A_0 - A_{15}$.
- The first 8 lines of address bus and 8 lines of data bus are multiplexed $AD_0 - AD_7$.
- Data bus is a group of 8 lines $D_0 - D_7$.
- It supports external interrupt request.
- A 16 bit program counter (PC)
- A 16 bit stack pointer (SP)
- Six 8-bit general purpose register arranged in pairs: BC, DE, HL.
- It requires a signal +5V power supply and operates at 3.2 MHZ single phase clock.
- It is enclosed with 40 pins DIP (Dual in line package).

Pin Diagram of the 8085 microprocessor



Pin Diagram of 8085



Memory

- Program, data and stack memories occupy the same memory space. The total addressable memory size is 64 KB.
- **Program memory** - program can be located anywhere in memory. Jump, branch and call instructions use 16-bit addresses, i.e. they can be used to jump/branch anywhere within 64 KB. All jump/branch instructions use absolute addressing.
- **Data memory** - the processor always uses 16-bit addresses so that data can be placed anywhere.
- **Stack memory** is limited only by the size of memory. Stack grows downward.
- First 64 bytes in a zero memory page should be reserved for vectors used by RST instructions.

Registers

- **Accumulator** or A register is an 8-bit register used for arithmetic, logic, I/O and load/store operations.
- **Flag Register** has five 1-bit flags.
- **Sign** - set if the most significant bit of the result is set.
- **Zero** - set if the result is zero.
- **Auxiliary carry** - set if there was a carry out from bit 3 to bit 4 of the result.
- **Parity** - set if the parity (the number of set bits in the result) is even.
- **Carry** - set if there was a carry during addition, or borrow during subtraction/comparison/rotation.

INDIVIDUAL	B, C, D, E, H, L
COMBININATON	B & C, D & E, H & L

General purpose registers

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
S	Z		AC		P		CY

Flag register

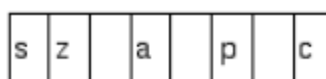


General Registers

- 8-bit B and 8-bit C registers can be used as one 16-bit BC register pair. When used as a pair the C register contains low-order byte. Some instructions may use BC register as a data pointer.
- 8-bit D and 8-bit E registers can be used as one 16-bit DE register pair. When used as a pair the E register contains low-order byte. Some instructions may use DE register as a data pointer.
- 8-bit H and 8-bit L registers can be used as one 16-bit HL register pair. When used as a pair the L register contains low-order byte. HL register usually contains a data pointer used to reference memory addresses.
- **Stack pointer** is a 16 bit register. This register is always decremented/incremented by 2 during push and pop.
- **Program counter** is a 16-bit register.

8085 Programmer's model

Accumulator	A (8)	PSW (8)	Processor status word
	B (8)	C (8)	
	D (8)	E (8)	
	H (8)	L (8)	
SP (16)			Stack pointer
PC (16)			Program counter



PSW

where
c = carry
z = zero
s = sign
p = parity
a = auxiliary carry (BCD arithmetic)

Instruction Types

1. Data transfer or movement
 - a. MOV
2. Arithmetic
3. Logical
4. Branching (Transfer of control)
5. Processor Control



8085 Addressing mode

Addressing modes are the manner of specifying effective address. 8085 Addressing mode can be classified into:

1 - Direct addressing mode: the instruction consist of three byte, byte for the opcode of the instruction followed by two bytes represent the address of the operand

Low order bits of the address are in byte 2

High order bits of the address are in byte 3

Ex: **LDA 2000h**

This instruction load the Accumulator is loaded with the 8-bit content of memory location [2000h]

2 - Register addressing mode

The instruction specifies the register or register pair in which the data is located

Ex: **MOV A,B**

Here the content of B register is copied to the Accumulator

3 - Register indirect addressing mode

The instruction specifies a register pair which contains the memory address where the data is located.

Ex. **MOV M , A**

Here the **HL** register pair is used as a pointer to memory location. The content of Accumulator is copied to that location

4- Immediate addressing mode:

The instruction contains the data itself. This is either an 8 bit quantity or 16 bit (the LSB first and the MSB is the second)

Ex: **MVI A , 28h**
LXI H , 2000h

First instruction loads the Accumulator with the 8-bit immediate data 28h

Second instruction loads the **HL** register pair with 16-bit immediate data 2000h



lecture 1 - Introduction to Microprocessors

- Objective:**
1. General Architecture of a Microcomputer System
 2. Types of Microprocessors
 3. Number Systems

1. General Architecture of a Microcomputer System

The hardware of a microcomputer system can be divided into four functional sections: the *Input unit*, *Microprocessing Unit*, *Memory Unit*, and *Output Unit*. See Fig. 1

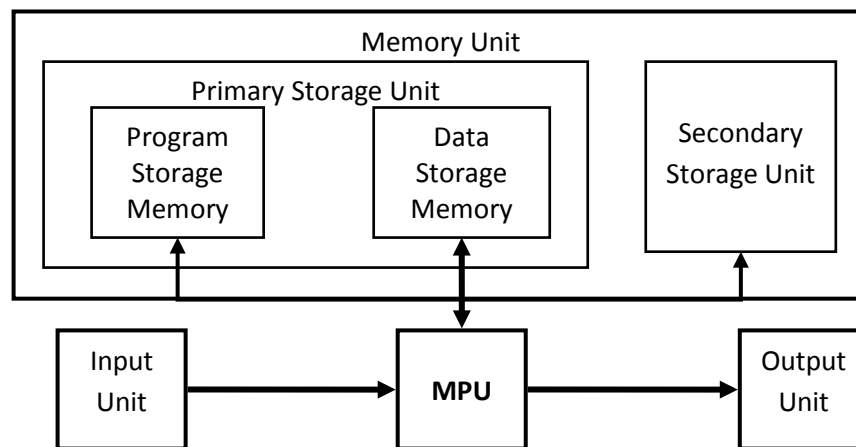


Figure 1

- **MicroProcessorUnit (MPU)** is the heart of a microcomputer. A microprocessor is a general purpose processing unit built into a single integrated circuit (IC).
The Microprocessor is the part of the microcomputer that executes instructions of the program and processes data. It is responsible for performing all arithmetic operations and making the logical decisions initiated by the computer's program. In addition to arithmetic and logic functions, the MPU controls overall system operation.
- Input and Output units are the means by which the MPU communicates with the outside world.
 - o Input unit: keyboard, mouse, scanner, etc.
 - o Output unit: monitor, printer, etc.
- Memory unit:
 - o Primary: is normally smaller in size and is used for temporary storage of active information. Typically ROM, RAM.
 - o Secondary: is normally larger in size and used for long-term storage of information. Like Hard disk, Floppy, CD, etc.



2. Types of Microprocessors

Microprocessors generally is categorized in terms of the maximum number of binary bits in the data they process – that is, their word length. Over time, five standard data widths have evolved for microprocessors: 4-bit, 8-bit, 16-bit, 32-bit, 64-bit.

There are so many manufacturers of Microprocessors, but only two companies have been produces popular microprocessors: *Intel* and *Motorola*. Table 1 lists some of types that belong to these companies (families) of microprocessors.

Table 1: Some Types of Microprocessors:

Type	Data bus width	Memory size
Intel family:		
8085	8	64K
8086	16	1M
80286	16	16M
80386EX , 80386DX	16 , 32	64M , 4G
80486DX4	32	4G + 16K cache
Pentium	64	4G + 16K cache
PentiumIII , Pentium4	64	64G+32K L1 cache +256 L2 cache
Motorola family:		
6800	8	64K
68060	64	4G + 16K cache

Note that the 8086 has data bus width of 16-bit, and it is able to address 1Megabyte of memory.

It is important to note that 80286, 80386, 80486, and Pentium-Pentium4 microprocessors are **upward compatible** with the 8086 Architecture. This mean that 8086/8088 code will run on the 80286, 80386, 80486, and Pentium Processors, but the reverse is not true if any of the new instructions are in use.

Beside to the *general-purpose* microprocessors, these families involve another type called *special-purpose* microprocessors that used in embedded control applications. This type of embedded microprocessors is called *microcontroller*. The 8080, 8051, 8048, 80186, 80C186XL are some examples of microcontroller.

3. Number Systems

For Microprocessors, information such as *instruction*, *data* and *addresses* are described with numbers. The types of numbers are not normally the decimal numbers we are familiar with; instead, binary and hexadecimal numbers are used. Table 2 shows Binary and Hexadecimal representations for some decimal numbers.



Table 1: Binary, and Hexadecimal representation of some numbers:

Decimal	Binary	Hexadecimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

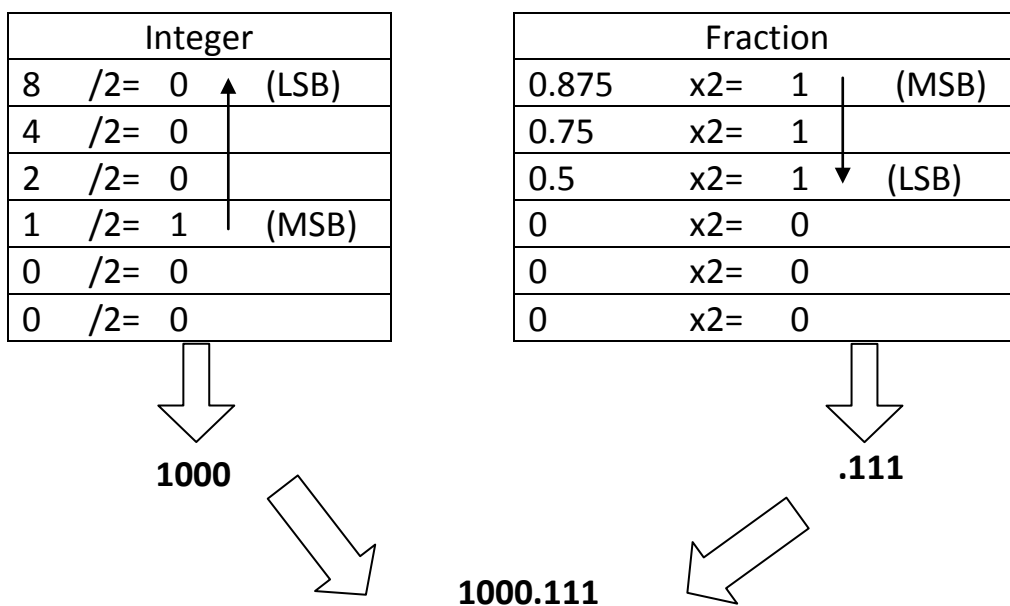
Example 1: Evaluate the decimal equivalent of binary number 101.01_2

Solution:

$$\begin{aligned}
 101.01_2 &= 1(2^2) + 0(2^1) + 1(2^0) + 0(2^{-1}) + 1(2^{-2}) \\
 &= 1(4) + 0(2) + 1(1) + 0(0.5) + 1(0.25) \\
 &= 4 + 0 + 1 + 0 + 0.25 \\
 &= 5.25
 \end{aligned}$$

Example2: Evaluate the binary representation of decimal number 8.875

Solution:





Generally, Binary numbers are expressed in fixed length either:

- 8-bit called Byte
- 16-bit called Word
- 32-bit called Double Word

Example3: Evaluate the 16-bit binary representation of decimal number 102_{10} , then evaluate its hexadecimal representation

Solution:

$$107_{10} = 01101011_2 = 6BH$$



Lecture 2- Software Architecture of 8086

1. Internal Architecture of the 8086

The internal architecture of the 8086 contains two processing units: the *bus interface unit (BIU)* and the *execution unit (EU)*. Each unit has dedicated functions and both operate at the same time. This parallel processing makes the fetch and execution of instructions independent operations. See Fig. 1

The BIU is responsible for performing all external bus operations, such as instruction fetching, reading and writing of data operands for memory, address generating, and inputting or outputting data for input/output peripherals. These operations take place over the system bus. This bus includes 16-bit bidirectional data bus, a 20-bit address bus, and the signals needed to control transfer over the bus.

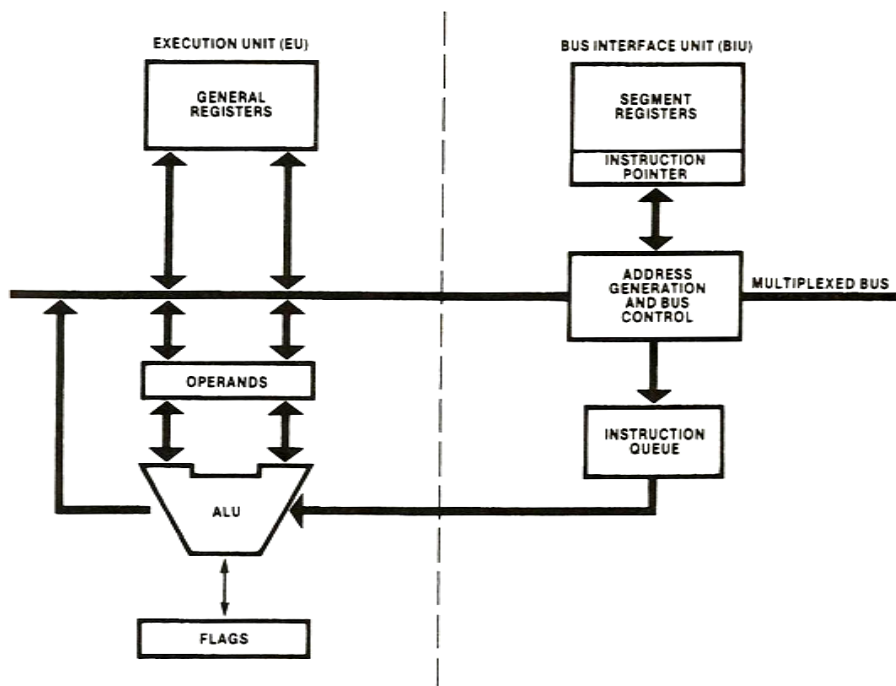


Fig 1: Execution and bus interface units

The BIU uses a mechanism known as *instruction queue*. This queue permits the 8086 to prefetch up to 6 bytes of instruction code.

The EU is responsible for decoding and executing instructions. It contains *arithmetic logic unit (ALU)*, status and control flags, general-purpose register, and temporary-operand registers.



2. Memory address space and data organization

8086 can support 1Mbyte of external memory that organized as individual bytes of data stored at consecutive addresses over the address range 00000_{16} to $FFFFFF_{16}$. The 8086 can access any two consecutive bytes as a *word* of data. The lower-addressed byte is the least significant byte of the word, and the higher-addressed byte is its most significant byte.

Example 1: For the 1Mbyte memory shown in Fig 2, storage location of address 00009_{16} contains the value $00000111_2 = 7_{16}$, while the location of address 00010_{16} contains the value $01111101 = 7D_{16}$. The 16-bit word **225A**₁₆ is stored in the locations $0000C_{16}$ to $0000D_{16}$.

00009	07
0000A	
0000B	
0000C	5A
0000D	22
0000E	
0000F	
00010	7D

Fig2: Part of 1Mbyte memory

The word of data is at an *even-address boundary* if its least significant byte is in even address. It's also called *aligned word*. The word of data is at an *odd-address boundary* if its least significant byte is in odd address. It's also called *misaligned word*, as shown in Fig 3.

To store double word four locations are needed. The double word that its least significant byte store at an address that is a multiple of 4 (e.g. 0_{16} , 4_{16} , 8_{16} , ...) as shown in Fig 4.

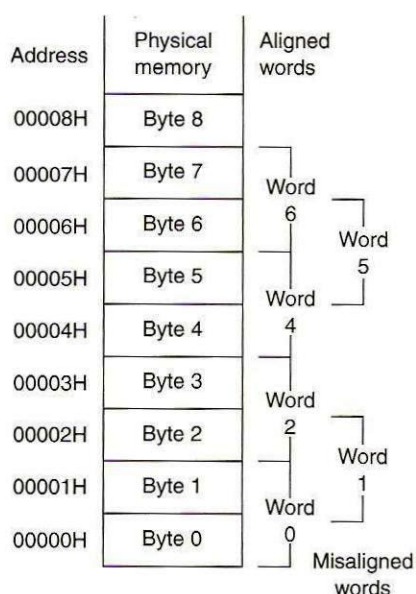


Fig 3 Aligned and misaligned word

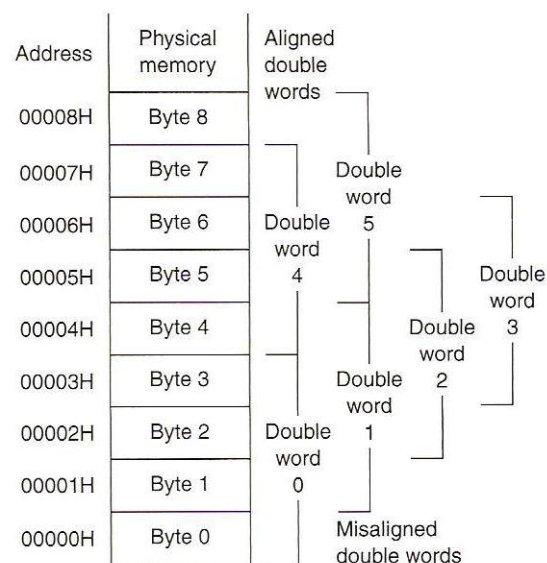


Fig 4 Aligned and misaligned double word



3. Segment registers and memory segmentation

Even though the 8086 has a 1Mbyte address space, not all this memory is active at one time. Actually, the 1Mbytes of memory are partitioned into 64Kbyte (65,536) *segments*. Each segment is assigned a *Base Address* that identifies its starting point (identify its lowest address byte-storage location).

Only four of these 64Kbyte segments are active at a time: the *code segment*, *stack segment*, *data segment*, and *extra segment*. The addresses of these four segments are held in four segment registers: **CS** (code segment), **SS** (stack segment), **DS** (data segment), and **ES** (extra segment). These registers contain a 16-bit base address that points to the lowest addressed byte of the segment (see Fig 5).

Note that the segment registers are *user accessible*. This means that the programmer can change their contents through software.

There is one restriction on the value assigned to a segment as base address: it must reside on a 16-byte address boundary. This is because the memory address is 20 bits while the segment register width is 16 bits. Four bits (0000) must be added to the segment register content to evaluate the segment starting address.

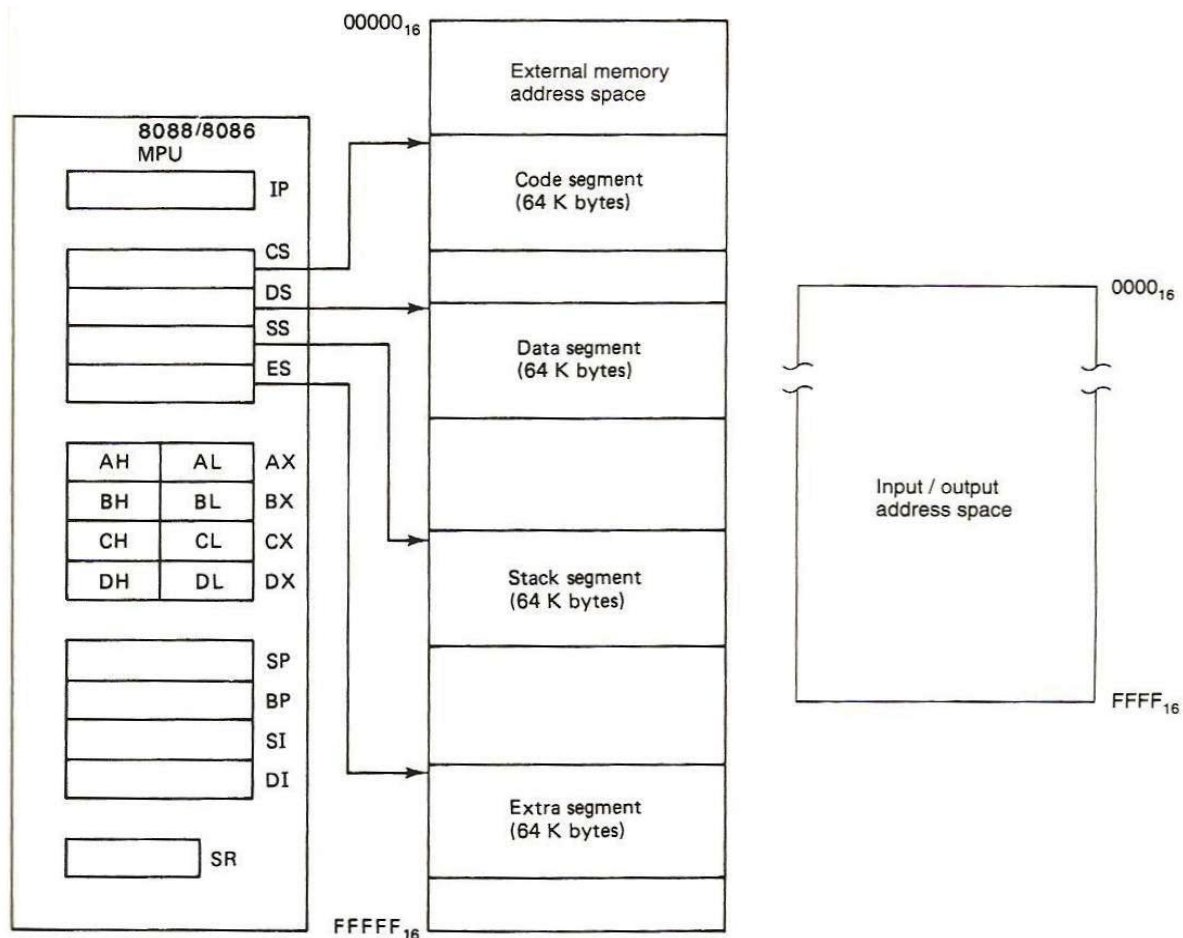


Fig 5: Software model of 8086 microprocessor



Example 2: Let the segment registers be assigned as follow:

CS = 0009H, DS = 0FFFH, SS = 10E0H, and ES = 3281H. We note here that code segment and data segment are overlapped while other segments are disjointed (see Fig 6).

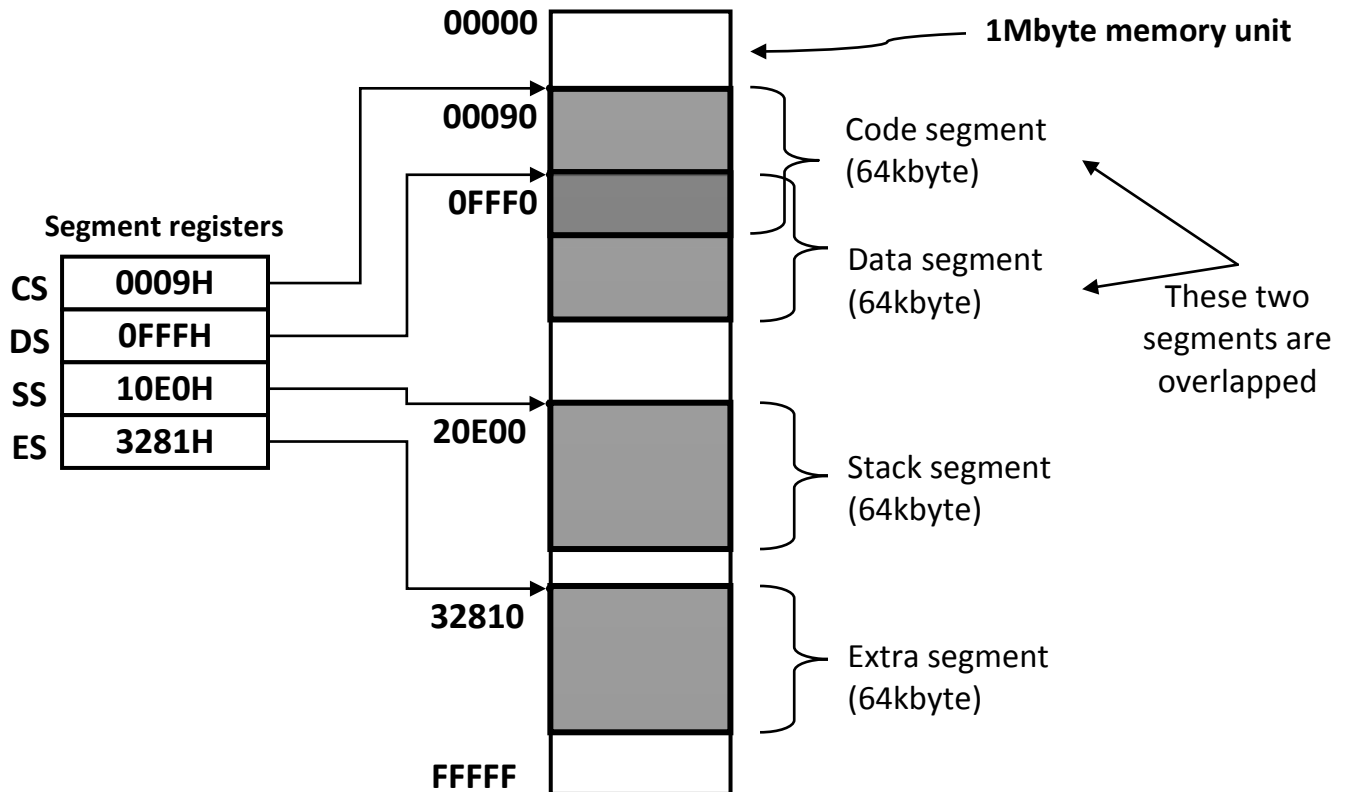


Fig 6: Overlapped and disjointed segments

4. Instruction Pointer

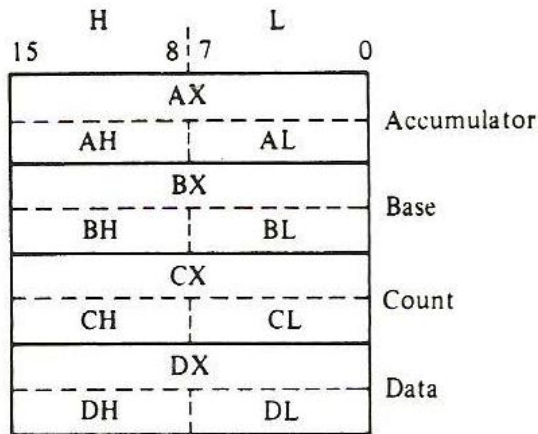
Instruction pointer (**IP**): is a 16 bits in length and identifies the location of the next word of instruction code to be fetched from the current code segment of memory, it contains the offset of the next word of instruction code instead of its actual address.

The offset in **IP** is combined with the current value in **CS** to generate the address of the instruction code (**CS:IP**).

5. Data Registers

The 8086 has four *general-purpose data register*, which can be used as the source or destination of an operand during arithmetic and logic operations (see Fig 5).

Notice that they are referred to as the *accumulator register* (A), the *base register* (B), the *count register* (C), and the *data register* (D). Each one of these registers can be accessed either as a whole (16 bits) for word data operations or as two 8-bit registers for byte-wide data operations.



(a)

Register	Operations
AX	Word multiply, word divide, word I/O
AL	Byte multiply, byte divide, byte I/O, translate, decimal arithmetic
AH	Byte multiply, byte divide
BX	Translate
CX	String operations, loops
CL	Variable shift and rotate
DX	Word multiply, word divide, indirect I/O

(b)

Fig 7: (a) General purpose data Registers, (b) dedicated register functions

6. Pointer and Index Registers

The 8086 has four other general-purpose registers, two pointer registers **SP** and **BP**, and two index registers **DI** and **SI**. These are used to store what are called *offset addresses*.

An **offset address** represents the displacement of a storage location in memory from the segment base address in a segment register.

Unlike the general-purpose data registers, the pointer and index registers are only accessed as words (16 bits).

- The stack pointer (**SP**) and base pointer (**BP**) are used with the stack segment register (**SS**) to access memory locations within the stack segment.
- The source index (**SI**) and destination index (**DI**) are used with **DS** or **ES** to generate addresses for instructions that access data stored in the data segment of memory.

7. Status Register

The *status register* also called *flag register*: is 16-bit register with only nine bits that are implemented (see Fig 8). Six of these are **status flags**:

1. *The carry flag (CF)*: CF is set if there is a carry-out or a borrow-in for the most significant bit of the result during the execution of an instruction. Otherwise CF is reset.



2. *The parity flag (PF)*: PF is set if the result produced by the instruction has even parity- that is, if it contains an even number of bits at the 1 logic level. If parity is odd, PF is reset.
3. *The auxiliary flag (AF)*: AF is set if there is a carry-out from the low nibble into the high nibble or a borrow-in from the high nibble into the low nibble of the lower byte in a 16-bit word. Otherwise, AF is reset.
4. *The zero flag (ZF)*: ZF is set if the result produced by an instruction is zero. Otherwise, ZF is reset.
5. *The sign flag (SF)*: The MSB of the result is copied into SF. Thus, SF is set if the result is a negative number or reset if it is positive.
6. *The overflow flag (OF)*: When OF is set, it indicates that the signed result is out of range. If the result is not out of range, OF remains reset.

The other three implemented flag bits are called **control flags**:

1. *The trap flag (TF)*: if TF is set, the 8086 goes into the single-step mode of operation. When in the single-step mode, it executes an instruction and then jumps to a special service routine that may determine the effect of executing the instruction. This type of operation is very useful for debugging programs.
2. *The interrupt flag (IF)*: For the 8086 to recognize *maskable interrupt requests* at its interrupt (INT) input, the IF flag must be set. When IF is reset, requests at INT are ignored and the maskable interrupt interface is disabled.
3. *The direction flag (DF)*: The logic level of DF determines the direction in which string operations will occur. When set, the string instructions automatically decrement the address; therefore the string data transfers proceed from high address to low address.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF

Fig 8: Flag register

The 8086 provides instructions within its instruction set that are able to use status flags to alter the sequence in which the program is executed. Also it contains instructions for saving, loading, or manipulation flags.

8. Generating a memory address

- In 8086, **logical address** is described by combining two parts: **Segment address** and **offset**.
- **Segment address** is 16-bit data from one of the segment registers (CS, SS, DS and ES).



- **Offset address** is 16-bit data from one of the index and pointer registers (DI, SI, SP and BP). Also it could be base register BX.
- To express the 20-bit **Physical Address** of memory
 - 1 Multiply Segment register by **10H** (or shift it to left by four bit)
 - 2 Add it to the offset(see Fig 9)

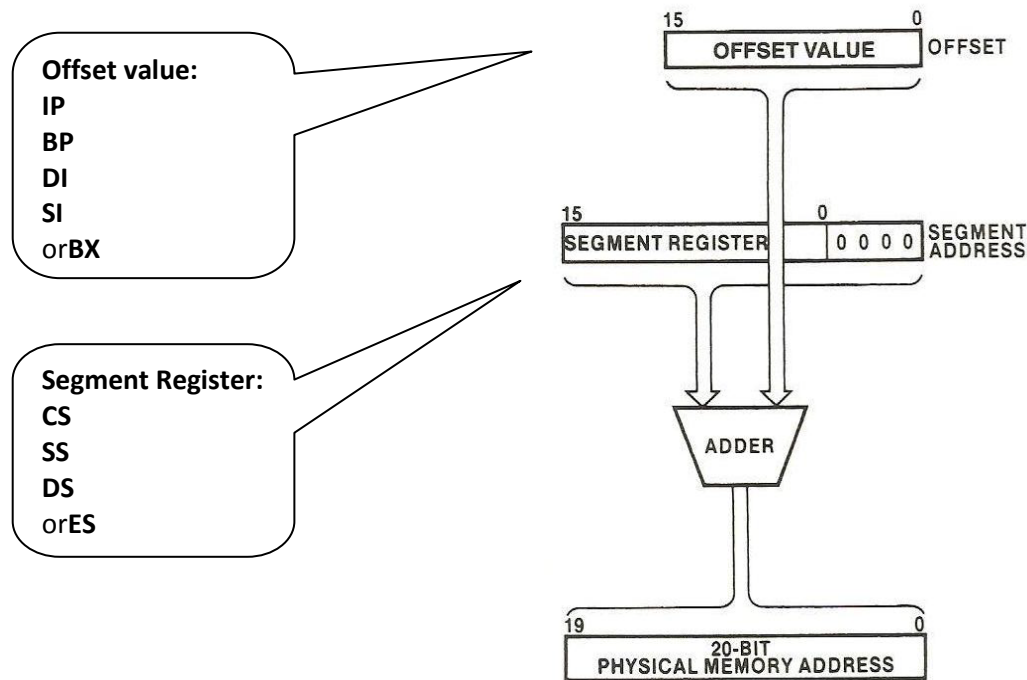


Fig 9: Generating a Memory Address

Example 3: if **CS** = 002AH, and **IP** = 0023H, write the **logical address** that they represent, then map it to **Physical address**.

Solution:

Logical address = **CS:IP**
002A : 0023

Physical address = (**CS X 10H**) + **IP** = 002A0 + 0023 = 002C3

Example 4: if **CS** = 002BH, and **IP** = 0013H, write the **logical address** that they represent, then map it to **Physical address**.

Solution:

Logical address = **CS:IP**
002B : 0013

Physical address = (**CS X 10H**) + **IP** = 002B0 + 0013 = 002C3

Physical addresses are identical here !

Actually, many different logical addresses map to the same physical address location in memory.



9. The stack

The stack is implemented in the memory and it is used for temporary storage of information such as data and addresses. The stack is 64Kbytes long and is organized from a software point of view as 32Kwords (see **Fig 10**).

- SS register points to the lowest address word in the stack
- SP and BP points to the address within stack
- Data transferred to and from the stack are **word-wide**, not **byte-wide**.
- The first address in the Stack segment (SS : 0000) is called *End of Stack*.
- The last address in the Stack segment (SS : FFFE) is called *Bottom of Stack*.
- The address (SS:SP) is called *Top of Stack*.
- POP instruction is used to read **word** from the stack.
- PUSH instruction is used to write **word** to the stack.
- When a word is to be pushed onto the top of the stack:
 - the value of SP is first automatically decremented by two
 - and then the contents of the register written into the stack.
- When a word is to be popped from the top of the stack the
 - the contents are first moved out the stack to the specific register
 - then the value of SP is first automatically incremented by two.

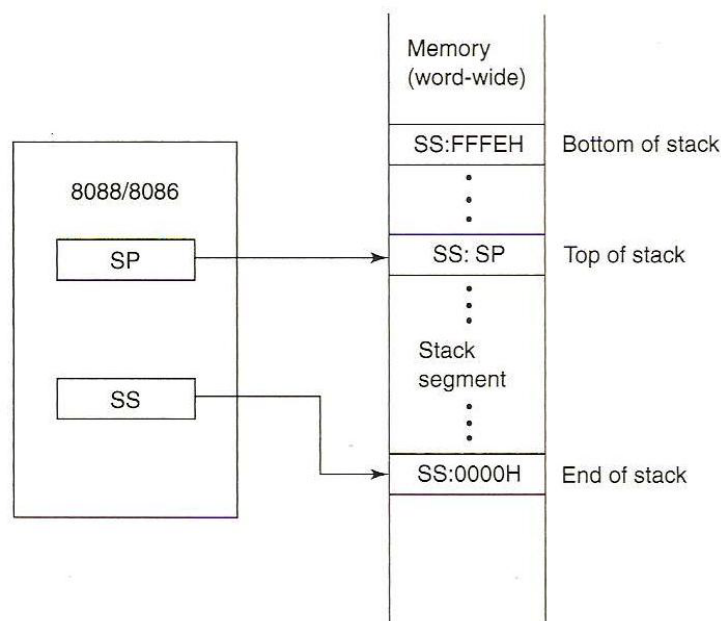


Fig 10: Stack segment of memory

Example 5: let **AX**=1234H ,**SS**=0105H and **SP**=0006H. Fig 11 shows the state of stack prior and after the execution of next program instructions:

```
PUSH AX
POP BX
POP AX
```

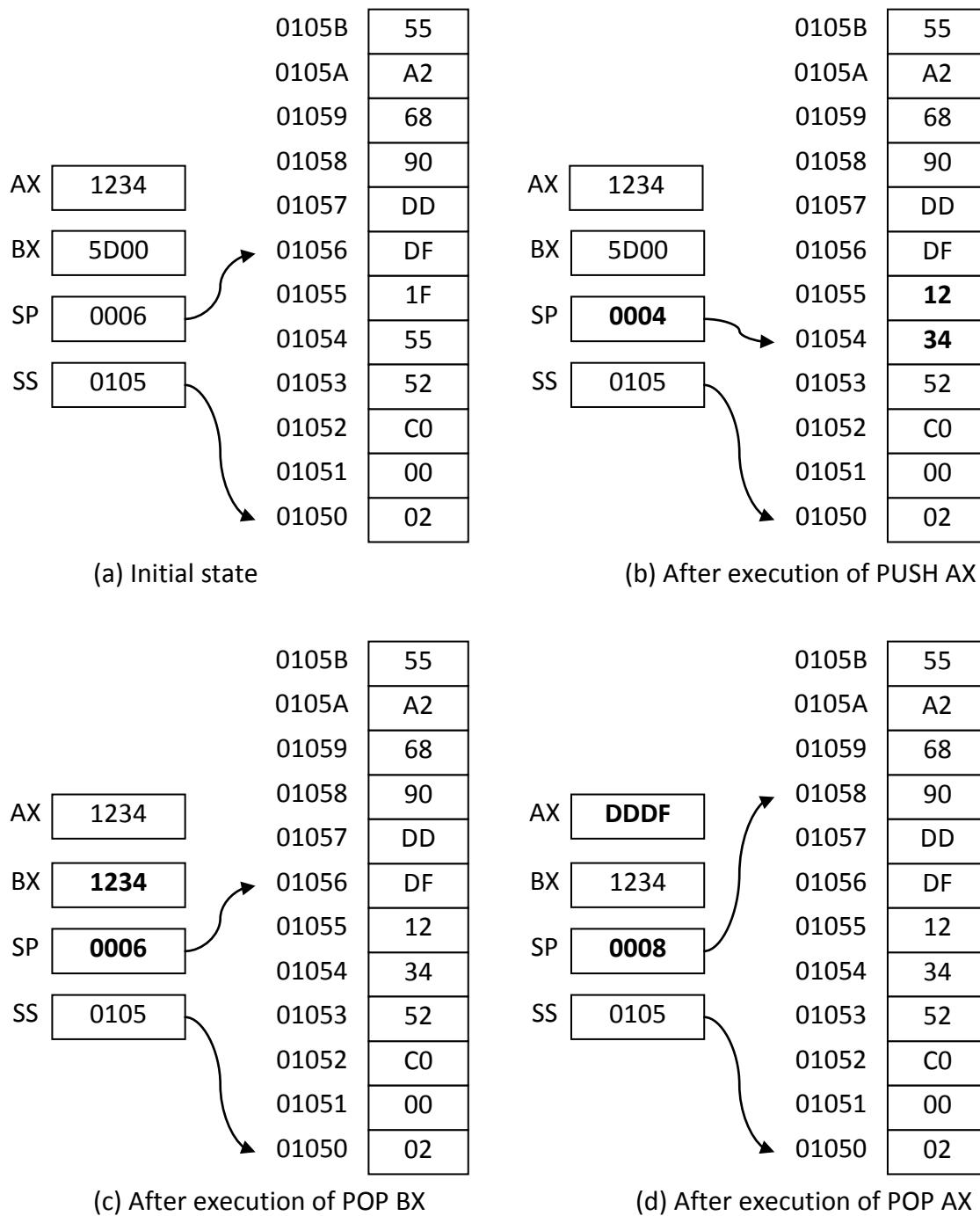


Fig 11 PUSH and POP instruction

10. Input and Output address space

The 8086 has separate memory and input/output (I/O) address spaces. The *I/O address space* is the place where I/O interfaces, such as printer and monitor ports, are implemented. Notice that this address range is from 0000H to FFFFH. This represents just 64Kbyte addresses; therefore only 16 bits of address are needed to address I/O space.



Problems

1. What are the length of the 8086's address bus and data bus?
2. How large is the instruction queue of the 8086?
3. List the elements of the execution unit.
4. What is the maximum amount of memory that can be active at a given time in the 8086?
5. Which part of the 8086's memory address space can be used to store the instruction of a program?
6. Name two dedicated operations assigned to the CX register.
7. Calculate the value of each of the physical addresses that follows. Assume all numbers are hexadecimal numbers.
 - a) $A000 : ? = A0123$
 - b) $? : 14DA = 235DA$
 - c) $D765 : ? = DABC0$
 - d) $? : CD21 = 32D21$
8. If the current values in the code segment register and the instruction pointer are 0200_{16} AND $01AC_{16}$, respectively, what physical address is used in the next instruction fetch?
9. If the current values in the stack segment register and stack pointer are $C000_{16}$ and $FF00_{16}$, respectively, what is the address of the current top of the stack?



Lecture 3- Addressing MODES

1. Introduction to assembly language programming

- Program is a sequence of commands used to tell a microcomputer what to do.
- Each command in a program is an instruction
- Programs must always be coded in machine language before they can be executed by the microprocessor.
- A program written in machine language is often referred to as *machine code*.
- Machine code is encoded using **0s** and **1s**
- A single machine language instruction can take up one or more bytes of code
- In assembly language, each instruction is described with alphanumeric symbols instead of with **0s** and **1s**
- Instruction can be divided into two parts : its *opcode* and *operands*
- Opcode identify the operation that is to be performed.
- Each opcode is assigned a unique letter combination called a *mnemonic*.
- Operands describe the data that are to be processed as the microprocessor carried out the operation specified by the opcode.
- Instruction set includes
 1. *Data transfer instructions*
 2. *Arithmetic instructions*
 3. *Logic instructions*
 4. *String manipulation instructions*
 5. *control transfer instructions*
 6. *Processor control instructions.*
- As an example for instructions, next section discusses the **MOV** instruction.

2. The MOV instruction

- The move instruction is one of the instructions in the data transfer group of the 8086 instruction set.
- Execution of this instruction transfers a byte or a word of data from a source location to a destination location. Fig 1 shows the general format of MOV instruction and the valid source and destination variations.

					Destination	Source
Mnemonic	Meaning	Format	Operation	Flags affected	Memory	Accumulator
MOV	Move	MOV D,S	(S) → (D)	None	Accumulator	Memory
					Register	Register
					Register	Memory
					Memory	Register
					Register	Immediate
					Memory	Immediate
					Seg-reg	Reg16
					Seg-reg	Mem16
					Reg16	Seg-reg
					Memory	Seg-reg

Fig 1 The MOV instruction and the valid source and destination variations



3. Addressing modes

An addressing mode is a method of specifying an operand. The 8086 addressing modes categorized into three types:

3.1 Register operand addressing mode

With register operand addressing mode, the operand to be accessed is specified as residing in an internal register. Fig 2 below shows the memory and registers before and after the execution of instruction:

MOV AX, BX

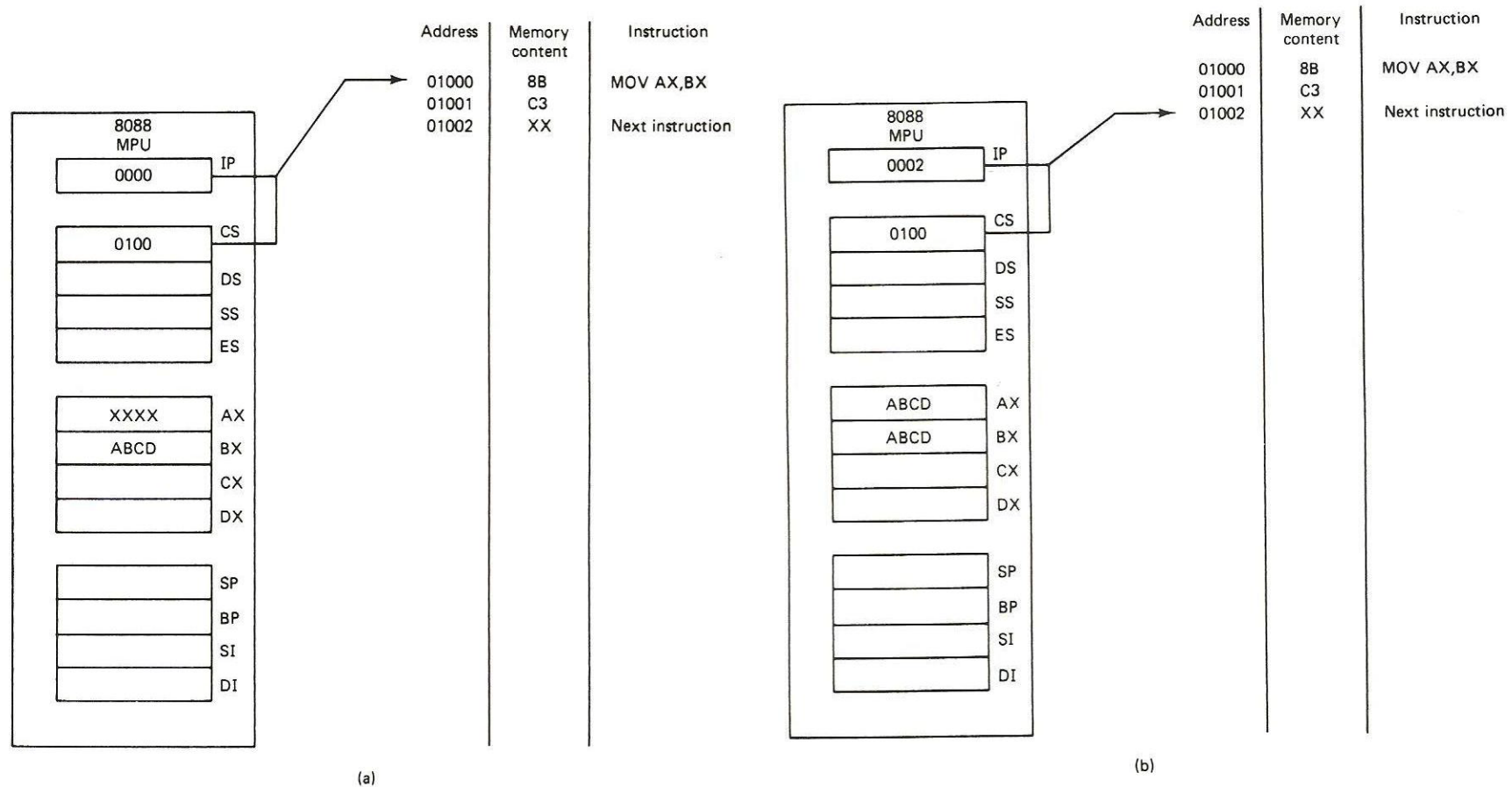


Fig 2 (a) before fetching and execution (b) after execution



3.2 Immediate operand addressing mode

With Immediate operand addressing mode, the operand is part of the instruction instead of the contents of a register or a memory location. Fig 3 below shows the memory and registers before and after the execution of instruction:

MOV AL, 15H

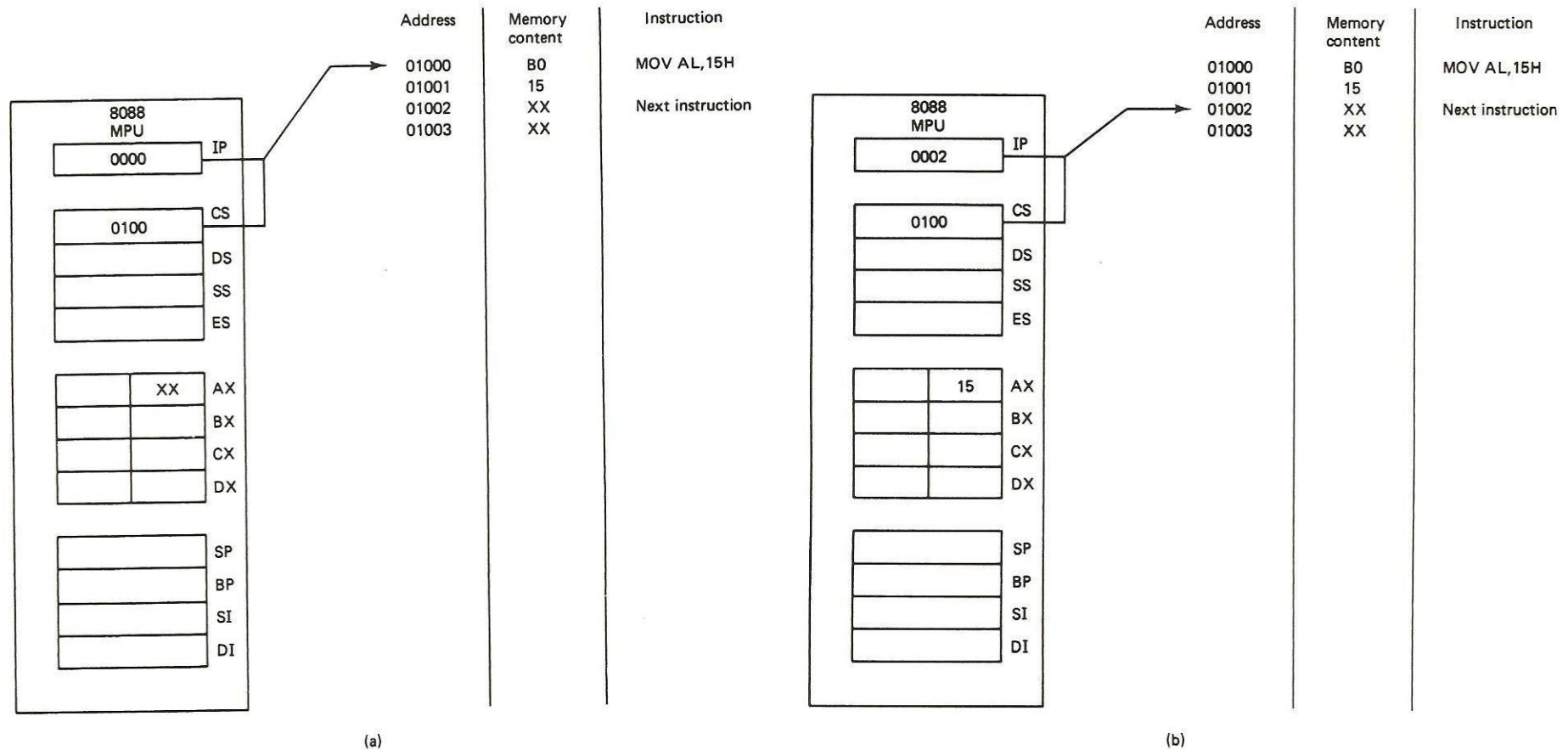


Fig 3(a) before fetching and execution (b) after execution



3.3 Memory Operand addressing modes: the 8086 use this mode to reference an operand in memory. The 8086 must calculate the physical address of the operand and then initiate a read or write operation of this storage location. The physical address of the operand is calculated from a *segment base address (SBA)* and an *effective address (EA)*. This mode includes five types:

3.3.1 Direct addressing: the value of the effective address is encoded directly in the instruction. Fig 4 below shows the memory and registers before and after the execution of instruction:

`MOV CX, [1234H]`

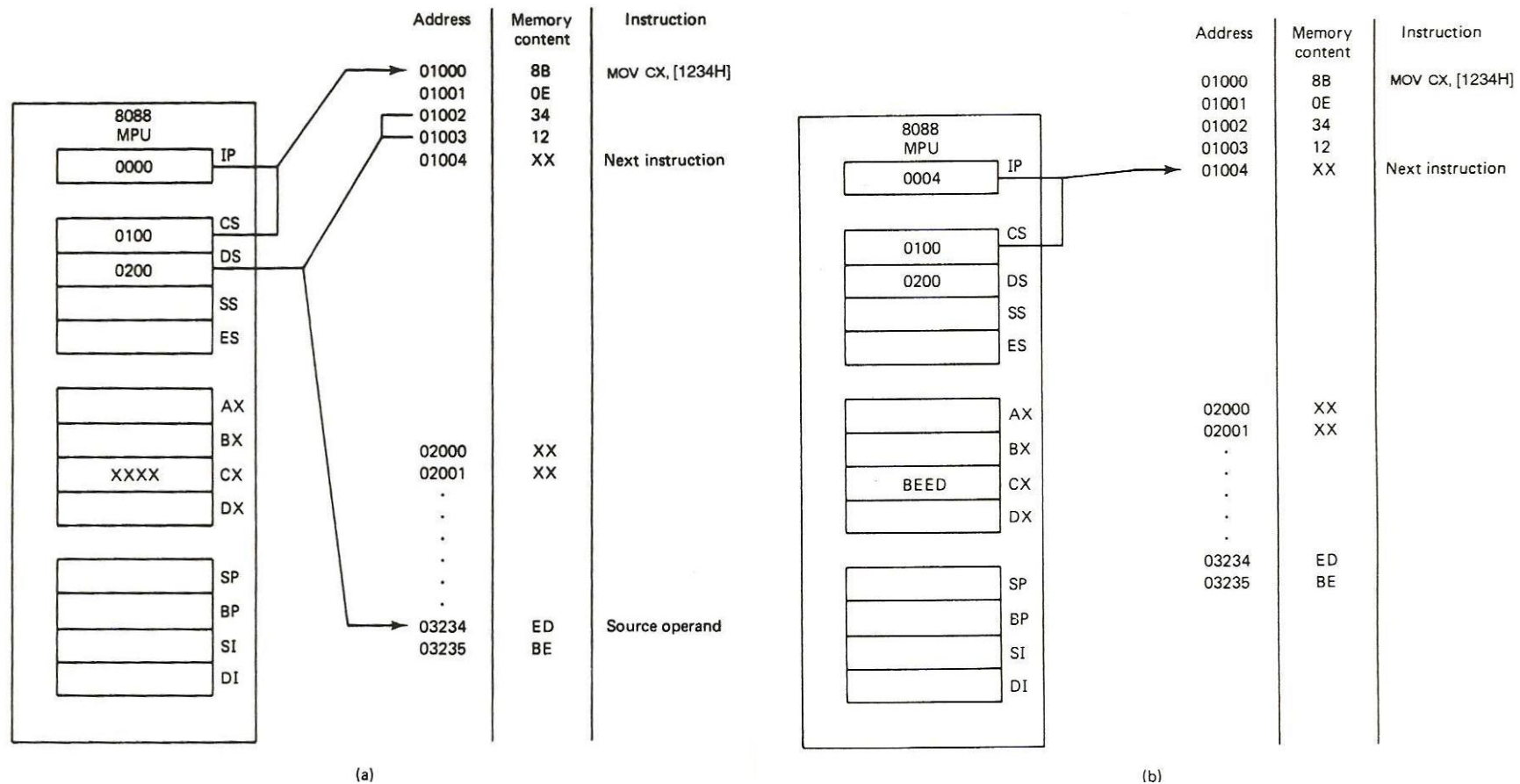


Fig 4 (a) before fetching and execution (b) after execution



3.3.2 Register indirect addressing: this mode is similar to the direct addressing but the offset is specified in a base register (BX), base pointer (BP) or an index register (SI or DI) within the 8086. Fig 5 below shows the memory and registers before and after the execution of instruction:

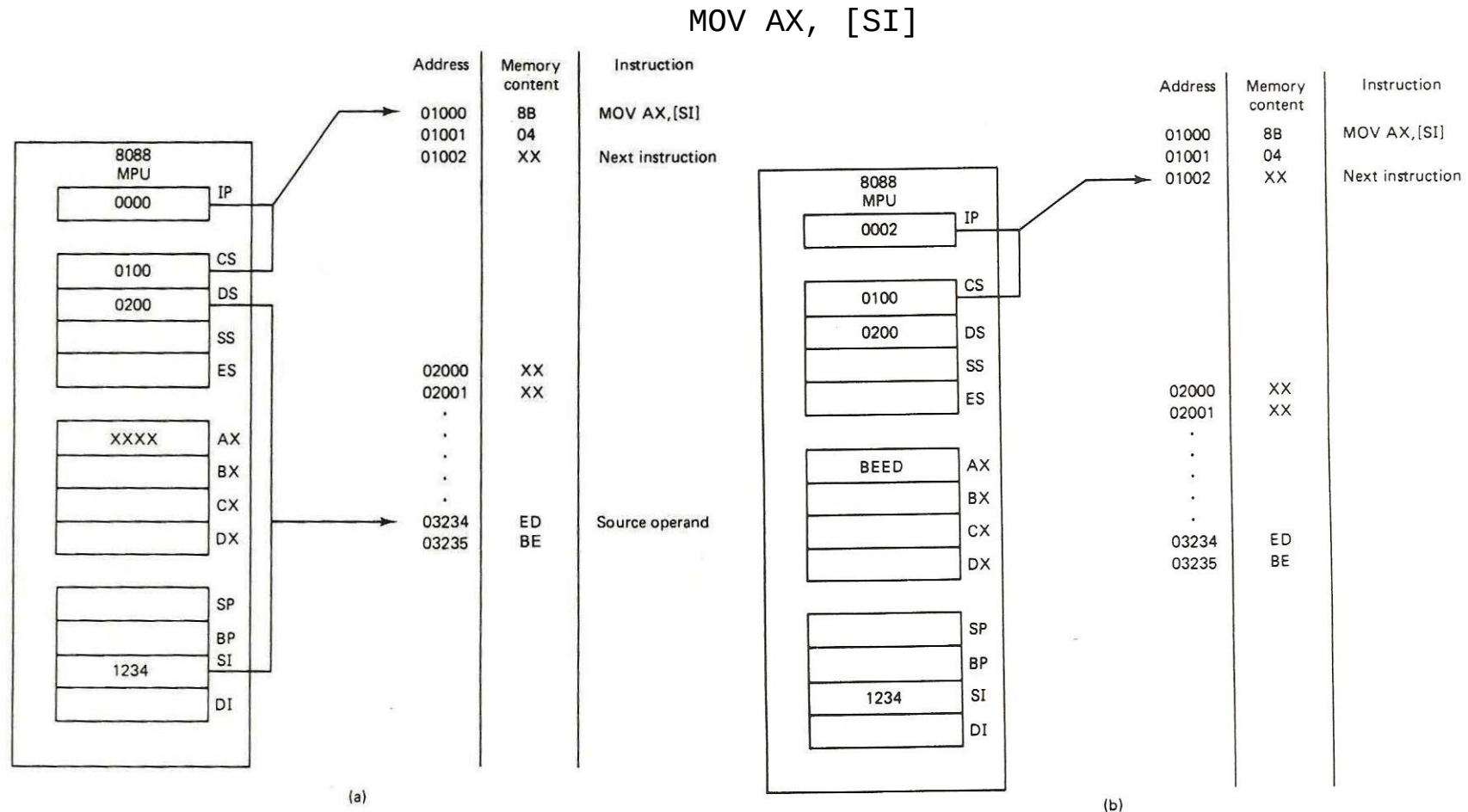


Fig 5(a) before fetching and execution (b) after execution



3.3.3 Based addressing: this mode, the effective address is obtained by adding a direct or indirect displacement to the contents of either base register **BX** or Base pointer register **BP**. Fig 6 below shows the memory and registers before and after the execution of instruction:

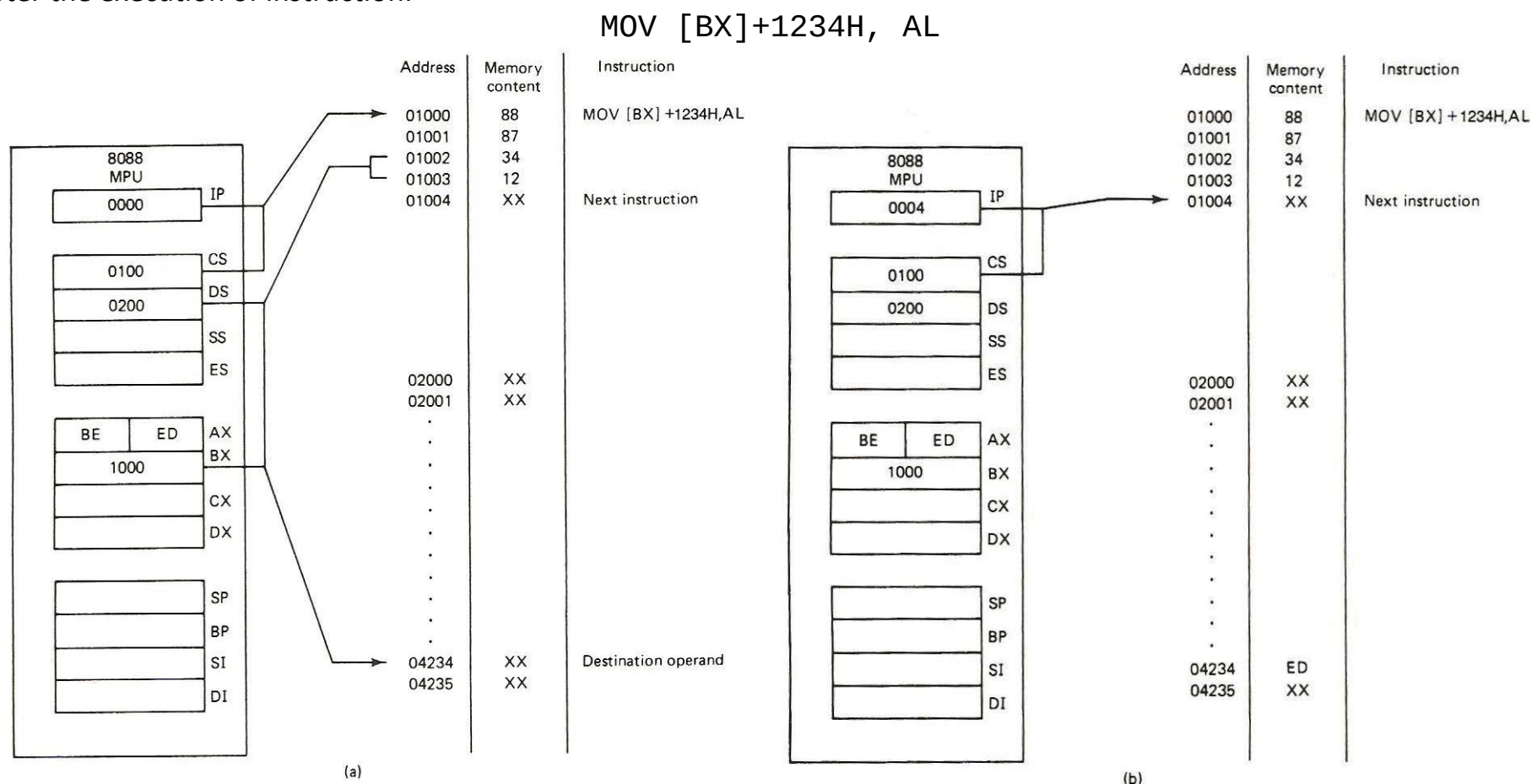


Fig 6(a) before fetching and execution (b) after execution

Note that if **BP** is used instead of **BX**, the calculation of the physical address is performed using the contents of the stack segment (**SS**) register instead of **DS**.

Note that The displacement could be **8 bits** or **16 bits**



3.3.4 Indexed addressing: this mode, work in similar manner to that of the based addressing mode but the effective address is obtained by adding the displacement to the value in an index register (**SI** or **DI**). Fig 7 below shows the memory and registers before and after the execution of instruction:

MOV AL, [SI]+1234H

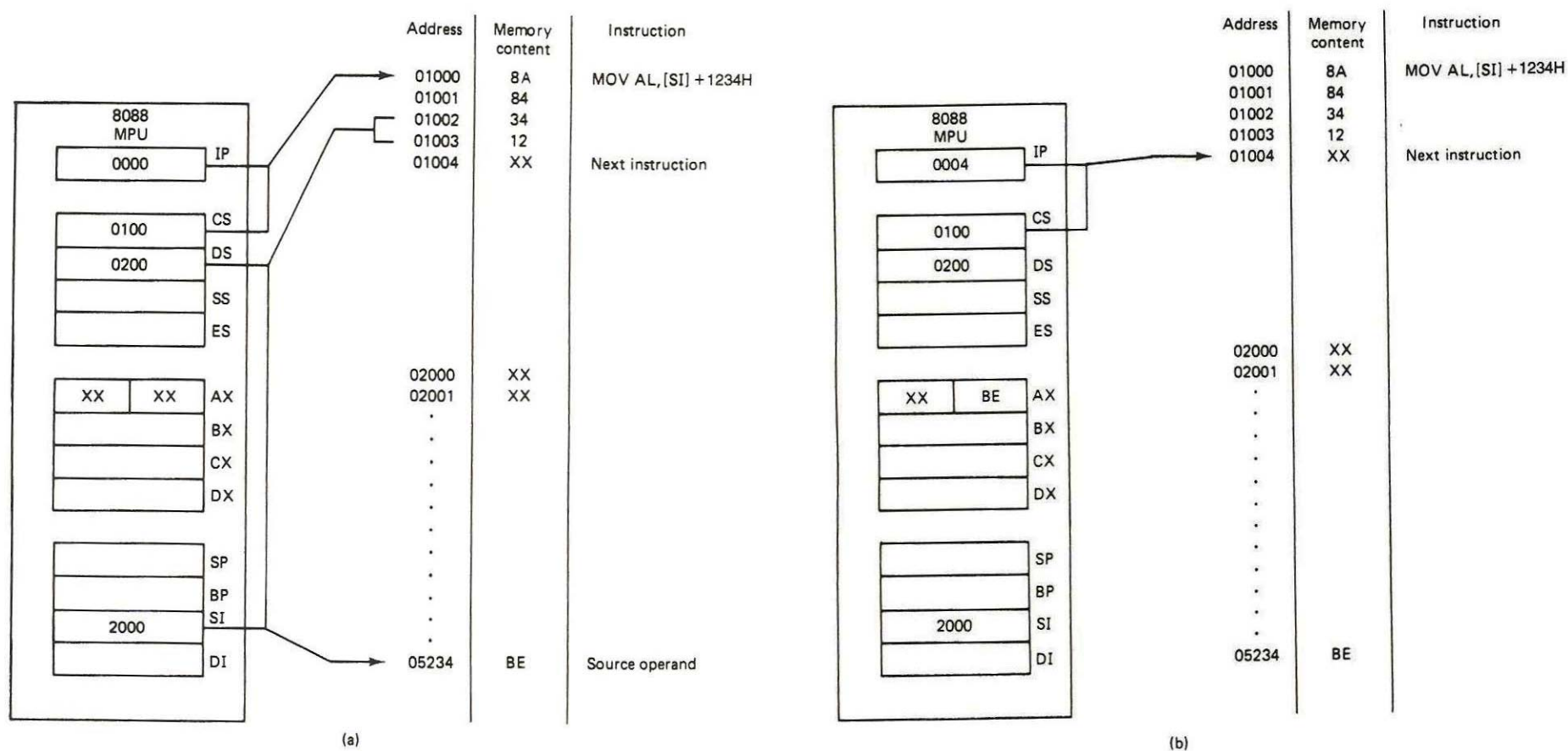


Fig 7(a) before fetching and execution (b) after execution

Note that The displacement could be 8 bits or 16 bits



3.3.5 Based-Indexed addressing: this mode combines the based addressing mode and indexed addressing mode. Fig 8 below shows the memory and registers before and after the execution of instruction:

MOV AH, [BX][SI]+1234H

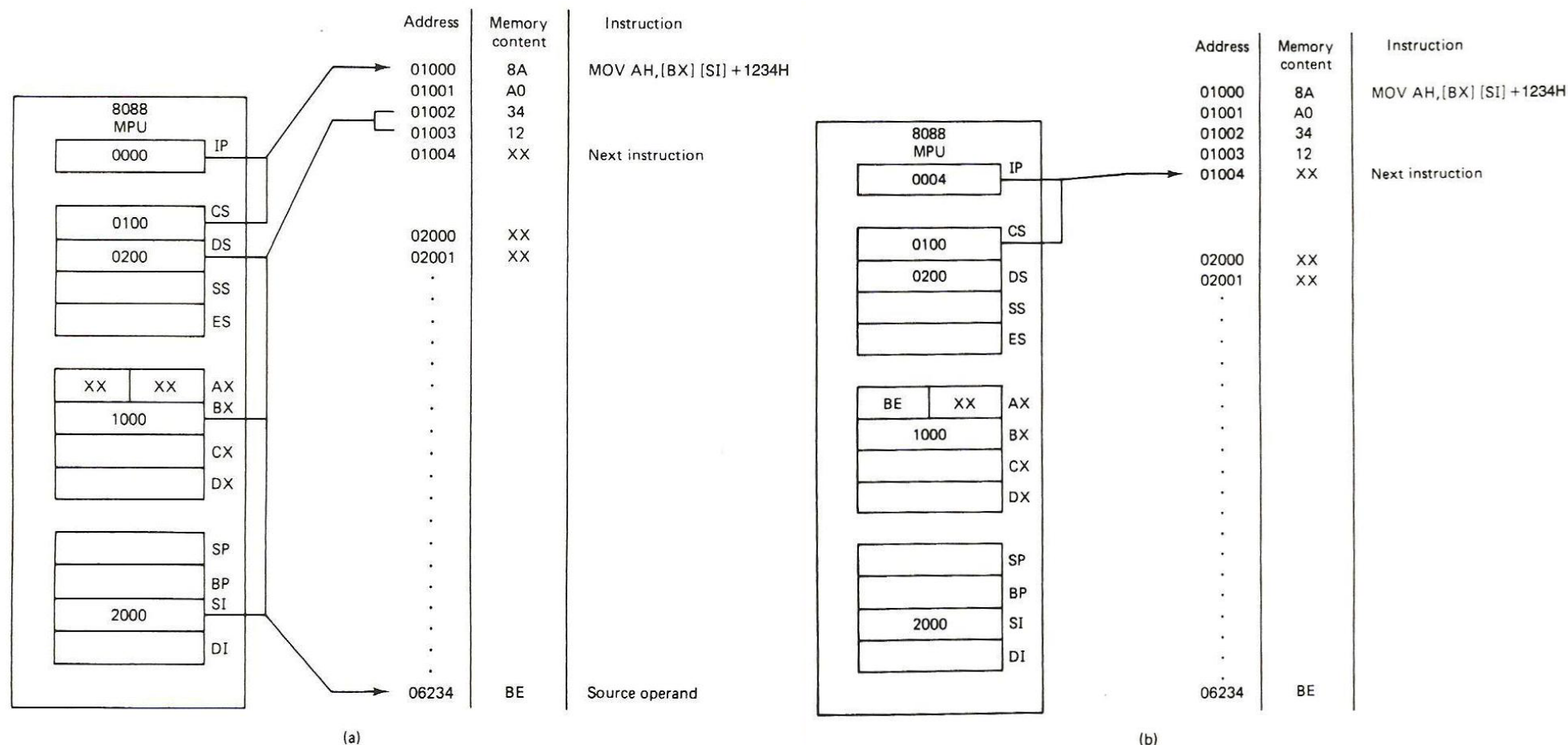


Fig 8 (a) before fetching and execution (b) after execution

Note that if **BP** is used instead of **BX**, the calculation of the physical address is performed using the contents of the stack segment (**SS**) register instead of **DS**.

Note that The displacement could be **8 bits** or **16 bits**



lecture 4- 8086 programming-Integer instructions and computations

- Objective:**
1. Data transfer instructions
 2. Arithmetic instructions
 3. Logic instructions
 4. Shift instructions
 5. Rotate instructions

1. Data transfer instructions

(a) MOV instruction

(b) XCHG Instruct

Mnemonic	Meaning	Format	Operation	Flags affected
XCHG	Exchange	XCHG D,S	(D) ↔ (S)	None

(a)

Destination	Source
Accumulator	Reg16
Memory	Register
Register	Register
Register	Memory

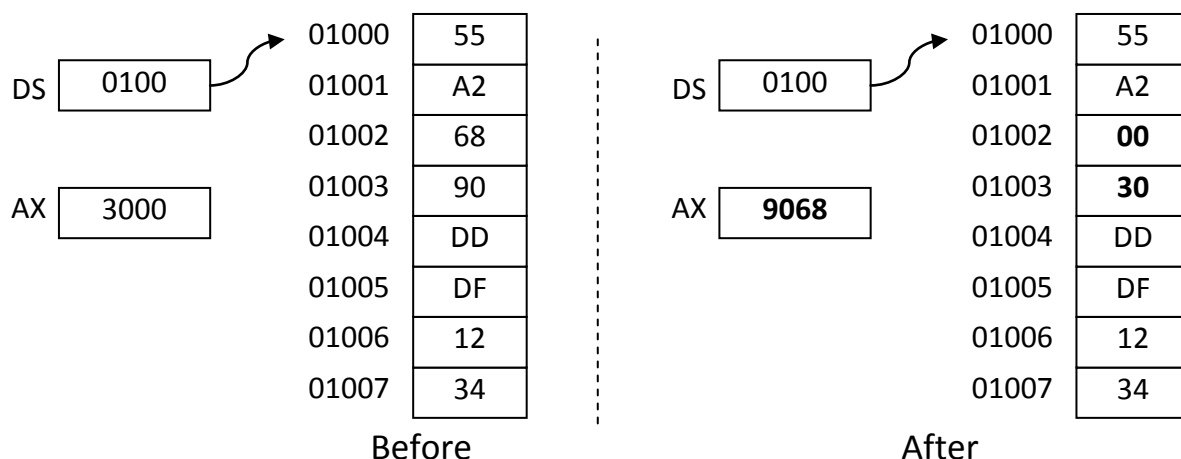
(b)

Fig 1 (a) XCHG data transfer instruction (b) Allowed operands

Example 1: For the figure below. What is the result of executing the following instruction?

XCHG AX , [0002]

Solution:





(c) XLAT

Mnemonic	Meaning	Format	Operation	Flags affected
XLAT	Translate	XLAT	$((AL) + (BX) + (DS) * 10) \rightarrow AL$	none

Fig 2 (a) XLAT data transfer instruction

Example 2: For the figure below, what is the result of executing the following instruction?

XLAT

Solution:

DS	0100	01040	55	DS	0100	01040	55
		01041	A2			01041	A2
		01042	68			01042	68
		01043	90			01043	90
		01044	DD			01044	DD
		01045	DF			01045	DF
		01046	12			01046	12
		01047	34			01047	34
Before				After			

(d) LEA, LDS, and LES instructions

Mnemonic	Meaning	Format	Operation	Flags affected
LEA	Load effective address	LEA Reg16,EA	EA $\rightarrow$ (Reg16)	None
LDS	Load register and DS	LDS Reg16,EA	EA $\rightarrow$ (Reg16) EA+2 $\rightarrow$ (DS)	None
LES	Load register and ES	LES Reg16,EA	EA $\rightarrow$ (Reg16) EA+2 $\rightarrow$ (ES)	None

Fig 3 (a) LEA, LDS and LES data transfer instruction

Example 3: For the figure below, what is the result of executing the following instruction?

LEA SI , [DI + BX + 2H]

Solution:

$$SI = (DI) + (BX) + 2H = 0062H$$

DS	0100		DS	0100
SI	F002		SI	0062
DI	0020		DI	0020
AX	0003		AX	0003
BX	0040		BX	0040
Before			After	



For these three instructions (LEA, LDS and LES) the effective address could be formed of **all** or any various combinations of the three elements in Fig 4

$$EA = \left\{ \begin{matrix} BX \\ BP \end{matrix} \right\} + \left\{ \begin{matrix} DI \\ SI \end{matrix} \right\} + \left\{ \begin{matrix} 8 - \text{bit displacement} \\ 16 - \text{bit displacement} \end{matrix} \right\}$$

Fig 4 The three element used to compute an effective address

Example 4: For the figure below, what is the result of executing the following instruction?

LEASI , [DI + BX + 2H]

Solution:

$$SI = (DI) + (BX) + 2H = 0062H$$

DS	0100	01040	55	DS	0100	01040	55
		01041	A2			01041	A2
		01042	68			01042	68
		01043	90			01043	90
		01044	DD			01044	DD
		01045	DF			01045	DF
		01046	12			01046	12
SI	F002	01047	34	SI	0062	01047	34
DI	0020			DI	0020		
AX	0003			AX	0003		
BX	0040			BX	0040		

Before

After

Example 5 :

Instruction Sample		Result
LEA SI , [BX + SI + 55]	Valid	SI= BX + SI + 55
LEA SI , [BX + SI]	Valid	SI= BX + SI
LEA BP , [890C]	valid	BP= 890C
LEA AX , [BX + SI + 20]	Valid	AX = BX + SI + 20
LEA DI , [BP + DI + 55]	Valid	DI = BP + DI + 55
LEA DI , [DI + DI + 55]	Not valid because EA doesn't involve DI twice	
LEA CS , [BP + DI + 55]	Not valid because destination cant be segment register	
LEA IP , [BP + 550C]	Not valid because destination cant be instruction pointer	
LEA AX , [CX + DI + 1D]	Not valid because EA doesn't involve CX	
LEA AL , [DI + 103D]	Not valid because destination must be 16 bit	

Example 6: What is the result after executing each one of the next instructions?

LEA BP, [F004]
MOV BP, F004
MOV BP, [F004]



Solution:

Instruction	Result
LEA BP, [F004]	The value F004 will be assigned to the Base Pointer
MOV BP, F004	The value F004 will be assigned to the Base Pointer
MOV BP, [F004]	The word at memory locations F004 and F005 (in the current Data Segment) will be assigned to Base Pointer

The instruction LES is similar to the instruction LDS except that it load the **Extra Segment Register** instead of **Data Segment Register**

2. Arithmetic instruction

- The 8086 microprocessor can perform addition operation between any two registers except segment register (**CS**, **DS**, **ES**, and **SS**) and instruction pointer (**IP**).
- Addition must occur between similar sizes

ADD AL, BL	Valid
ADD BX, SI	Valid
ADD BX, CL	Not Valid (different sizes)

- Addition can occur between register and memory

Example 7: For the figure below,

- ❖ What is the result of executing the following instruction?
 - ❖ What is the addressing mode for this instruction?
 - ❖ What is the PA if BP register used instead of BX register?
- ADDAX , [DI + BX + 2H]

Solution:

$$EA = [DI + BX + 2H] = [0020 + 0040 + 02H] = 0062H$$

$$PA = (DS \times 10H) + EA = 1000H + 0062H = 1062H$$

Memory word stored at location 1062H is 9067

$$AX = AX + 9067$$

DS	0100	01060	55	DS	0100	01060	55
		01061	A2			01061	A2
SS	0200	01062	67	SS	0200	01062	67
		01063	90			01063	90
DI	0020	01064	DD	DI	0020	01064	DD
AX	0003	01065	DF	AX	906A	01065	DF
BX	0040	01066	12	BX	0040	01066	12
BP	0040	01067	34	BP	0040	01067	34
Before				After			

- ❖ The addressing mode for this instruction is **Based Indexed** mode.
- ❖ If BP used in the EA, then $PA = (SS \times 10H) + 0062 = 2000H + 0062H = 2062H$



Lecture 5

8086 programming - Integer instructions and computations (continue)

Mnemonic	Meaning	Format	Operation	Flags Affected
ADD	Addition	ADD D, S	$(S) + (D) \rightarrow (D)$ Carry $\rightarrow (CF)$	OF, SF, ZF, AF, PF, CF
ADC	Add with carry	ADC D, S	$(S) + (D) + (CF) \rightarrow (D)$ Carry $\rightarrow (CF)$	OF, SF, ZF, AF, PF, CF
INC	Increment by 1	INC D	$(D) + 1 \rightarrow (D)$	OF, SF, ZF, AF, PF
AAA	ASCII adjust for addition	AAA		AF, CF OF, SF, ZF, PF undefined
DAA	Decimal adjust for addition	DAA		SF, ZF, AF, PF, CF, OF, undefined

(a)

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

(b)

Destination
Reg16
Reg8
Memory

(c)

(a) Addition instructions (b) Allowed operands for ADD and ADC.

(c) Allowed operands for INC instruction

- The instruction *add with carry*(ADC) work similarly to ADD, but in this case the content of the carry flag is also added, that is
- $(S) + (D) + (CF) \rightarrow (D)$
- ADC is primarily used for multiword add operation.

Example 8: let *num1*=11223344H and *num2*=55667788H are stored at memory locations 200 and 300 respectively in the current data segment. ADD *num1* and *num2* and store the result at memory location 400.

Solution:

```
MOV AX, [0200]
MOV BX, [0202]
ADD AX, [0300]
ADC BX, [0302]
MOV [0400], AX
MOV [0402], BX
```



- **INC** instruction add 1 to the specified operand
- Note that the **INC** instruction don't affect the carry flag

Example 9: For the figure below, what is the result of executing the following instructions?

```
INC WORD PTR [0040]
INC BYTE PTR [0042]
```

Solution:

$$SI = (DI) + (BX) + 2H = 0062H$$

DS	0100	CF	X	Before	01040	FF	After	01040	00
					01041	03		01041	04
					01042	FF		01042	00
					01043	03		01043	03
					01044	DD		01044	DD
					01045	DF		01045	DF
					01046	12		01046	12
					01047	34		01047	34

- **AAA** instruction specifically used to adjust the result after the operation of addition two binary numbers which represented in ASCII.
- **AAA** instruction should be executed immediately after the **ADD** instruction that adds ASCII data.
- Since **AAA** can adjust only data that are in **AL**, the destination register for **ADD** instructions that process ASCII numbers should be **AL**.

Example 10: what is the result of executing the following instruction sequence?

```
ADD AL, BL
AAA
```

Assume that **AL** contains 32H (the ASCII code for number 2), **BL** contain 34H (the ASCII code for number 4), and **AH** has been cleared.

Solution :



AL 32	AL 66	AL 06
BL 34	BL 34	BL 34
CF X	CF 0	CF 0
Before	After ADD instruction	After AAA instruction

- **DAA** instruction used to perform an adjust operation similar to that performed by **AAA** but for the addition of packed BCD numbers instead of ASCII numbers.
- Since **DAA** can adjust only data that are in **AL**, the destination register for ADD instructions that process BCD numbers should be **AL**.
- DAA must be invoked after the addition of two *packed BCD numbers*.

Example 11: what is the result of executing the following instruction sequence?

ADD AL , BL
DAA

Assume that AL contains 29H (the BCD code for decimal number 29), BL contain 13H (the BCD code for decimal number 13) , and AH has been cleared.

Solution :

AL 29	AL 3C	AL 42
BL 13	BL 13	BL 13
CF X	CF 0	CF 0
Before	After ADD instruction	After DAA instruction

- Subtraction subgroup of instruction set is similar to the addition subgroup.
- For subtraction the carry flag **CF** acts as borrow flag
- If borrow occur after subtraction then **CF = 1**.
- If NO borrow occur after subtraction then **CF = 0**.
- Subtraction subgroup content instruction shown in table below



Mnemonic	Meaning	Format	Operation	Flags affected
SUB	Subtract	SUB D,S	$(D) - (S) \rightarrow (D)$ Borrow $\rightarrow (CF)$	OF, SF, ZF, AF, PF, CF
SBB	Subtract with borrow	SBB D,S	$(D) - (S) - (CF) \rightarrow (D)$	OF, SF, ZF, AF, PF, CF
DEC	Decrement by 1	DEC D	$(D) - 1 \rightarrow (D)$	OF, SF, ZF, AF, PF
NEG	Negate	NEG D	$0 - (D) \rightarrow (D)$ $1 \rightarrow (CF)$	OF, SF, ZF, AF, PF, CF
DAS	Decimal adjust for subtraction	DAS		SF, ZF, AF, PF, CF OF undefined
AAS	ASCII adjust for subtraction	AAS		AF, CF OF, SF, ZF, PF undefined

(a)

Destination	Source
Register	Register
Register	Memory
Memory	Register
Accumulator	Immediate
Register	Immediate
Memory	Immediate

(b)

Destination
Reg16
Reg8
Memory

(c)

Destination
Register
Memory

(d)

(a) Subtraction instructions (b) Allowed operands for SUB and SBB.
(c) Allowed operands for INC instruction (d) Allowed operands for NEG instruction

- **SBB** is primarily used for multiword subtract operations.
- Another instruction called **NEG** is available in the subtraction subgroup
- The **NEG** instruction evaluates the 2's complement of an operand

Example 12: what is the result of executing the following instruction sequence?
NEG BX

Solution :

BX **0013**

CF **0**

Before

BX **FFED**

CF **1**

After



Multiplication and Division instructions:

Mnemonic	Meaning	Format	Operation	Flags Affected
MUL	Multiply (unsigned)	MUL S	$(AL) \cdot (S8) \rightarrow (AX)$ $(AX) \cdot (S16) \rightarrow (DX), (AX)$	OF, CF SF, ZF, AF, PF undefined
DIV	Division (unsigned)	DIV S	(1) $Q((AX)/(S8)) \rightarrow (AL)$ $R((AX)/(S8)) \rightarrow (AH)$ (2) $Q((DX,AX)/(S16)) \rightarrow (AX)$ $R((DX,AX)/(S16)) \rightarrow (DX)$ If Q is FF_{16} in case (1) or $FFFF_{16}$ in case (2), then type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined
IMUL	Integer multiply (signed)	IMUL S	$(AL) \cdot (S8) \rightarrow (AX)$ $(AX) \cdot (S16) \rightarrow (DX), (AX)$	OF, CF SF, ZF, AF, PF undefined
IDIV	Integer divide (signed)	IDIV S	(1) $Q((AX)/(S8)) \rightarrow (AL)$ $R((AX)/(S8)) \rightarrow (AH)$ (2) $Q((DX,AX)/(S16)) \rightarrow (AX)$ $R((DX,AX)/(S16)) \rightarrow (DX)$ If Q is positive and exceeds $7FFF_{16}$ or if Q is negative and becomes less than 8001_{16} , then type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined
AAM	Adjust AL for multiplication	AAM	$Q((AL)/10) \rightarrow (AH)$ $R((AL)/10) \rightarrow (AL)$	SF, ZF, PF OF, AF, CF undefined
AAD	Adjust AX for division	AAD	$(AH) \cdot 10 + (AL) \rightarrow (AL)$ $00 \rightarrow (AH)$	SF, ZF, PF OF, AF, CF undefined
CBW	Convert byte to word	CBW	$(MSB \text{ of } AL) \rightarrow (\text{All bits of } AH)$	None
CWD	Convert word to double word	CWD	$(MSB \text{ of } AX) \rightarrow (\text{All bits of } DX)$	None

(a)

Source
Reg8
Reg16
Mem8
Mem16

(b)

(a) Multiplication and division arithmetic instructions (b) Allowed operands.



- **MUL** instruction used to multiply unsigned number in **AL** with an 8 bit operand (in register or memory) and store the result in **AX**
- **MUL** instruction used to multiply unsigned number in **AX** with an 16 bit operand (in register or memory) and store the result in **DX** and **AX**
- Note that the multiplication of two 8-bit number is 16-bit number
- Note that the multiplication of two 16-bit number is 32-bit number
- **IMUL** is similar to **MUL** but is used for signed numbers
- Note that the destination operand for instructions **MUL** and **IMUL** is either **AX** or **both DX and AX**

Example 13: what is the result of executing the following instruction?

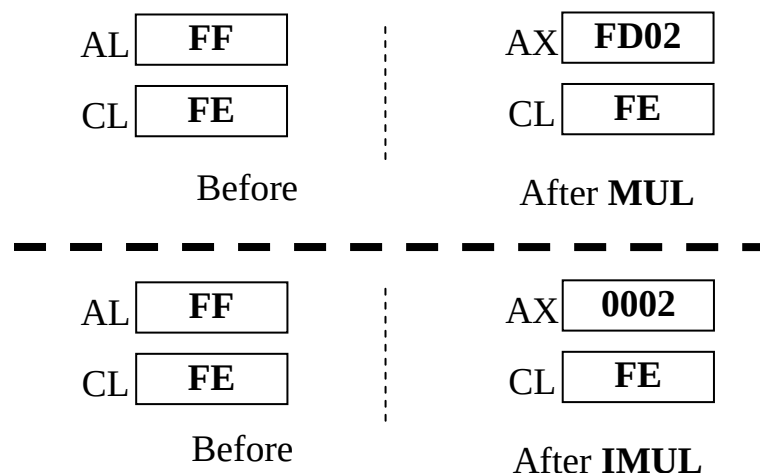
MUL CL

What is the result of executing the following instruction?

IMUL CL

Assume that AL contains FFH (the 2's complement of the number 1), CL contain FEH (the 2's complement of the number 2).

Solution :





lecture 6

8086 programming - Integer instructions and computations (continue)

Ex1: Assume that each instruction starts from these values:

AL = 85H, BL = 35H, AH = 0H

1. MUL BL = AL * BL = 85H * 35H = 1B89H → AX = 1B89H

2. IMUL BL = AL * BL = 2'SAL * BL = 2'S(85H) * 35H

= 7BH * 35H = 1977H → 2's comp → E689H → AX.

3. DIV BL = $\frac{AX}{BL} = \frac{0085H}{35H} =$

AH (remainder)	AL (quotient)
1B	02

4. IDIV BL = $\frac{AX}{BL} = \frac{0085H}{35H} =$

AH (remainder)	AL (quotient)
1B	02

Example: Assume that each instruction starts from these values:

AL = F3H, BL = 91H, AH = 00H

1. MUL BL = AL * BL = F3H * 91H = 89A3H → AX = 89A3H

2. IMUL BL = AL * BL = 2'SAL * 2'SBL = 2'S(F3H) * 2'S(91H)
= 0DH * 6FH = 05A3H → AX.

3. IDIV BL = $\frac{AX}{BL} = \frac{00F3H}{2'(91H)} = \frac{00F3H}{6FH} = 2$ quotient and 15H remainder:

AH (remainder)	AL (quotient)
15	02

→, but $\frac{\text{Positive}}{\text{negative}} = \text{negative}$, so

AH (remainder)	AL (quotient)
15	2'comp(02)

→

AH (remainder)	AL (quotient)
15	FE

4. DIV BL $\frac{AX}{BL} = \frac{00F3H}{91H} = 01$

AH (remainder)	AL (quotient)
62	01



Example: Assume that each instruction starts from these values:

AX= F000H, BX= 9015H, DX= 0000H

1. MUL BX = F000H * 9015H =

DX	AX
8713	B000

2. IMUL BX = 2'S(F000H) * 2'S(9015H) = 1000 * 6FEB =

DX	AX
06FE	B000

3. DIV BL $\frac{AX}{BL} = \frac{F000H}{15H} = 0B6DH \rightarrow$ more than FFH $\rightarrow$ Divide Error

4. IDIV BL $\frac{AX}{BL} = \frac{2'(F000H)}{15H} = \frac{1000H}{15H} = C3H \rightarrow$ more than 7FH $\rightarrow$ Divide Error

Example : Assume that each instruction starts from these values:

AX= 1250H, BL= 90H

1. IDIV BL $\frac{AX}{BL} = \frac{1250H}{90H} = \frac{\text{positive}}{\text{negative}} = \frac{\text{positive}}{2'\text{negative}} = \frac{1250}{2'(90H)} = \frac{1250H}{70H}$
= 29H quotient and 60H remainder

But 29H(positive) $\rightarrow 2'S(29H) = D7H \rightarrow$

AH (Remainder)	AL (quotient)
60H	D7H

2. DIV $\frac{AX}{BL} = \frac{1250H}{90H} = 20H \rightarrow$

AH (Remainder)	AL (quotient)
50H	20H

To divide an 8-bit dividend by and 8-bit divisor by extending the sign bit of AL to fill all bits of AH. This can be done automatically by executing the instruction CBW).

In a similar way 16-bit dividend in AX can be divided by 16-bit divisor. In this case the sign bit in AX is extended to fill all bits of DX. The instruction CWD perform this operation automatically.

Note that CBW extend 8-bit in AL to 16-bit in AX while the value in AX will be equivalent to the value in AL. Similarly, CWD convert the value in AX to 32-bit in (DX,AX) without changing the original value.



3. Logical & Shift Instructions

- Logical instructions: The 8086 processor has instructions to perform bit by bit logic operation on the specified source and destination operands.
- Uses any addressing mode except **memory-to-memory** and **segment registers**

Mnemonic	Meaning	Format	Operation	Flags Affected
AND	Logical AND	AND D,S	$(S) \cdot (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined
OR	Logical Inclusive OR	OR D,S	$(S) + (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined
XOR	Logical Exclusive OR	XOR D,S	$(S) \oplus (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined
NOT	LOGICAL NOT	NOT D	$\overline{(D)} \rightarrow (D)$	None

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Allowed operands for AND,
OR, and XOR operations

Destination
Register
Memory

Allowed operand
for NOT Operation

AND

- used to clear certain bits in the operand(masking)

Example Clear the high nibble of BL register

AND BL, 0FH (xxxxxxxx**AND** 0000 1111 = 0000 xxxx)

Example Clear bit 5 of DH register

AND DH, DFH (xxxxxxxx**AND** 1101 1111 = xx0xxxxx)

OR

- Used to set certain bits

Example Set the lower three bits of BL register

OR BL, 07H (xxxxxxxx**OR** 0000 0111 = xxxx x111)

Example Set bit 7 of AX register

ORAH, 80H (xxxxxxxx**OR** 1000 0000 = 1xxxxxxx)



XOR

- Used to invert certain bits (toggling bits)
- Used to clear a register by XORed it with itself

Example Invert bit 2 of DL register

XOR BL, 04H (xxxxxxxx**OR** 0000 0100 = xxxxx x $\bar{x}$ xx)

Example Clear DX register

XOR DX, DX (DX will be 0000H)

Example

XOR AX , DL	not valid	size don't match
OR AX,DX	valid	
NOT CX , DX	not valid	Not instruction has one operand
AND WORD PTR [BX + DI + 5H], BX		valid
AND WORD PTR [BX +DI] , DS	not valid	source must not be segment register

4. Shift instruction

- The four shift instructions of the 8086 can perform two basic types of shift operations: the logical shift, the arithmetic shift
- Shift instructions are used to
 - o Align data
 - o Isolate bit of a byte of word so that it can be tested
 - o Perform simple multiply and divide computations
- The source can specified in two ways
 - Value of 1 : Shift by One bit
 - Value of CL register : Shift by the value of CL register

Note that the amount of shift specified in the source operand can be defined explicitly if it is **one bit** or should be stored in CL if **more than 1**.

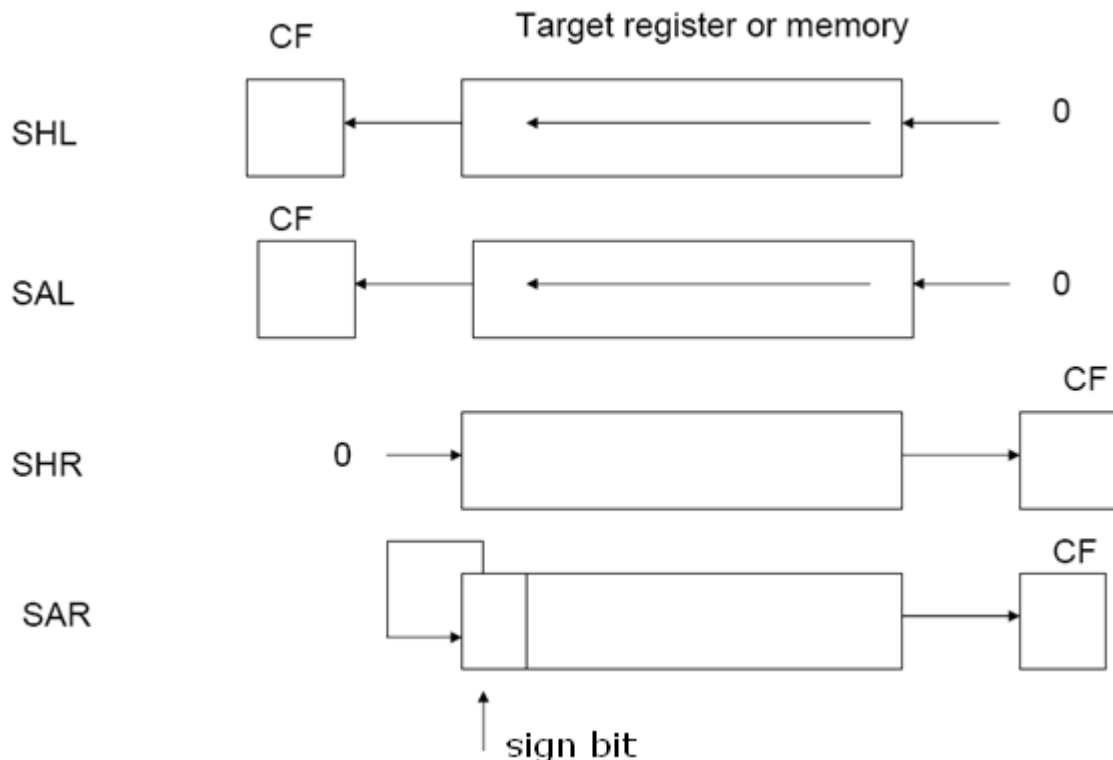
Mnemonic	Meaning	Format	Operation	Flags Affected
SAL/SHL	Shift arithmetic Left/shift Logical left	SAL/SHL D, Count	Shift the (D) left by the number of bit positions equal to count and fill the vacated bits positions on the right with zeros	CF,PF,SF,ZF AF undefined OF undefined if count $\neq 1$
SHR	Shift logical right	SHR D, Count	Shift the (D) right by the number of bit positions equal to count and fill the vacated bits positions on the left with zeros	CF,PF,SF,ZF AF undefined OF undefined if count $\neq 1$
SAR	Shift arithmetic right	SAR D, Count	Shift the (D) right by the number of bit positions equal to count and fill the vacated bits positions on the left with the original most significant bit	CF,PF,SF,ZF AF undefined OF undefined if count $\neq 1$



Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Allowed operands

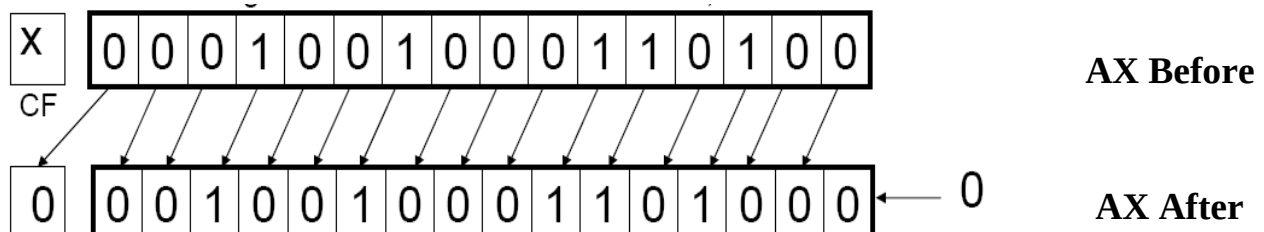
- The SHL and SAL are identical: they shift the operand to left and fill the vacated bits to the right with **zeros**.
- The SHR instruction shifts the operand to right and fill the vacated bits to the left with **zeros**.
- The SAR instruction shifts the operand to right and fill the vacated bits to the left with the value of MSB (this operation used to shift the signed numbers)



Example let AX=1234H what is the value of AX after execution of next instruction

SHL AX,1

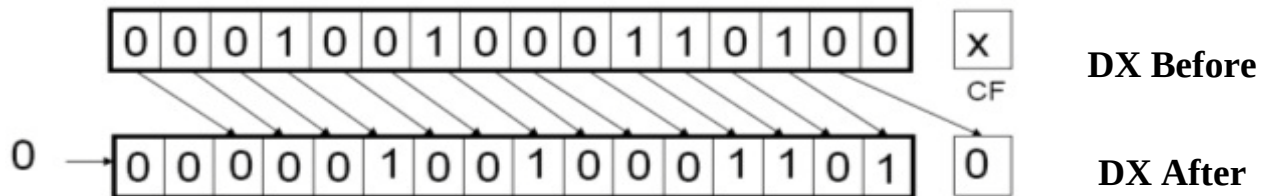
Solution: causes the 16-bit register to be shifted 1-bit position to the left where the vacated LSB is filled with zero and the bit shifted out of the MSB is saved in CF



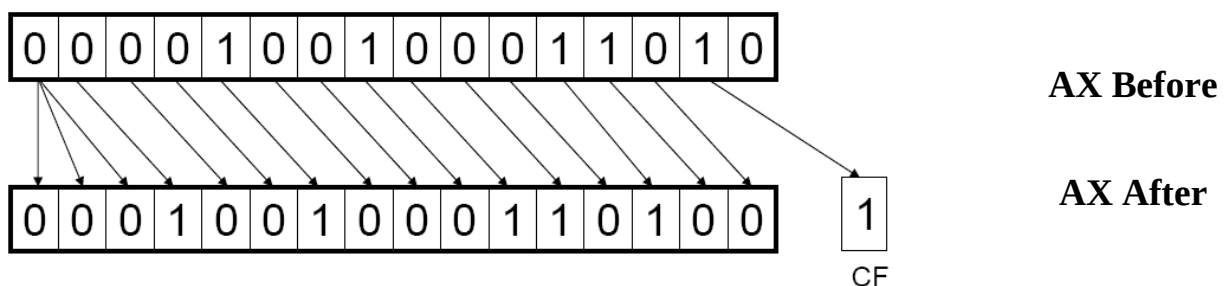


Example: **MOV CL, 2H**
 SHR DX, CL

The two MSBs are filled with zeros and the LSB is thrown away while the second LSB is saved in CF.



Example: Assume CL= 2 and AX= 091AH. Determine the new contents of AX and CF after the instruction **SAR AX, CL** is executed.



- This operation is equivalent to division by powers of 2 as long as the bits shifted out of the LSB are zeros.

Example: Multiply AX by 10 using shift instructions

Solution: **SHL AX, 1**
 MOV BX, AX
 MOV CL, 2
 SHL AX, CL
 ADD AX, BX

Example: What is the result of **SAR CL, 1**, if CL initially contains B6H?

Solution: **DBH**

Example: What is the result of **SHL AL, CL**, if AL contains 75H and CL contains 3?

Solution: **A8H**

Example: Assume DL contains signed number; divide it by 4 using shift instruction?

Solution: **MOV CL, 2**
 SAR DL, CL

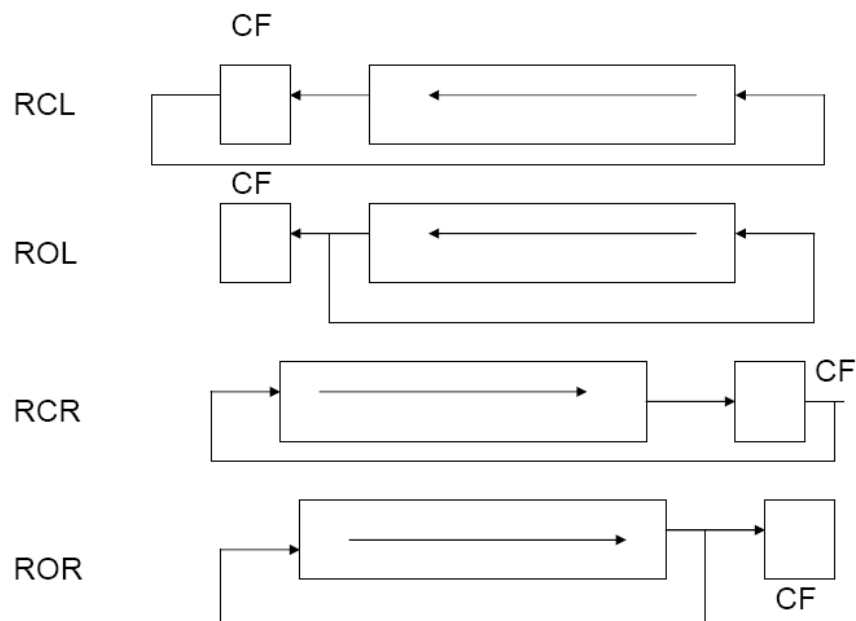


Rotate Instructions

Mnemonic	Meaning	Format	Operation	Flags Affected
ROL	Rotate left	ROL D, Count	Rotate the (D) left by the number of bit positions equal to Count. Each bit shifted out from the left most bit goes back into the rightmost bit position.	CF OF undefined if count $\neq 1$
ROR	Rotate right	ROR D, Count	Rotate the (D) right by the number of bit positions equal to Count. Each bit shifted out from the rightmost bit goes back into the leftmost bit position.	CF OF undefined if count $\neq 1$
RCL	Rotate left through carry	RCL D, Count	Same as ROL except carry is attached to (D) for rotation.	CF OF undefined if count $\neq 1$
RCR	Rotate right through carry	RCR D, Count	Same as ROR except carry is attached to (D) for rotation.	CF OF undefined if count $\neq 1$

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Allowed Operands for rotate instruction

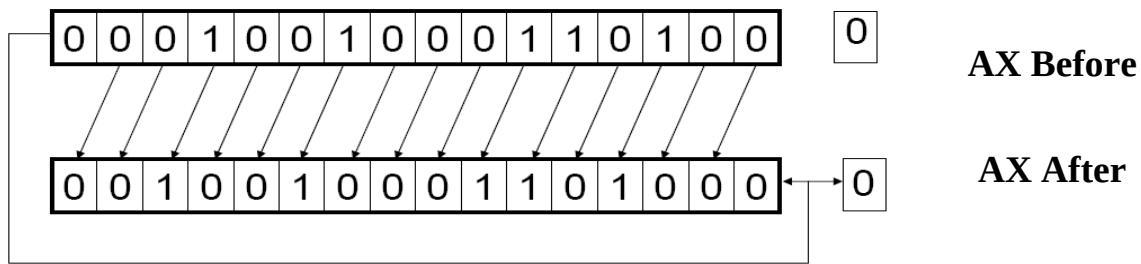




Example : Assume AX = 1234H , what is the result of executing the instruction

ROL AX, 1

Solution :



The original value of bit 15 which is 0 is rotated into CF and bit 0 of AX. All other bits have been rotated 1 bit position to the left.

Rotate right **ROR** instruction operates the same way as **ROL** except that data is rotated to the right instead of left.

In rotate through carry left **RCL** and rotate through carry right **RCR** the bits rotate through the carry flag.

Example: Find the addition result of the two hexadecimal digits packed in DL.

Solution:

```
MOV CL , 04H
MOV BL , DL
ROR DL , CL
AND BL , 0FH
AND DL , 0FH
ADD DL , BL
```



lecture 7

8086 programming –Control Flow Instructions and Program Structures

1. Flag Control

A group of instructions that directly affect the state of the flags:

LAHF Load AH from flags (AH) $\leftarrow$ (Flags)

SAHF Store AH into flags (Flags) $\leftarrow$ (AH) Flags affected: SF, ZF, AF, PF, CF

CLC Clear Carry Flag (CF) $\leftarrow$ 0

STC Set Carry Flag (CF) $\leftarrow$ 1

CLI Clear Interrupt Flag (IF) $\leftarrow$ 0

STI Set interrupts flag (IF) $\leftarrow$ 1

CMC

SF	ZF		AF		PF		CF
-----------	-----------	--	-----------	--	-----------	--	-----------

Format of the AH register for the LAHF and SAHF instructions

Example: Write an instruction sequence to save the current contents of the 8086's flags in the memory location pointed to by SI and then reload the flags with the contents of memory location pointed to by DI

Solution:

```

LAHF
MOV [SI], AH
MOV AH, [DI]
SAHF

```

The instructions CLC, STC, and CMC are used to clear, set, and complement the carry flag.

Example: Clear the carry flag without using CLC instruction.

Solution:

```

STC
CMC

```



2. Compare instruction

Mnemonic	Meaning	Format	Operation	Flag affected
CMP	Compare	CMP D,S	(D) – (S) is used in setting or resetting the flags	CF, AF , OF, PF, SF ,ZF

Compare instruction

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Allowed operands for compare instruction

Example: Describe what happens to the status flags as the sequence of instructions is executed

```
MOV AX, 1234H
MOV BX, 0ABCDH
CMP AX, BX
```

Solution :

The First two instructions makes

$$(AX) = 0001001000110100B$$

$$(BX) = 1010101111001101B$$

The compare instruction performs

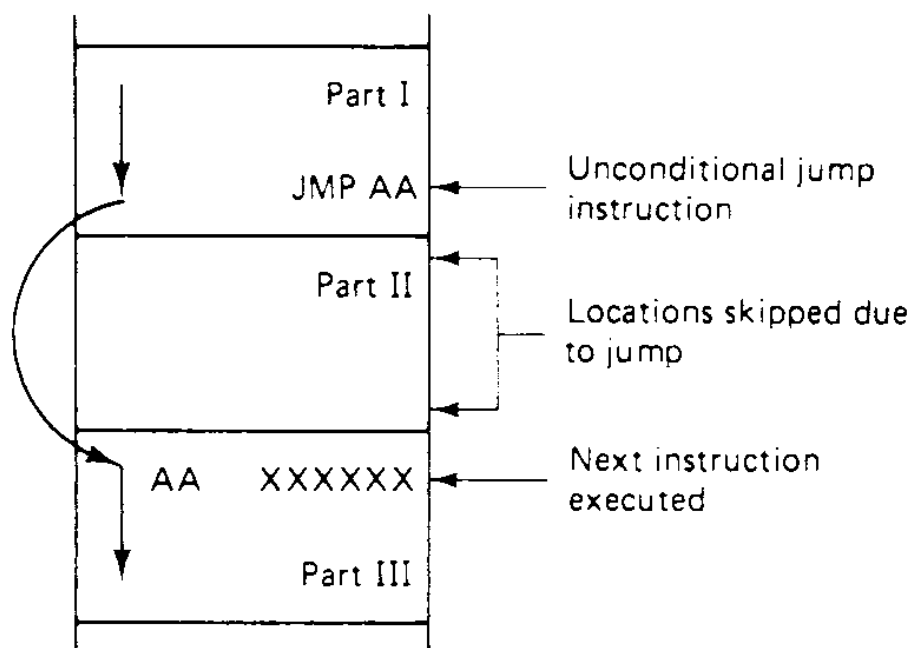
$$(AX) - (BX) = 0001001000110100B - 1010101111001101B = 0110011001100111B$$

The results of the subtraction is nonzero (ZF=0), positive (SF=0), overflow did not occur OF=0, Carry and auxiliary carry occurred therefore, (CF=1, and AF =1). Finally, the result has odd parity (PF=0).

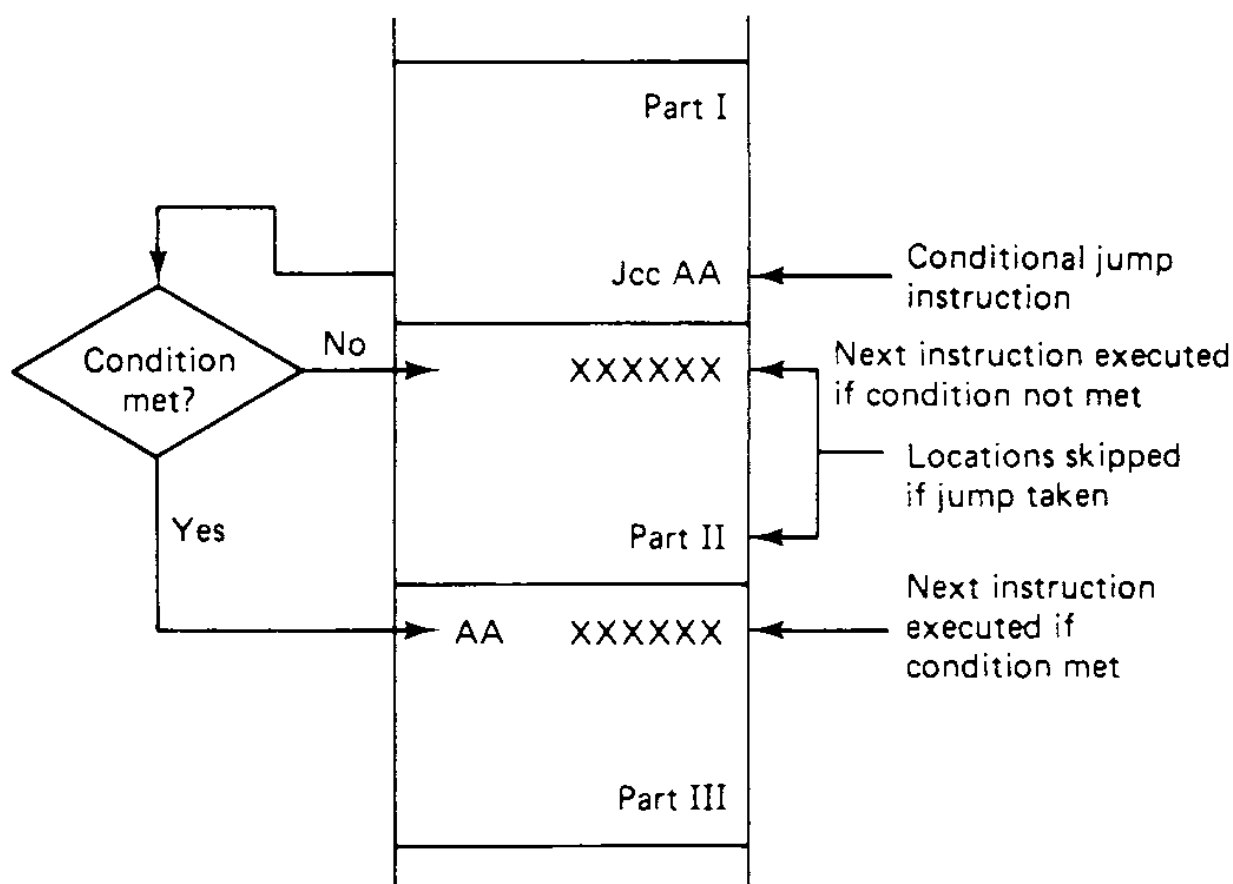
3. Jump Instructions

There are two types of jump, **unconditional** and **conditional**

In **unconditional jump**, as the instruction is executed, the jump always takes place to change the execution sequence.



(a)



(b)

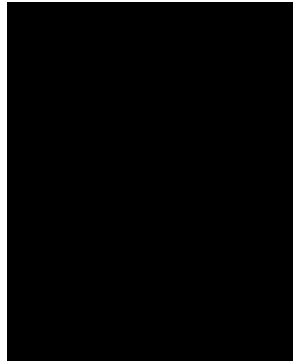
(a) Unconditional jump program sequence (b) Conditional jump program sequence.



1.1. Unconditional Jump

Mnemonic	Meaning	Format	Operation	Flag affected
JMP	Unconditional jump	JMP Operand	Jump is initiated to the address specified by the operand	none

(a)



(b)

Examples:

JMP 1234H; IP will take the value 1234H

JMP BX; IP will take the value in BX

JMP [BX]; IP will take the value in memory location pointed to by BX

JMP DWORD PTR [DI] ; DS:DI points to two words in memory, the first word identifies the new IP and the next word identifies the new CS.

Unconditional Jump types:

a) Intrasegment: this is a jump within the current segment

i) Short Jump: Format → **JMP short Label (8 bit)**

ii) Near Jump: Format → **JMP near Label (16 bit)**

Example: Consider the following example of an unconditional jump instruction:

JMP 1234H

It means jump to address 1234H. However, the value of the address encoded in the instruction is not 1234H. Instead, it is the difference between the incremented value in IP and 1234H. This offset is encoded as either an 8-bit constant (short label) or a 16-bit constant (near label), depending on the size of the difference.



iii) Memptr16: Format → JMP Memptr16

iv) Regptr16:: Format → JMP Regptr16

Example: the jump-to address can also be specified indirectly by the contents of a memory location or the contents of a register, corresponding to the Memptr16 and Regptr16 operand, respectively. Just as for the Near-label operand, they both permit a jump to any address in the current code segment. Forexample,

JMP BX

uses the contents of register BX for the offset in the current code segment that is, the value in BX is copied into IP.

To specify an operand as a pointer to memory, the various addressing modes of 8086 can be used, For instance:

JMP [BX]

uses the contents of BX as the offset address of them memory location that contains the value of IP (Memptr16 operand).

Example

JMP [SI] will replace the IP with the contents of the memorylocations pointed by DS:SI and DS:SI+1

JMP [BP + SI + 1000] like previous but in SS

b) Intersegment :this is a jump out of the current segment.

i) Far Jump: Format → JMP far Label (32 bit label)

The first 16 bit are loaded in **IP**. The other 16 bit are loaded in **CS**

Example:

JMP 2000h : 400h (if this address is out of the range of current code segment)

ii) Memptr32: Format →JMP Memptr32

An indirect way to specify the offset and code-segment address for an intersegment jump is by using the Memptr32 operand. This time the four consecutive memory bytes starting at the specified address contain the offset address and the new code segment address respectively.

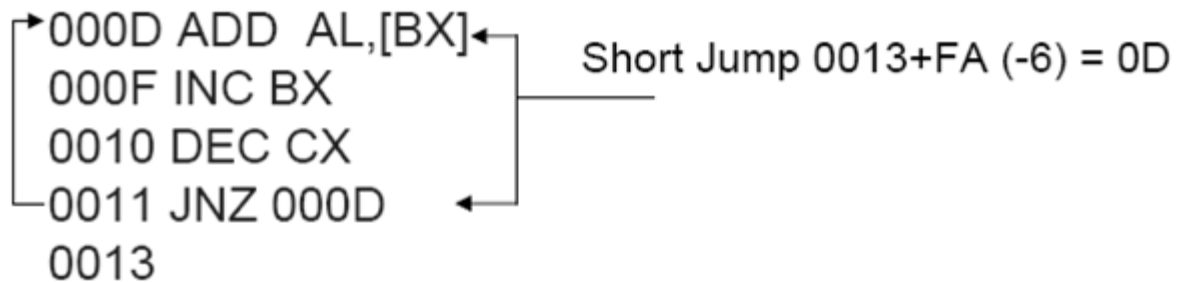
Example: JMP DWORD PTR [DI]



1.2. Conditional Jump

- Conditional Jump is a two byte instruction.
- In a jump backward the second byte is the 2's complement of the displacement value.
- To calculate the target the second byte is added to the IP of the instruction right after the jump.

Example :



The JNZ instruction will be encoded as :**75FA H**

- Next table is a list of each of the conditional jump instructions in the 8086.
- Each one of these instructions tests for the presence or absence of certain status conditions
- Note that for some of the instructions in next table, two different mnemonics can be used. This feature can be used to improve program readability.

For instance the **JP** and **JPE** are identical. Both instructions test the Parity flag (PF) for logic 1.

Example : Write a program to add (50)H numbers stored at memory locations starting at 4400:0100H, then store the result at address 200H in the same data segment.

Solution:

```

MOV AX , 4400H
MOV DS , AX
MOV CX , 0050H ← counter
MOV BX , 0100H ← offset
  
```

Again:

```

ADD AL, [BX]
INC BX
DEC CX
JNZ Again ← label
MOV [0200], AL
  
```



JNB	not below	CF = 0
JNBE	not below nor equal	CF = 0 and ZF = 0
JNC	not carry	CF = 0
JNE	not equal	ZF = 0
JNG	not greater	((SF xor OF) or ZF) = 1
JNGE	not greater nor equal	(SF xor OF) = 1
JNL	not less	SF = OF
JNLE	not less nor equal	ZF = 0 and SF = OF
JNO	not overflow	OF = 0
JNP	not parity	PF = 0
JNS	not sign	SF = 0
JNZ	not zero	ZF = 0
JO	overflow	OF = 1
JP	parity	PF = 1
JPE	parity even	PF = 1
JPO	parity odd	PF = 0
JS	sign	SF = 1
JZ	zero	ZF = 1

Conditional Jump instructions



Example: Write a program to move a block of 100 consecutive bytes of data starting at offset address 400H in memory to another block of memory locations starting at offset address 600H. Assume both block at the same data segment F000H.

Solution:

MOV AX, F000H

MOV DS, AX

MOV SI, 0400H

MOV DI, 0600H

MOV CX, 64H

← 64 Hexadecimal == 100 Decimal

LableX: MOV AH, [SI]

MOV [DI], AH

INC SI

INC DI

DEC CX

JNZ *LableX*

HLT

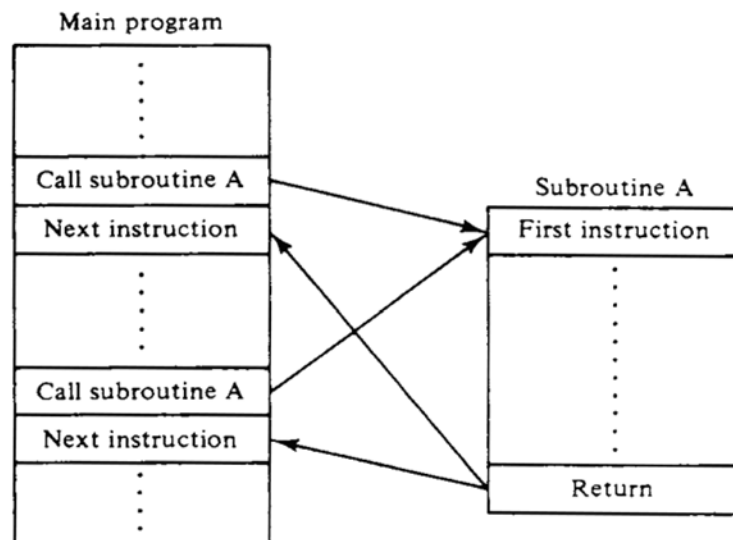
← End of program

- To distinguish between comparisons of signed and unsigned numbers by jump instructions, two different names are used.
- **Above** and **Below** used for comparison of unsigned numbers.
- **Less** and **Greater** used for comparison of signed numbers.
- For instance, the numbers $ABCD_{16}$ is above the number 1234_{16} if they are considered to be unsigned numbers. ON the other hand, if they are treated as signed numbers, $ABCD_{16}$ is negative and 1234_{16} is positive. Therefore, $ABCD_{16}$ is less than 1234_{16} .

4. Subroutines and subroutine-handling instructions

- A subroutine is a special segment of program that can be called for execution form any point in program.
- There two basic instructions for subroutine : **CALL** and **RET**
- **CALL** instruction is used to call the subroutine.
- **RET** instruction must be included at the end of the subroutine to initiate the return sequence to the main program environment.
- Just like the **JMP** instruction, **CALL** allows implementation of two types of operations: the *intrasegment call* and *intersegment call*.

Examples: CALL 1234h
 CALL BX
 CALL [BX]
 CALL DWORD PTR [DI]



(a)

Mnemonic	Meaning	Format	Operation	Flags Affected
CALL	Subroutine call	CALL operand	Execution continues from the address of the subroutine specified by the operand. Information required to return back to the main program such as IP and CS are saved on the stack.	None

(b)

Operand
Near-proc
Far-proc
Memptr16
Regptr16
Memptr32

(c)

(a) Subroutine concept (b) Subroutine call instruction (c) Allowed operands

- Every subroutine must end by executing an instruction that returns control to the main program. This is the return (**RET**)
- The operand of the call instruction initiates an intersegment or intrasegment call
- The intrasegment call causes contents of **IP** to be saved on Stack.
- The Operand specifies new value in the **IP** that is the first instruction in the subroutine.
- The Intersegment call causes contents of **IP** and **CS** to be saved in the stack and new values to be loaded in **IP** and **CS** that identifies the location of the first instruction of the subroutine.
- Execution of RET instruction at the end of the subroutine causes the original values of **IP** and **CS** to be POPed from stack.



Mnemonic	Meaning	Format	Operation	Flags affected
RET	Return	RET or RET operand	Return to the main program by restoring IP (and CS for far-proc). If operand is present, it is added to the contents of SP	None

Ret instruction

There is an additional option with the return instruction. It is that a 2-byte constant can be included with the return instruction. This constant is added to the stack pointer after restoring the return address. The purpose of this stack pointer displacement is to provide a simple means by which the parameters that were saved on the stack before the call to the subroutine was initiated can be discarded. For instance, the instruction

RET 2

when executed adds 2 to SP. This discards one word parameter as part of the return sequence.

PUSH and POP instruction

- Upon entering a subroutine, it is usually necessary to save the contents of certain registers or some other main program parameters. Pushing them onto the stack saves these values.
- Before return to the main program takes place, the saved registers and main program parameters are restored. Popping the saved values from the stack back into their original locations does this.

Mnemonic	Meaning	Format	Operation	Flags affected
PUSH	Push word onto stack	PUSH S	$((SP)) \leftarrow (S)$ $(SP) \leftarrow (SP)-2$	None
POP	Pop word off stack	POP D	$(D) \leftarrow ((SP))$ $(SP) \leftarrow (SP)+2$	None

PUSH and POP instructions

Operand (S or D)
Register
Seg-reg (CS illegal)
Memory

Allowed operand



lecture 8

8086 programming –Control Flow Instructions and Program Structures (continue)

Example: write a procedure named *Square* that squares the contents of BL and places the result in BX.

Solution:

```
Square:  PUSH AX
         MOV AL, BL
         MUL BL
         MOV BX, AX
         POP AX
         RET
```

Example: write a program that computes $y = (AL)^2 + (AH)^2 + (DL)^2$, places the result in CX. Make use of the SQUARE subroutine defined in the previous example. (Assume result y doesn't exceed 16 bit)

Solution:

```
MOV CX, 0000H
MOV BL, AL
CALL Square
ADD CX, BX
MOV BL, AH
CALL Square
ADD CX, BX
MOV BL, DL
CALL Square
ADD CX, BX
HLT
```

- Sometimes we want to save the content of the flag register, and if we save them, we will later have to restore them, these operations can be accomplished with push flags (PUSHF) and pop flags (POPF) instructions, respectively.

Mnemonic	Meaning	Operation	Flags affected
PUSHF	Push flags onto stack	$((SP)) \leftarrow (flags)$ $(SP) \leftarrow (SP) - 2$	None
POPF	Pop flags from stack	$(flags) \leftarrow ((SP))$ $(SP) \leftarrow (SP) + 2$	OF, DF, IF, TF, SF ZF, AF, PF, CF

Push flags and pop flags instructions



LOOPS AND LOOP-HANDLING INSTRUCTIONS

The 8086 microprocessor has three instructions specifically designed for implementing loop operations. These instructions can be use in place of certain conditional jump instruction and give the programmer a simpler way of writing loop sequences. The loop instructions are listed in table below:

Mnemonic	Meaning	Format	Operation
LOOP	Loop	LOOP Short-label	$(CX) \leftarrow (CX) - 1$ Jump is initiated to location definedby short-label if $(CX) \neq 0$; otherwise, execute next sequential instruction
LOOPE LOOPZ	Loop while equal/loop while zero	LOOPE/LOOPZ short-label	$(CX) \leftarrow (CX) - 1$ Jump to location defined by short-label if $(CX) \neq 0$ and $ZF = 1$; otherwise, execute next sequential instruction
LOOPNE LOOPNZ	Loop while not equal/ loop while not zero	LOOPNE/LOOPNZ short-label	$(CX) \leftarrow (CX) - 1$ Jump to location defined by short-label if $(CX) \neq 0$ and $ZF = 0$; otherwise, execute next sequential instruction

Example: Write a program to move a block of 100 consecutive bytes of data starting at offset address 400H in memory to another block of memory locations starting at offset address 600H. Assume both block at the same data segment F000H. (Similar to the example viewed in lecture 7at page 8). Use loop instructions.

Solution:

```

      MOV AX,F000H
      MOV DS,AX
      MOV SI,0400H
      MOV DI,0600H
      MOV CX, 64H
NEXTPT: MOV AH,[SI]
      MOV [DI], AH
      INC SI
      INC DI
      LOOP NEXTPT
      HLT
  
```

In this way we see that LOOP is a single instruction that functions the same as a decrement CX instruction followed by a JNZ instruction.



STRINGS AND STRING-HANDLING INSTRUCTIONS

80x86 is equipped with special instructions to handle string operations.

String: A series of data words (or bytes) that reside in consecutive memory locations

Permits operations:

- Move data from one block of memory to a block elsewhere in memory,
- Scan a string of data elements stored in memory to look for a specific value,
- Compare two strings to determine if they are the same or different.

Five basic String Instructions define operations on one element of a string:

- Move byte or word string MOVSB/MOVSW
- Compare string CMPSB/CMPSW
- Scan string SCASB/SCASW
- Load string LODSB/LODSW
- Store string STOSB/STOSW

Repetition is needed to handle more than one element of a string.

Mnemonic	Meaning	Format	Operation	Flags affected
MOVS	Move string	MOVSB MOVSW	$((ES)0+(DI)) \leftarrow ((DS)0+(SI))$ $(SI) \leftarrow (SI) \pm 1 \text{ or } 2$ $(DI) \leftarrow (DI) \pm 1 \text{ or } 2$	None
CMPS	Compare string	CMPSB CMPSW	set flags as per $((DS)0+(SI)) - ((ES)0+(DI))$ $(SI) \leftarrow (SI) \pm 1 \text{ or } 2$ $(DI) \leftarrow (DI) \pm 1 \text{ or } 2$	CF, PF, AF, ZF, SF, OF
SCAS	Scan string	SCASB SCASW	set flags as per $(AL \text{ or } AX) - ((ES)0+(DI))$ $(DI) \leftarrow (DI) \pm 1 \text{ or } 2$	CF, PF, AF, ZF, SF, OF
LODS	Load string	LODSB LODSW	$(AL \text{ or } AX) \leftarrow ((DS)0+(SI))$ $(SI) \leftarrow (SI) \pm 1 \text{ or } 2$	None
STOS	Store string	STOSB STOSW	$((ES)0+(DI)) \leftarrow (AL \text{ or } AX)$ $(DI) \leftarrow (DI) \pm 1 \text{ or } 2$	None

Basic string instructions



Auto-indexing of String Instructions

Execution of a string instruction causes the address indices inSI and DI to be either automatically incremented or decremented. The decision to increment or decrement is made based on the status of the direction flag.

The direction Flag: Selects the auto increment (D=0) or the autodecrement (D=1) operation for the DI and SI registers during string operations.

Mnemonic	Meaning	Format	Operation	Flags affected
CLD	Clear DF	CLD	$(DF) \leftarrow 0$	DF
STD	Set DF	STD	$(DF) \leftarrow 1$	DF

Instruction for selecting autoincrementing and autodecrementing in string instruction

Example: Using string operation, implement the previous example to copy block of memory to another location.

Solution :

```

MOV CX,64H
MOV AX,F000H
MOV DS,AX
MOV ES,AX
MOV SI,400H
MOV DI,600H
CLD
NXTPT: MOVSB
        LOOP NXTPT
        HTL
  
```

Example: Explain the function of the following sequence of instructions

```

MOV DL, 05
MOV AX, 0A00H
MOV DS, AX
MOV SI, 0
MOV CX, 0FH
AGAIN: INC SI
        CMP [SI], DL
        LOOPNE AGAIN
  
```

**Solution:**

The first 5 instructions initialize internal registers and set up a data segment. The loop in the program searches the 15 memory locations starting from Memory location A001H for the data stored in DL (05H). As long as the value in DL is not found, the zero flag is reset, otherwise it is set. The LOOPNE instruction decrements CX and checks for CX=0 or ZF=1. If neither of these conditions is met, the loop is repeated. If either condition is satisfied, the loop is complete. Therefore, the loop is repeated until either 05 is found or all locations in the address range A001H through A00FH have been checked and are found not to contain 5.

Example: Implement the previous example using SCAS instruction.

Solution:

```
MOV AX, 0H
MOV DS, AX
MOV ES, AX
MOV AL, 05
MOV DI, A001H
MOV CX, 0FH
CLD
AGAIN: SCASB
        LOOPNE AGAIN
```

Example: Write a program that loads the block of memory locations from A000H through 0A00FH with number 5H.

Solution:

```
MOV AX, 0H
MOV DS, AX
MOV ES, AX
MOV AL, 05
MOV DI, 0A000H
MOV CX, 0FH
CLD
AGAIN: STOSB
        LOOP AGAIN
```

In most applications, the basic string operations must be repeated in order to process arrays of data. Inserting a repeat prefix before the instruction that is to be repeated does this, the *repeat prefixes* of the 8086 are shown in table below

For example, the first prefix, **REP**, caused the basic string operation to be repeated until the contents of register CX become equal to 0. Each time the instruction is executed, it causes CX to be tested for 0. If CX is found not to be 0, it is decremented by 1 and the basic string operation is repeated. On the other hand, if it is 0, the repeat



string operation is done and the next instruction in the program is executed, the repeat count must be loaded into CX prior to executing the repeat string instruction.

Prefix	Used with:	Meaning
REP	MOVS STOS	Repeat while not end of string $CX \neq 0$
REPE / REPZ	CMPS SCAS	Repeat while not end of string and strings are equal $CX \neq 0$ and $ZF = 1$
REPNE / REPNZ	CMPS SCAS	Repeat while not end of string and strings are not equal $CX \neq 0$ and $ZF = 0$

Prefixes for use with the basic string operations

Example: write a program to copy a block of 32 consecutive bytes from the block of memory locations starting at address 2000H in the current Data Segment (DS) to a block of locations starting at address 3000H in the current Extra Segment (ES).

```

CLD
MOV AX, data_seg
MOV DS, AX
MOV AX, extra_seg
MOV ES, AX
MOV CX, 20H
MOV SI, 2000H
MOV DI, 3000H
REPZMOVSB
  
```

Example: Write a program that scans the 70 bytes start at location D0H in the current Data Segment for the value 45H, if this value is found replace it with the value 29H and exit scanning.

```

MOV ES, DS
CLD
MOV DI, 00D0H
MOV CX, 0046H
MOV AL, 45H
REPNE SCASB
DEC DI
MOV BYTE PTR [DI], 29H
HLT
  
```



lecture 9

8086 Microprocessor and its Memory and Input / Output Interface

In this lecture, we cover the 8086 microcomputer from the hardware point of view. The 8086, announced in 1978, was the first 16-bit microprocessor introduced by Intel Corporation. The 8086 is manufactured using *high-performance metal-oxide semiconductor* (HMOS) *technology*, and the circuitry on its chips is equivalent to approximately 29000 transistors. It is housed in a 40-pin dual in-line package.

As seen from Pin diagram of the 8086 (Figure 1) that many of its pins have multiple function.

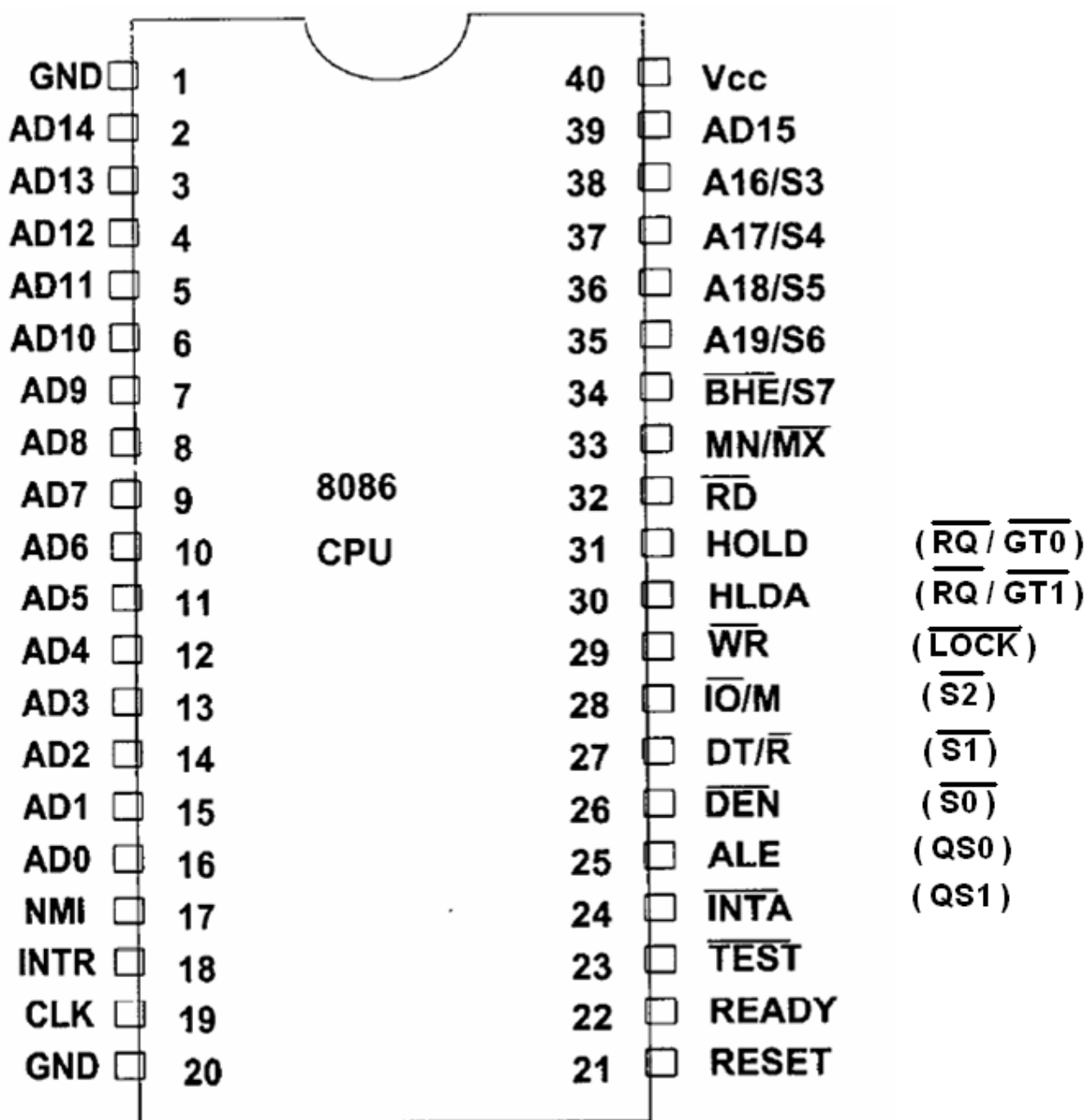


Figure 1: Pin layout of the 8086



For example, we see that address bus lines A_0 through A_{15} and data bus lines D_0 through D_{15} are multiplexed. For this reason, these leads are labeled AD_0 through AD_{15} . By multiplexed we mean that the same physical pin carries an address bit at one time and the data bit at another time.

The 8086 can be configured to work in either of two modes:

- The *minimum mode* is selected by applying logic 1 to the $MN/\overline{MX}$ input lead. Minimum mode 8086 systems are typically smaller and contain a single microprocessor.
- The *maximum mode* is selected by applying logic 0 to the $MN/\overline{MX}$ input lead. Maximum mode configures 8086 systems for use in larger systems and with multiple processors.

Depending on the mode of operation selected, the assignments for a number of pins on the microprocessor package are changed. As Figure 1 shows, the pin function of the 8086 specified in parentheses relate to a maximum-mode system. Figure 2 below list the names, types and functions of the 8086 signals

Common signals		
Name	Function	Type
AD15-AD0	Address /data bus	Bidirectional , 3-state
A19/S6-A16/S3	Address / status	Output/ , 3-state
$MN/\overline{MX}$	Minimum/Maximum mode control	Input
$\overline{RD}$	Read control	Output, 3-state
$\overline{TEST}$	Wait on test control	Input
READY	Wait state control	Input
RESET	System reset	Input
NMI	Non-maskable interrupt request	Input
INTR	Interrupt request	Input
CLK	System clock	Input
V_{CC}	+5 volt	Input
GND	Ground	Input

(a)

Minimum mode signals($MN/\overline{MX}=V_{CC}$)		
Name	Function	Type
HOLD	Hold request	Input
HLDA	Hold acknowledgment	Output
$\overline{WR}$	Write control	Output, 3-state
$M/\overline{IO}$	IO/memory control	Output, 3-state
$DT/\overline{R}$	Data transmit /receive	Output, 3-state
$\overline{DEN}$	Data enable	Output, 3-state
$\overline{BHE} \setminus S7$	Bank high enable/Status line 7	Output, 3-state
ALE	Address latch enable	Output
$\overline{INTA}$	Interrupt acknowledgment	Output

(b)



Maximum mode signals(MN/ $\overline{MX}$ =Ground)		
Name	Function	Type
RQ/GT1,0	Request/grant bus access control	Bidirectional
$\overline{LOCK}$	Bus priority lock control	Output, 3-state
$\overline{S2} - \overline{S0}$	Bus cycle status	Output, 3-state
QS1, QS0	Instruction queue status	Output

(c)

Figure 2 (a) signals common to both minimum and maximum mode. (b) Unique minimum-mode signals. (c) Unique maximum-mode signals.

Minimum mode interface signals

The minimum-mode signals can be divided into the following basic groups:

1. Address/Data Bus

The address bus is 20 bits long and consists of signal lines A_0 (the LSB) to A_{19} (the MSB).

The data bus is 16 bits long and consists of signals lines D_0 (the LSB) to D_{15} (the MSB). When acting as a data bus, they carry read/write data for memory, input/output data for I/O devices, and interrupt-type codes from an interrupt controller.

2. Status signals

The four most significant address lines, A_{19} through A_{16} are also multiplexed, but with status signals S_6 through S_3 . These status bits are output on the bus at the same time that data are transferred over the other bus lines. Bits S_4 and S_3 together form a 2-bit binary code that identifies which of the internal segment registers was used to generate the physical address that was output on the address bus during the current bus cycle (See Figure 3)

S_4	S_3	Address Status
0	0	alternate(relative to the ES segment)
0	1	Stack (relative to the SS segment)
1	0	Code/None (relative to the CS segment or a default of zero)
1	1	Data (relative to the DS segment)

Figure 3 address bus status codes

Status line S_5 reflects the status of logic level of the internal interrupt enable flag.

3. The control signals

These are provided to support the memory and I/O interfaces of the 8086.

- **ALE signal:** is a pulse to logic 1 that signals external circuitry when a valid address is on the bus. This address can be latched in external circuitry on the 1-to-0 edge pulse at ALE.



- **$\overline{M}/\overline{IO}$ signal:** tells external circuitry whether a memory or I/O transfer is taking place over the bus. (Logic 1 for memory operation, logic 0 for I/O operation).
- **$\overline{DT}/\overline{R}$ signal:** when this line is logic 1 the bus is in Transmit Mode (data are either written into memory or output to an I/O device). When this line is logic 0 the bus is in Receive Mode (data are either read from memory or input to an I/O device).
- **$\overline{BHE}$ signal:** logic 0 on this line is used as a memory enable signal for the most significant byte half of the data bus, D_8 through D_{15} .
- **$\overline{RD}$ signal:** indicate that a read bus cycle is in progress.
- **$\overline{WR}$ signal:** indicate that a write bus cycle is in progress.
- **$\overline{DEN}$ signal:** during read operations, this signal is also supplied to enables external devices to supply data to the microprocessor.
- **READY signal:** used to insert wait states into the bus cycle so that it is extended by a number of clock periods.

4. **Interrupt signals:** ($\overline{INTR}$, $\overline{INTA}$, $\overline{TEST}$, RESET, NMI)

5. **Direct memory access (DMA) interface signals:** (HOLD, $\overline{HLDA}$)

Maximum mode interface signals

When the 8086 microprocessor is set for the maximum-mode configuration, it produces signals for implementing a *multiprocessor/coprocessor system environment*. By *multiprocessor system environment* we mean that multiple microprocessors exist in the system and that each processor executes its own program.

8288 bus controller: Bus Commands and Control Signals

During the maximum mode (as shown in figure 6) operation, the $\overline{WR}$, $\overline{M}/\overline{IO}$, $\overline{DT}/\overline{R}$, $\overline{DEN}$, ALE, and $\overline{INTA}$ signals are no longer produced by the 8086. Instead, it outputs a status code on three signals lines, $\overline{S}_0$, $\overline{S}_1$, and $\overline{S}_2$, prior to the initiation of each bus cycle.

- **$\overline{S}_2, \overline{S}_1, \overline{S}_0$:** These three bits are input to the external *bus controller* device, **the 8288**, which decodes them to identify the type of next bus cycle, as shown in figure 5. In addition to the signal produced (figure 5) the 8288 bus controller produce $\overline{DEN}$, $\overline{DT}/\overline{R}$, and ALE
- **$\overline{LOCK}$ signal:** this signal is meant to be output (logic 0) whenever the processor wants to lock out the other processors from using the bus.
- **Queue Status Signals (QS_1, QS_0):** these two bits tell the external circuitry what type of information was removed from the queue.
- **$\overline{RQ}/\overline{GT}_1, \overline{RQ}/\overline{GT}_0$:** these two signals provide a prioritized bus access mechanism for accessing the local bus.

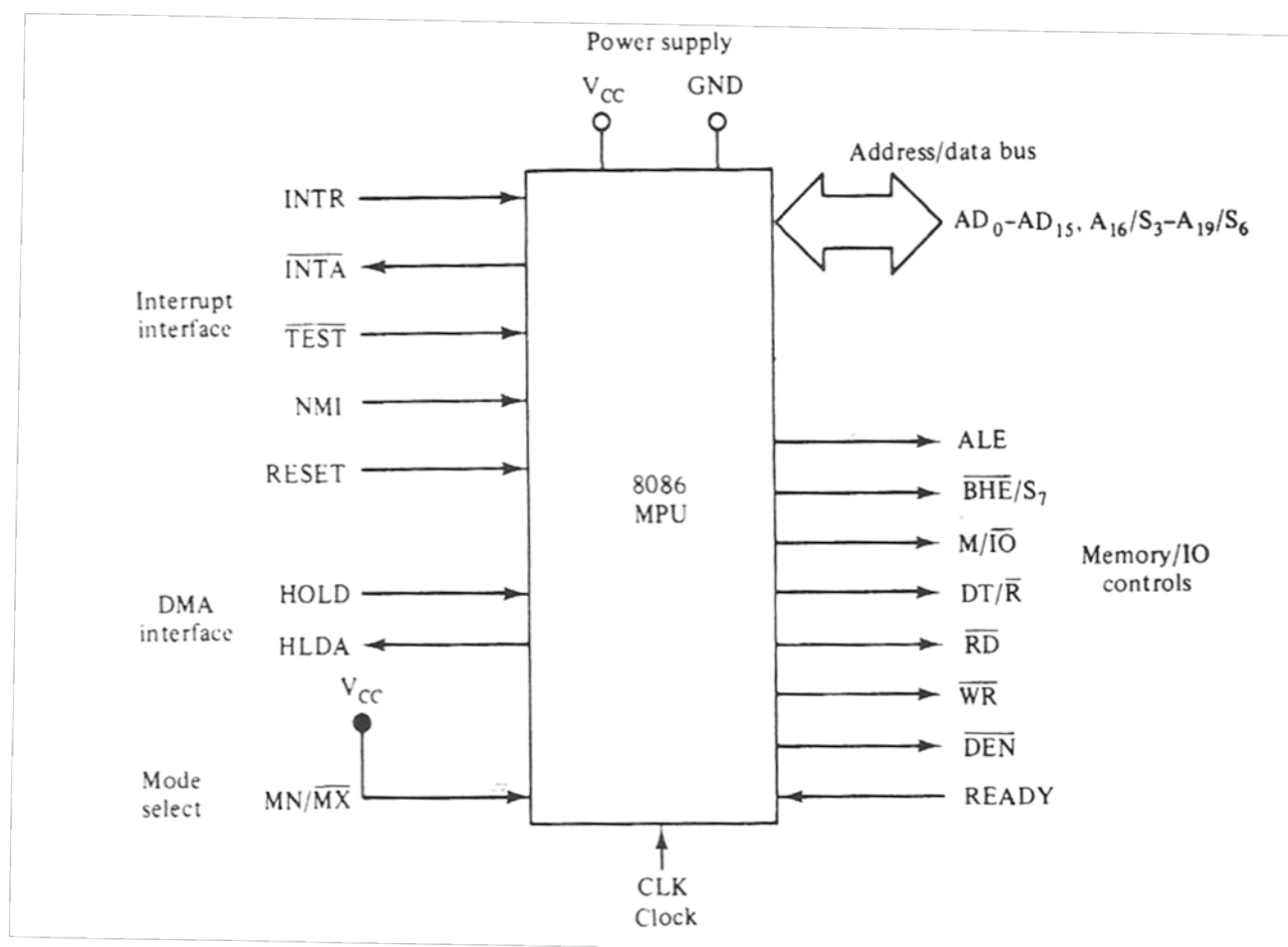


Figure 4 Minimum-Mode block diagrams

Status inputs			CPU Cycle	8288 Command	Meaning
S2	S1	S0			
0	0	0	Interrupt Acknowledge	$\overline{\text{INTA}}$	Interrupt acknowledge
0	0	1	Read I/O port	$\overline{\text{IORC}}$	I/O read control
0	1	0	Write I/O port	$\overline{\text{IOWC}}, \overline{\text{AIOWC}}$	I/O write control, Advanced I/O write control
0	1	1	Halt	None	---
1	0	0	Instruction Fetch	$\overline{\text{MRDC}}$	Memory read control
1	0	1	Read Memory	$\overline{\text{MRDC}}$	Memory read control
1	1	0	Write Memory	$\overline{\text{MWTC}}, \overline{\text{AMWC}}$	Memory write control, advanced memory write control
1	1	1	Passive	None	---

Figure 5 Bus Status Codes

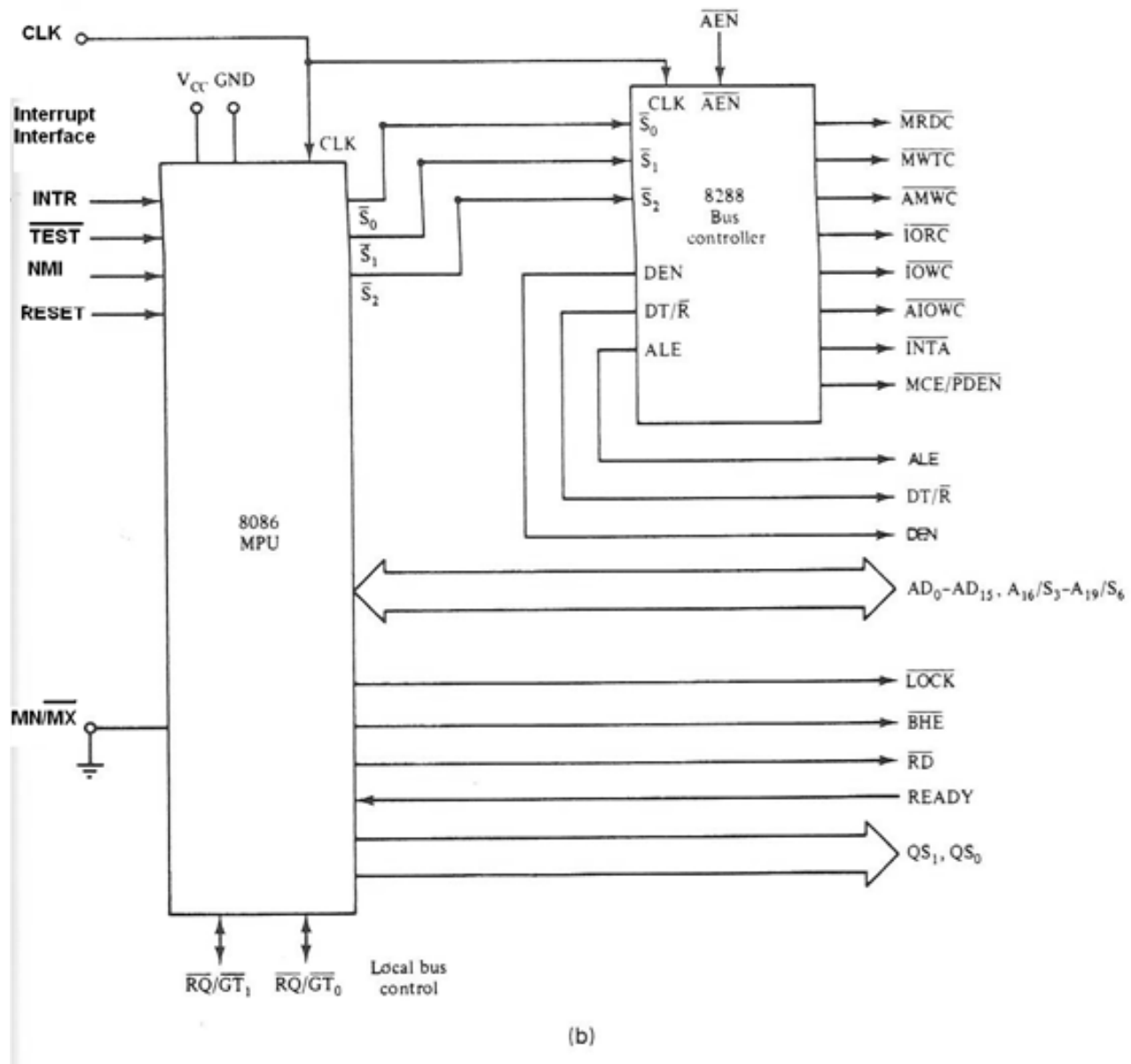


Figure 6 Maximum-Mode block diagram with the 8288 Bus Controller

System Clock

The time base for synchronization of the internal and external operations of the microprocessor in a microcomputer system is provided by the *clock* (CLK) input signal. The 8086 microprocessor is manufactured in three speeds: the **5-MHz** 8086, the **8-MHz** 8086-2 and the **10-MHz** 8086-1. The 8284 clock generator and driver IC generates CLK (Figure 7)

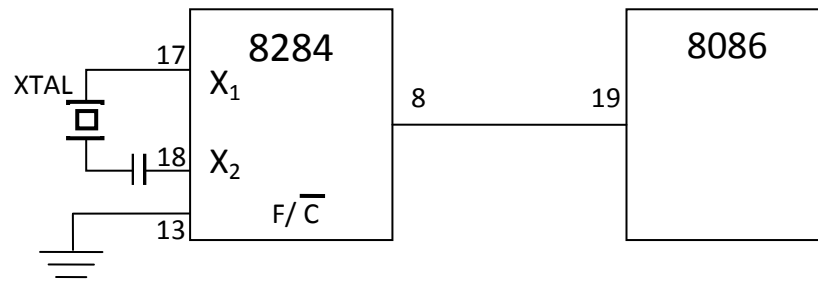


Figure 7 Connecting the 8284 to the 8086.

Bus cycle and time state

A *bus cycle* defines the basic operation that a microprocessor performs to communicate with external devices. Example of bus cycles are

- Memory read
- Memory write
- IO read
- IO write

The bus cycle of 8086 microprocessors consists of at least four clock periods (T_1 , T_2 , T_3 , and T_4):

- During T_1 the 8086 puts an address on the bus.
- During T_2 the 8086 puts the data on the bus (for write memory cycle) and maintained through T_3 and T_4 .
- During T_2 the 8086 puts the bus in high-Z state (for read cycle) and then the data to read must be available on the bus during T_3 and T_4 .

These four clock states give a bus cycle duration of $125 \text{ ns} \times 4 = 500 \text{ ns}$ in an 8-MHz system.

Idle States

If no bus cycles are required, the microprocessor performs what are known as *idle state*. During these states, no bus activity takes place. Each idle state is one clock period long, and any number of them can be inserted between bus cycles. Idle states are performed if the instruction queue inside the microprocessor is full and it does not need to read or write operands from memory.

Wait States

Wait states can be inserted into a bus cycle. This is done in response to request by an event in external hardware instead of an internal event such as a full queue. The READY input of the 8086 is provided specifically for this purpose. As long as READY is held at the 0 level, wait states are inserted between states T_3 and T_4 of the current bus cycle, and the data that were on the bus during T_3 are maintained. The bus cycle is not completed until the external hardware returns READY back to the 1 logic level.

Read Cycle

The read bus cycle begins with state T_1 . During this period, the 8086 outputs the 20-bit address of the memory location to be accessed on its multiplexed address/data bus AD_0 through AD_{15} and multiplexed lines A_{16}/S_3 through A_{19}/S_6 . Note that at the same time a pulse is also produced at ALE. The signal $\overline{BHE}$ is also supplied with the address lines. (Figure 8)

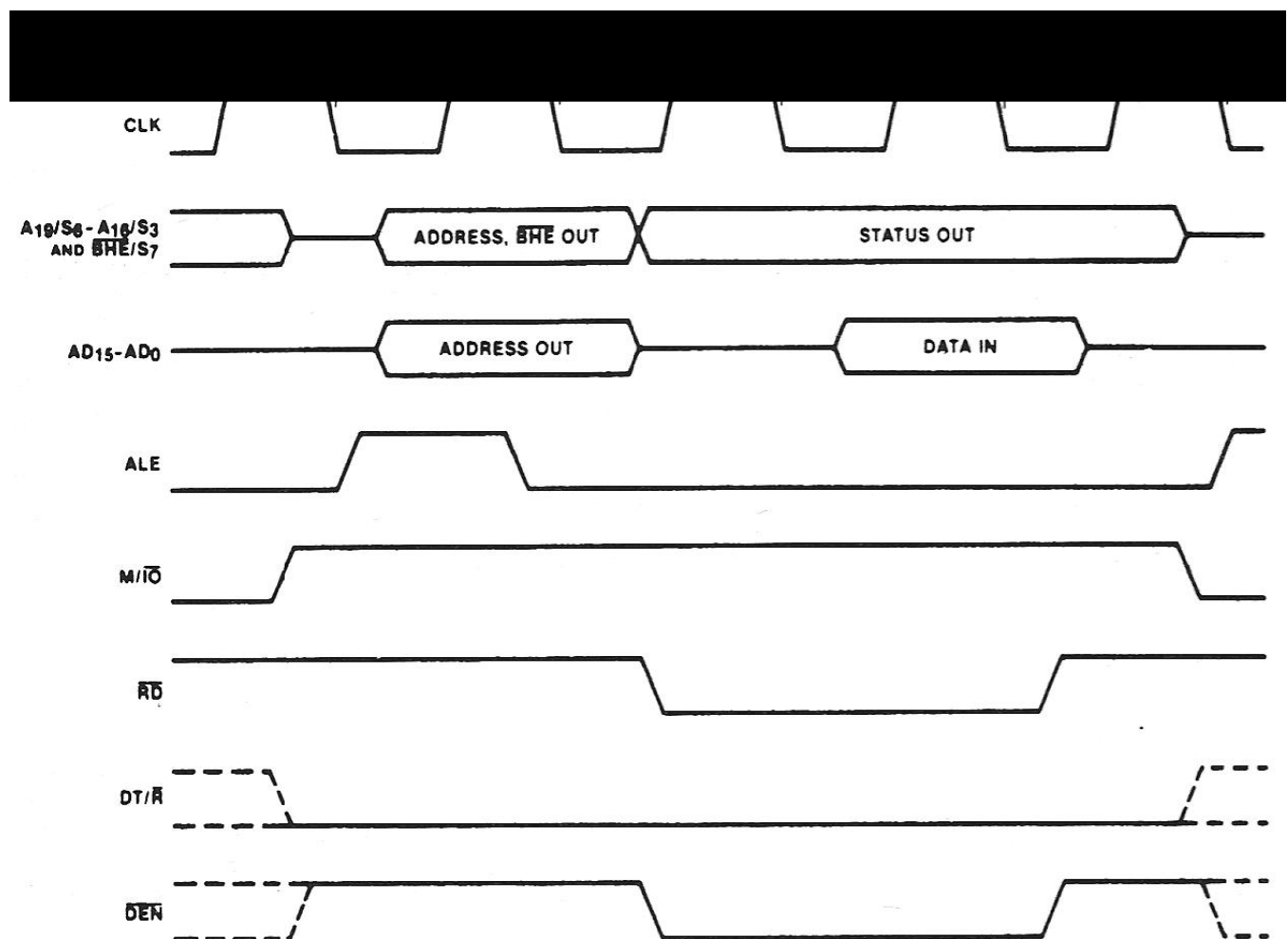


Figure 8 Minimum-mode memory read bus cycle of the 8086.



Write Cycle

The write bus cycle is similar to the read bus cycle except that signal $\overline{WR}$ is set 0 instead of the signal $\overline{RD}$ and signal $DT/\overline{R}$ is set to 1.

Hardware organization of the 8086 memory address space

The 8086's 1Mbyte memory address space is implemented as two independent 512Kbyte banks: the *low (even) bank* and the *high (odd) bank*. Figure 9 shows four different cases that happen during accessing data:

1. When a **byte** of data at an even address (such as X) is to be accessed:
 - A_0 is set to logic 0 to enable the low bank of memory.
 - $\overline{BHE}$ is set to logic 1 to disable the high bank. (Figure 9-a).
2. When a **byte** of data at an odd address (such as X+1) is to be accessed:
 - A_0 is set to logic 1 to disable the low bank of memory.
 - $\overline{BHE}$ is set to logic 0 to enable the high bank. (Figure 9-b).
3. When a **word** of data at an even address (**aligned word**) is to be accessed:
 - A_0 is set to logic 0 to enable the low bank of memory.
 - $\overline{BHE}$ is set to logic 0 to enable the high bank. (Figure 9-c).
4. When a **word** of data at an odd address (**misaligned word**) is to be accessed the 8086 need two bus cycles to access it (Figure 9-d):
 - a. During the first bus cycle, the odd byte of the word (in the high bank) is addressed
 - A_0 is set to logic 1 to disable the low bank of memory.
 - $\overline{BHE}$ is set to logic 0 to enable the high bank.
 - b. During the second bus cycle, the odd byte of the word (in the low bank) is addressed
 - A_0 is set to logic 0 to enable the low bank of memory.
 - $\overline{BHE}$ is set to logic 1 to disable the high bank.

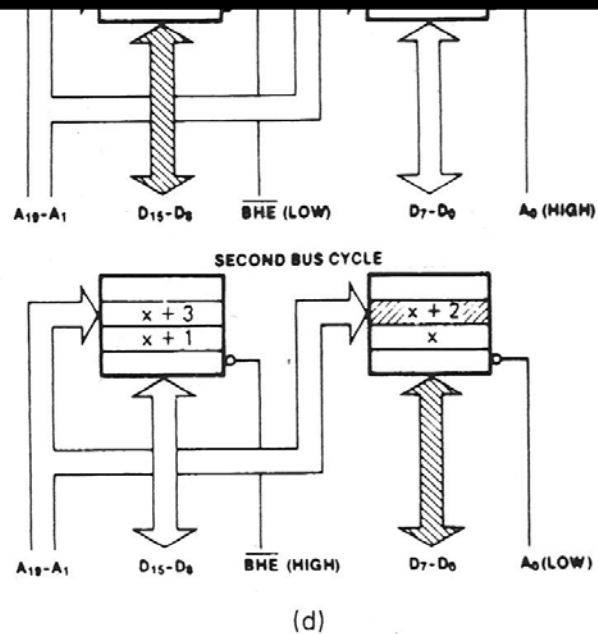


Figure 9 (a) Even-address byte transfer by the 8086. (b) Odd-address byte transfer by the 8086. (c) Even-address word transfer by the 8086. (d) Odd-word transfer by the 8086



lecture 10

Memory Interface Circuits

This lecture describes the memory interface circuits of an 8086-based microcomputer system. Figure 10-1 shows a memory interface diagram for a maximum-mode 8086-based microcomputer system. Here we find that the interface includes

- The 8288 bus controller (see lecture 9)
- Address bus latches and an address decoder (see figure 10-4).
- Bank read and writ control logic (see figures 10-5 and 10-6).
- Data bus transceiver/buffer (see figure 10-8).

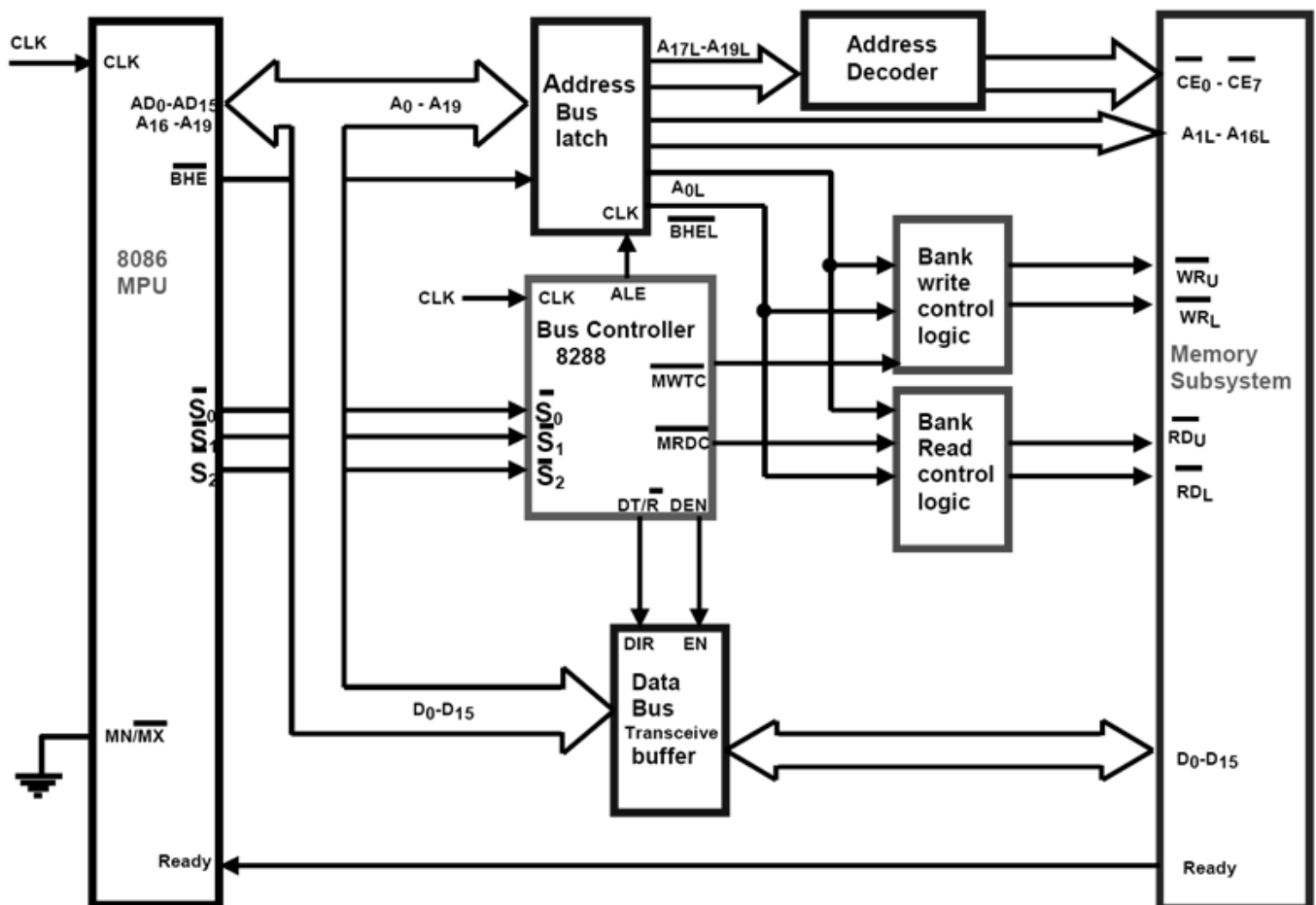


Figure 10-1 memory interface block diagram (for maximum mode)

In the figure above the address bus is latched, buffered, and decoded. We see that address lines A_0 through A_{19} are latched along with control signal $\overline{BHE}$ in the address bus latch. The latched address lines A_{17L} through A_{19L} are decoded to produce chip enable output $\overline{CE}_0$ through $\overline{CE}_7$.

Notice that the 8288 bus controller produces the address latch enable (ALE) control signal from $\bar{S}_2\bar{S}_1\bar{S}_0$.

For the minimum mode, the memory interface is similar to figure 10-1 except that

- The signals ALE , $\overline{DEN}$ and $DT/\bar{R}$ are delivered by 8086 directly.
- $\overline{MWT\bar{C}}$ and $\overline{MRD\bar{C}}$ are produced as shown in figure 10-2.

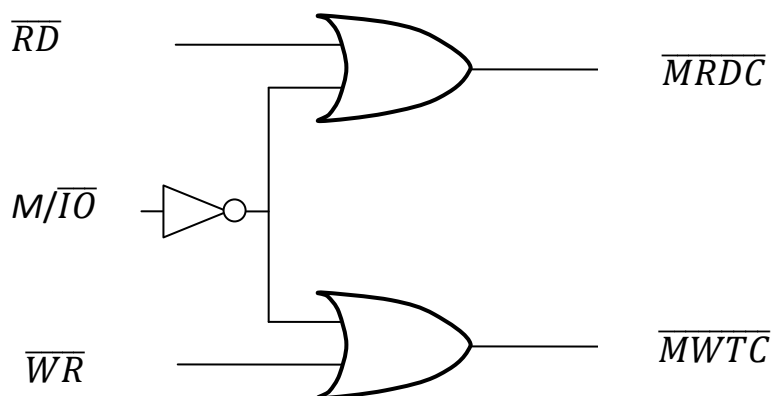


Figure 10-2

Address Bus Latches and Buffers

The 74LS373 is an example of an octal latch device that can be used to implement the *address latch* section of the 8086's memory interface circuit. A block diagram of this device is shown in figure 10-3.

When the clock (input C) is at logic 1, the outputs of the D-type flip-flops follow the logic level of input. When the clock is at logic 0, the current content of the D-type flip-flops are latched.

If the output-control ($\overline{OC}$) input of the buffers is at logic 1, the outputs are in the high-impedance state.

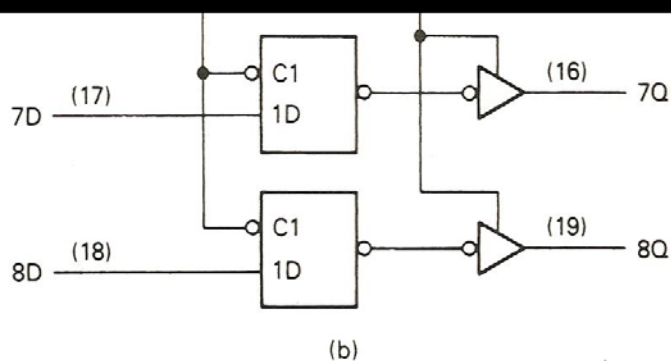
In the 8086 microcomputer system, the 20 address lines (AD_0 - AD_{15} , A_{16} - A_{19}) and the bank high enable signal $\overline{BHE}$ are normally latched in the address bus latch. The circuit configuration shown Figure 10-4 can be used to latch these signals. These latches also provide buffering for the 8086's address lines.

The address information is latched at the outputs when the ALE signal returns to 0.



C
1D
2D
3D
4D
5D
6D
7D
8D
 $\overline{OC}$

الرسم
للاطلاع
فقط



Inputs			Output
$\overline{OC}$	Enable C	D	Q
L	H	H	H
L	H	L	L
L	L	X	Q_0
H	X	X	Z

(c)

Figure 10-3 (a) Block diagram of an octal D-type latch. (b) Circuit diagram of the 74LS373 (c) Operation of the 74LS373

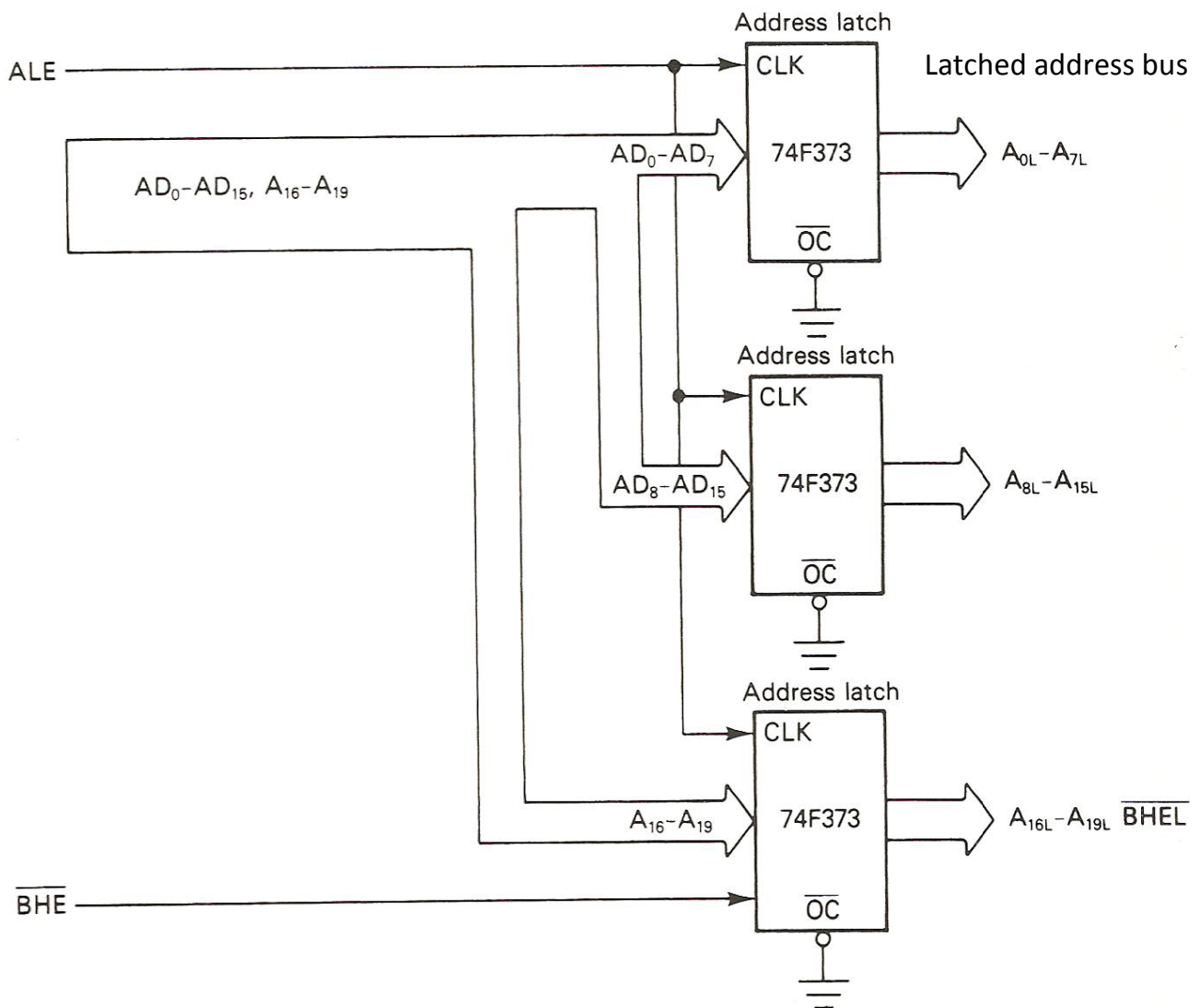


Figure 10-4 Address latch circuit

Bank Write and Bank Read Control Logic

The memory of the 8086 microcomputer is organized in upper and lower banks, it requires separate write and read control signals for the two banks.

The logic circuit in figure 10-5 shows how the bank write control signals, $\overline{WR}_U$ for the upper bank and $\overline{WR}_L$ for the lower bank can be generated from the bus controller signals $\overline{MWT\overline{C}}$, the address bus latch signal A_{0L} and $\overline{BHEL}$.

Similar to the bank write control logic circuit, the bank read control logic circuit can be designed to generate $\overline{RD}_U$, the read for the upper bank memory, and the $\overline{RD}_L$, the read for the lower bank (see figure 10-6).

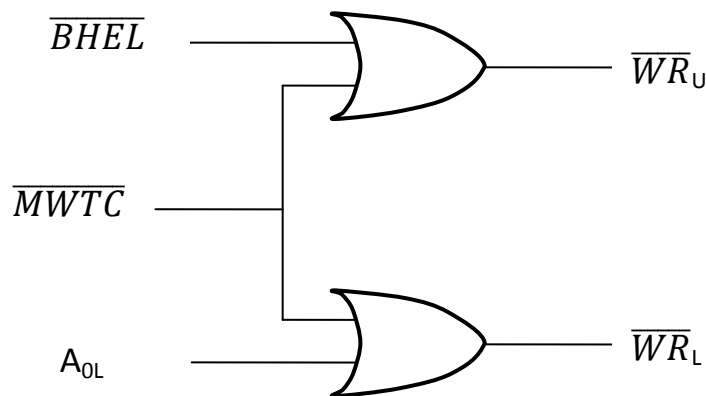


Figure 10-5 Bank write control logic.

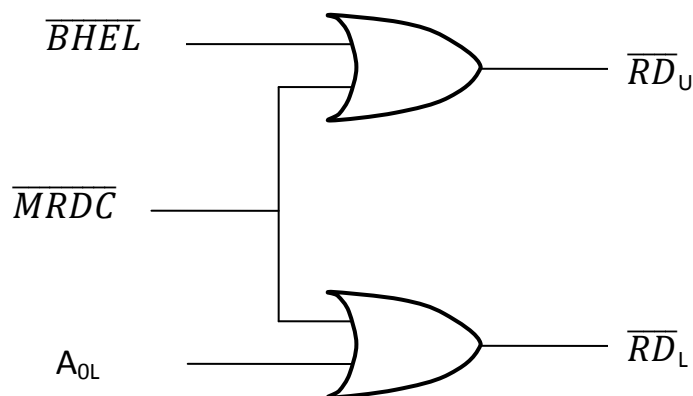


Figure 10-6 Bank Read control logic.

Data Bus Transceivers

The *data bus transceivers* block of the bus interface circuit can be implemented with 74F245 octal bus transceiver ICs. Figure 10-7 shows a block diagram of this device.

Note that:

- $\overline{G}$ input is used to enable the buffer for the operation.
- **DIR** input is used to select the direction in which data are transferred through the device. (if DIR=0 the data pass from B lines to A lines, else if DIR =1 data pass from A lines to B lines)



الرسم
للاطلاع
فقط

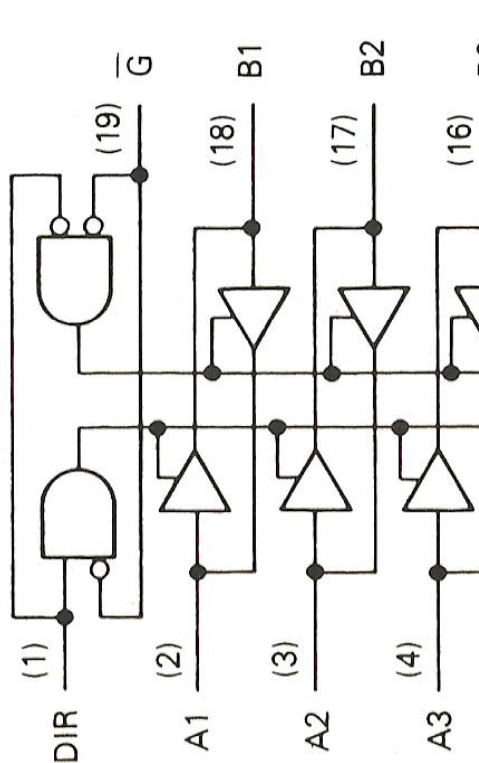
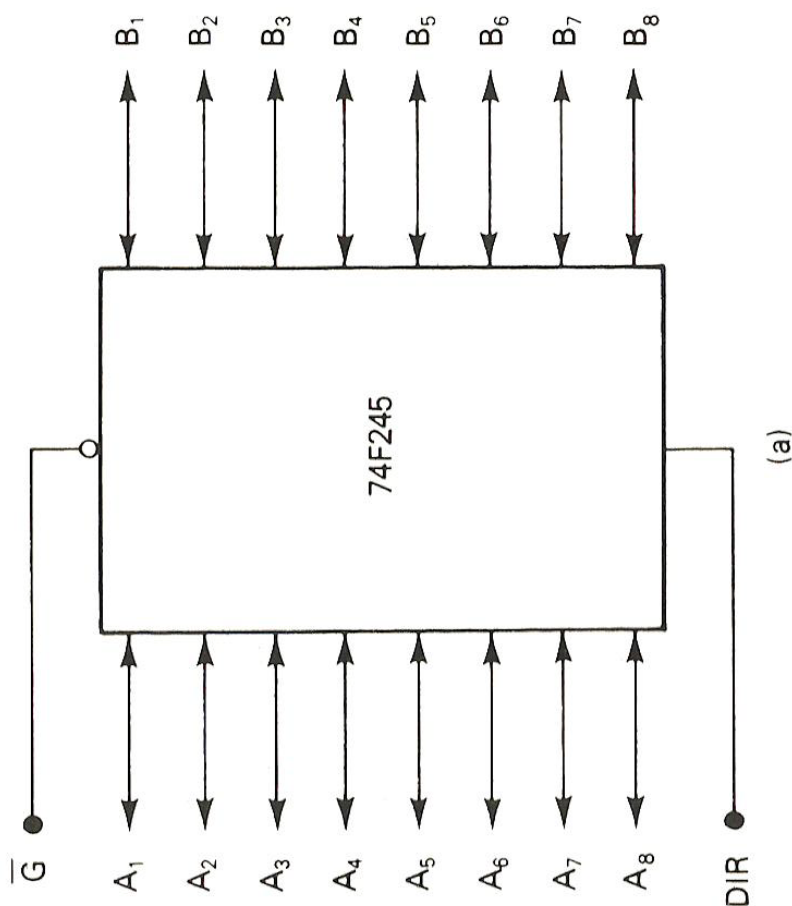


Figure 10-7 (a) Block diagram of the 74LS245 octal bidirectional bus transceiver.
(b) Circuit diagram of the 74LS245.

Figure 10-8 shows a circuit that implements the data bus transceiver block of the bus interface circuit using the 74LS245. For the 16-bit data bus of the 8086 microcomputer, two devices are required.

Here the DIR input is driven by the signal data transmit/receive ($DT/\bar{R}$), and $\bar{G}$ is supplied by data bus enable DEN (from the bus controller 8288 in the maximum mode) or by $\bar{DEN}$ (from 8086 in the minimum mode).

Another key function of the data bus transceiver circuit is to buffer the data bus lines, this capability is defined by how much current the devices can sink at their outputs.

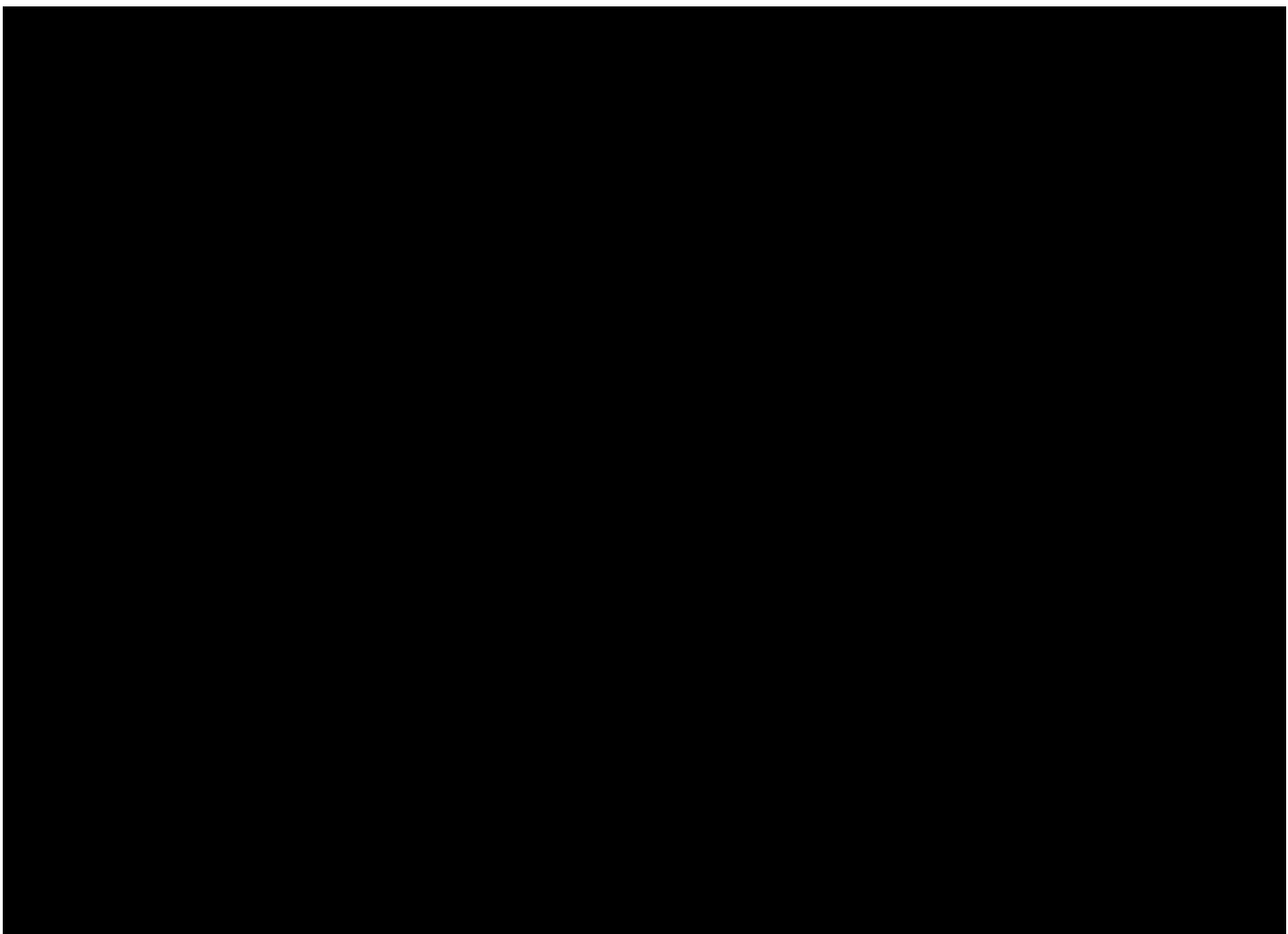


Figure 10-8 Data bus transceiver circuit.

As shown in figure 10-9, the *addressdecoder* in the 8086 microcomputer system is located at the output side of the address latch.

A typical device used to perform this decode function is the 74LS138 decoder. The circuit in Figure 10-10 uses the 74LS138 to generate chip enable signals $\bar{CE}_0$ through $\bar{CE}_7$ by decoding address lines A_{17L} , A_{18L} , and A_{19L} .

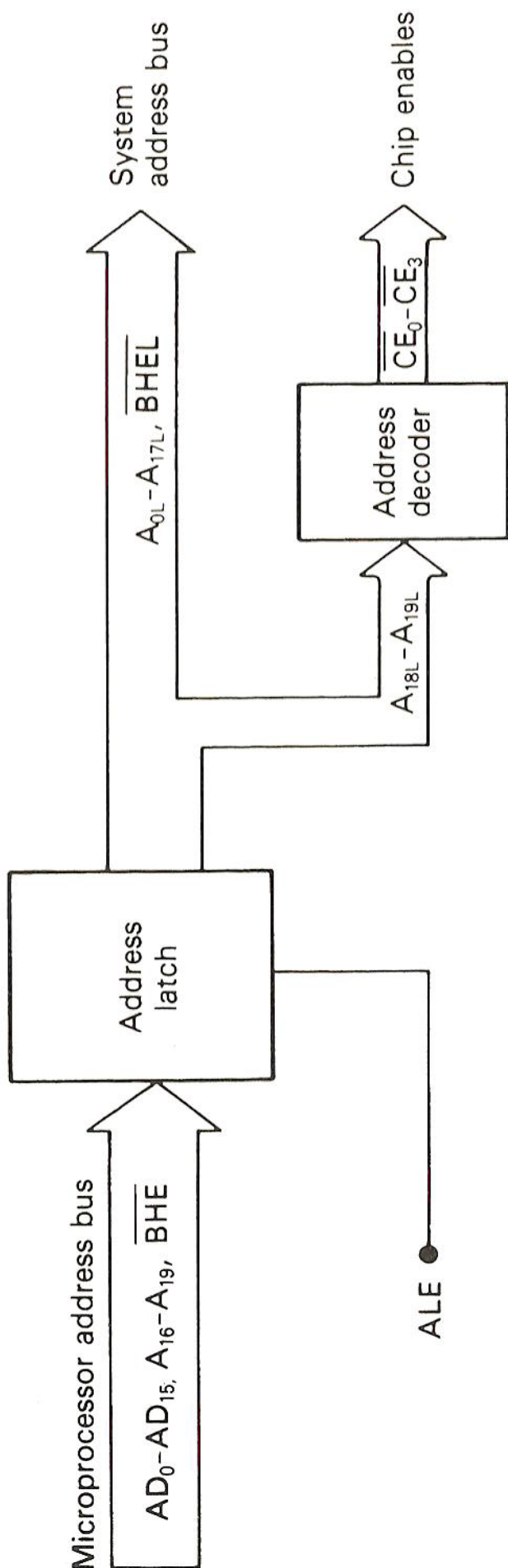


Figure 10-9 Address bus configuration with address decoding.



The **G1** input must be tied to +5V permanently, while **G2A** and **G2B** inputs must be tied to ground permanently.

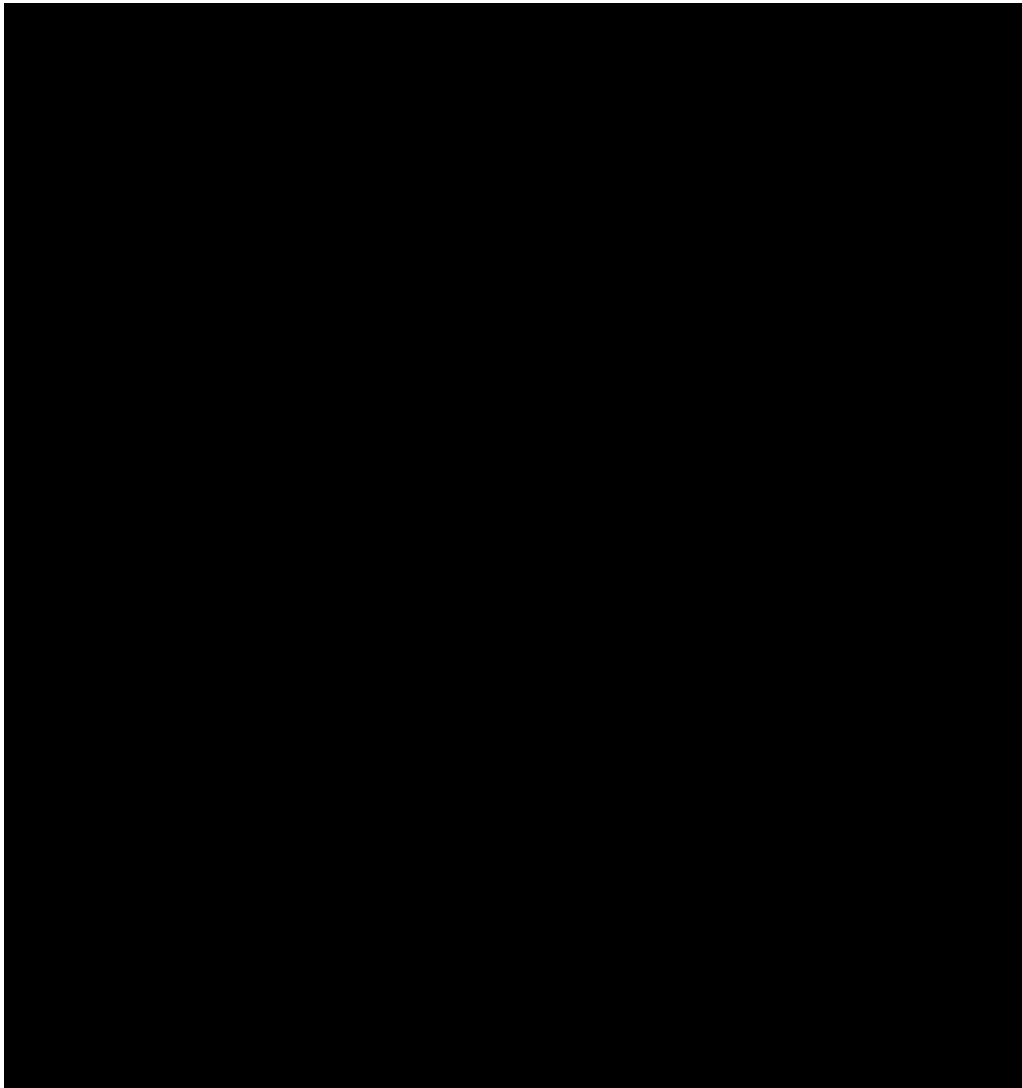


Figure 10-10 Address decoder circuit using 74LS138.

Problems(for lecture 9 and 10)

- 1) Name the technology used to fabricate the 8086 microprocessors.
- 2) What is the transistor count of the 8086?
- 3) Which pin is used as the NMI input on the 8086?
- 4) How much memory can the 8086 directly address?
- 5) How large is the I/O address space of the 8086?
- 6) How is minimum or maximum mode of operation selected?
- 7) Describe the difference between the minimum-mode 8086 system and the maximum-mode 8086 system.
- 8) Is the signal $M/\overline{IO}$ an input or output of the 8086?
- 9) Are the signals QS_0 and QS_1 produced in the minimum mode or maximum mode?
- 10) Does the 8086 have a multiplexed address/data or independent address and data busses?



- 11) What does status code $S_4S_3 = 01$ mean in terms of the memory segment being accessed?
- 12) Which output is used to signal external circuitry that a byte of data is available on the upper half of the 8086's data bus?
- 13) Which output is used to signal external circuitry in an 8086-based microcomputer that valid data is on the bus during a write cycle?
- 14) What signal does a minimum-mode 8086 respond with when it acknowledges and active interrupt request?
- 15) Which signals implement the DMA interface in a minimum-mode 8086 microcomputer system?
- 16) Identify the signal lines of the 8086 that are different for the minimum-mode and maximum-mode interfaces.
- 17) What status outputs of the 8086 are inputs to the 8288?
- 18) What maximum-mode control signals are generated by 8288?
- 19) What status code is output by the 8086 to the 8288 if a memory read bus cycle is taking place?
- 20) What command output becomes active if the status inputs of the 8288 are 100_2 ?
- 21) At what speeds are 8086s generally available?
- 22) How many clock states are in an 8086 bus cycle that has no wait states?
- 23) What is the duration of the bus cycle for a 5-MHz 8086 that is running at full speed and with no wait states?
- 24) What is an idle state?
- 25) What is a wait state?
- 26) If an 8086 running at 10 MHz performs bus cycles with two wait states, what is the duration of the bus cycle?
- 27) In which bank of memory in an 8086-based microcomputer are odd-addressed bytes of data stored? What bank select signal is used to enable this bank of memory?
- 28) List the memory control signals together with their active logic levels that occur when a word of data is written to memory address $A0000_{16}$ in a minimum-mode 8086 microcomputer system.
- 29) Draw the minimum-mode memory write bus cycle of the 8086.
- 30) Draw memory interface block diagram for minimum-mode 8086.
- 31) How many address lines must be decoded to generate five chip select signals?
- 32) How many 74LS373 chips used to latch the 8086's address lines and the $\overline{BHE}/S_7$ signal?



Lecture 11- Memory types and memory expansion

Memory provides the ability to store and retrieve digital information and it is one of the key elements of a microcomputer system. Previously; we indicated that the memory unit of the microcomputer is partitioned into a primary storage section and secondary storage section. The main differences between them are summarized in the table below:

Primary storage memory	Secondary storage memory
Used for working information, such as the instruction of the program currently being run and data that it is processing .	Used for storage of data, information, and programs that are not in use.
This part normally requires high speed operation but does not normally require very large storage capacity.	This part of the memory unit can be slow speed, but it requires very large storage capacity.
It is implemented with semiconductor memory devices, such as ROM , RAM and FLASH.	It is normally implemented with magnetic storage device, such as the floppy disk and hard disk drive.

Read Only Memory is one type of semiconductor memory device. It is most widely used in microcomputer systems for storage of the program that determines overall system operation. The information stored within a ROM integrated circuit is permanent (nonvolatile). Three types of ROM devices are in wide use today:

1. The mask programmable read only memory (ROM).
2. The one time programmable read only memory (PROM).
3. The erasable programmable read only memory (EPROM).

A large number of standard EPROM ICs are available today. The Table below lists the part number, bit densities, and byte capacities of nine popular devices.

EPROM	Density (bits)	Capacity (bytes)
2716	16K	2K× 8
2732	32K	4 K×8
27C64	64K	8 K×8
27C128	128K	16 K×8
27C256	256K	32 K×8
27C512	512K	64 K×8
27C010	1M	128 K×8
27C020	2M	256 K×8
27C040	4M	512 K×8

الجدول
للاطلاع فقط

Random Access Memory

RAM is similar to ROM in that its storage location can be accessed in a random order, but it is different from ROM in two important ways:

1. Data stored in RAM is not permanent.
2. RAM is volatile

Two types of RAMs are in wide use today:

Static RAM (SRAM): data remain valid as long as the power supply is not turned off.

Dynamic RAM (DRAM): to retain data in a DRAM, it is not sufficient just to maintain the power supply; we must periodically restore the data in each storage location (*Refreshing* the DRAM).

Table below list a number of standard static RAM ICs.

SRAM	Density (bits)	Organization
4361	64K	64K×1
4363	64K	16 K×4
4364	64K	8 K×8
43254	256K	64 K×4
43256A	256K	32 K×8
431000A	1M	128 K×8

الجدول
للاطلاع فقط

Memory expansion

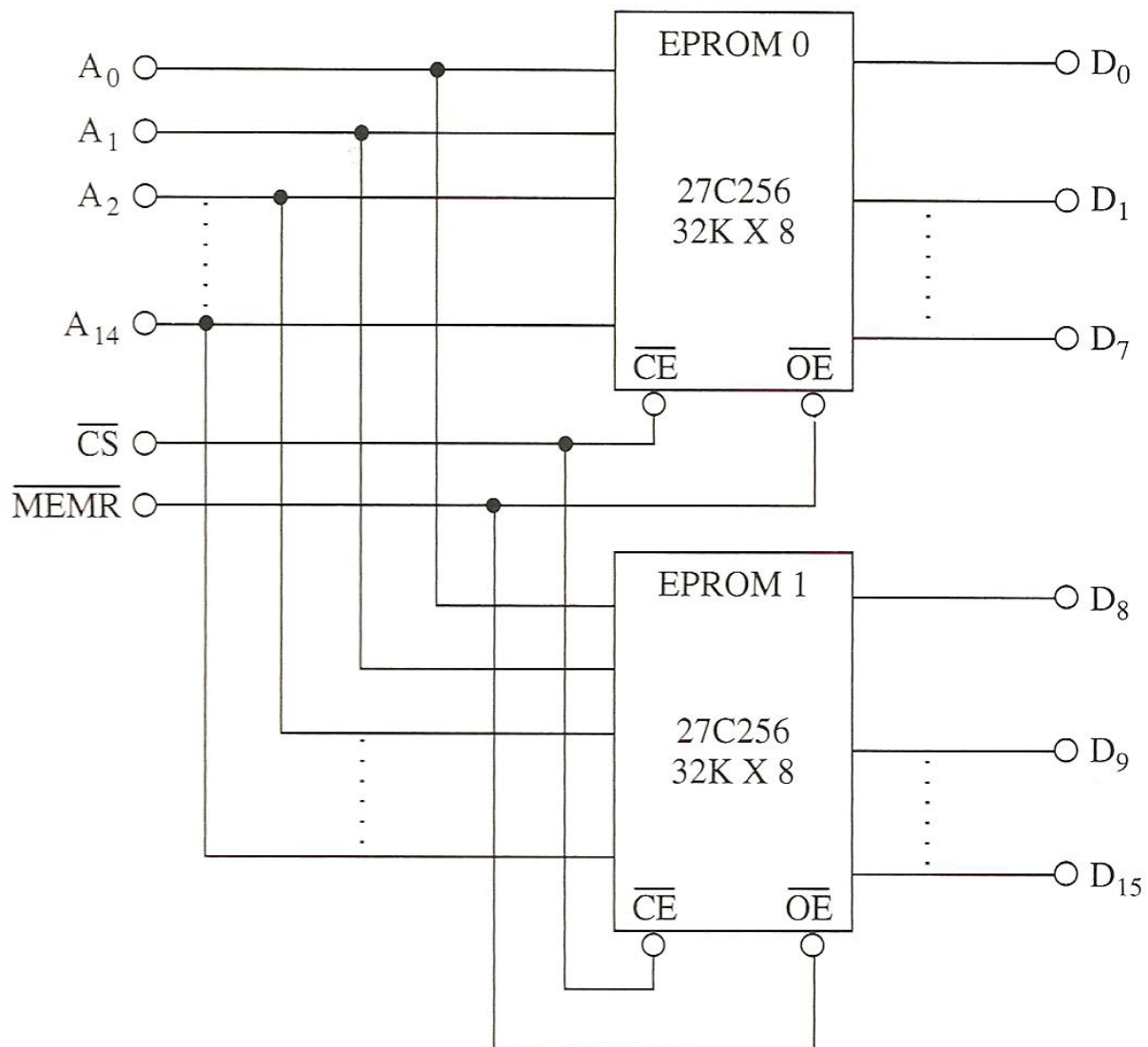
In many applications, the microcomputer system requirement for memory is greater than what is available in a single device. There are two basic reasons for expanding memory capacity:

1. The byte-wide length is not large enough
2. The total storage capacity is not enough bytes.

Both of these expansion needs can be satisfied by interconnecting a number of ICs.

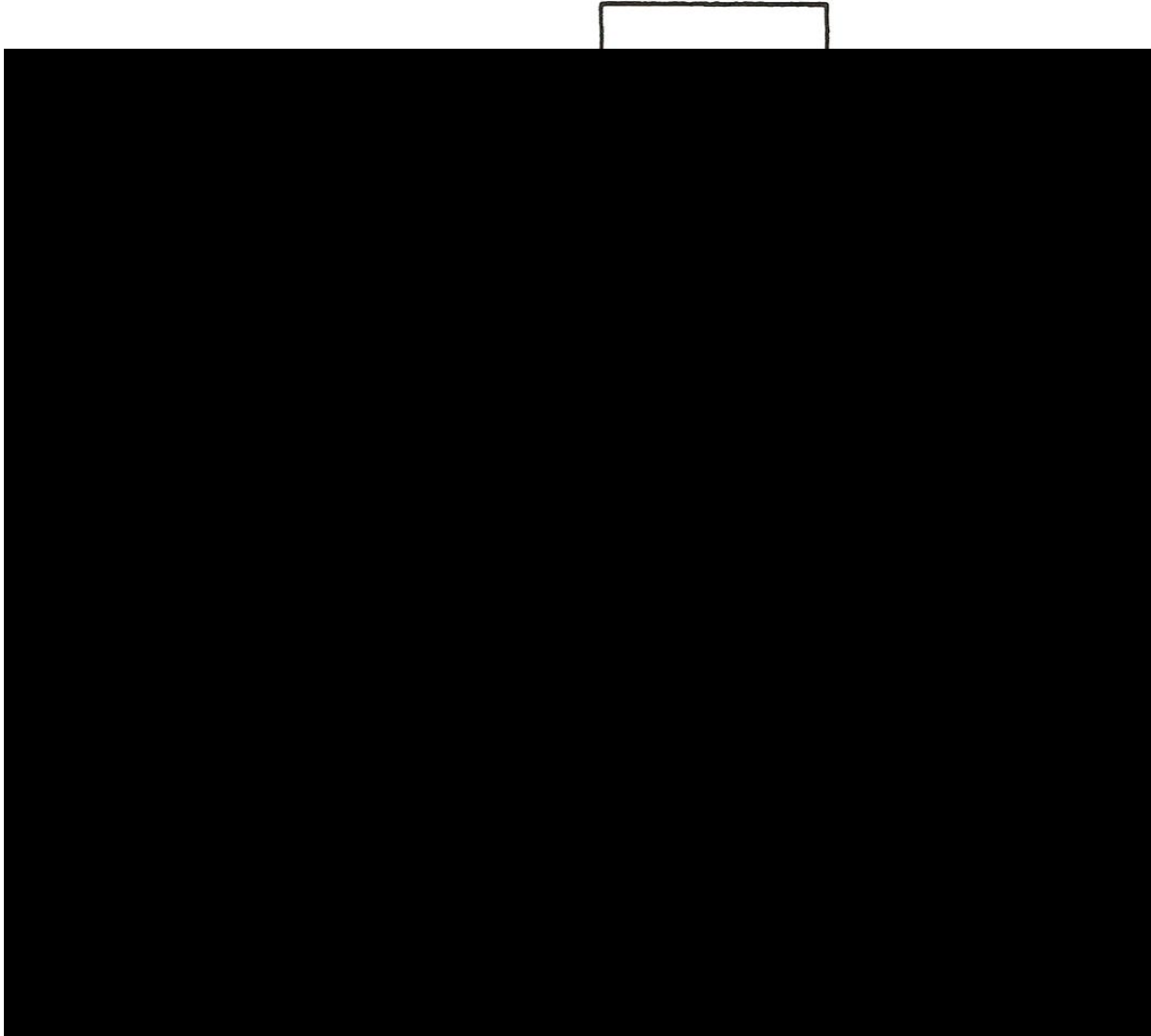
Example 1: show how to implement $32K \times 16$ EPROM using two $32K \times 8$ EPROM?

Solution:

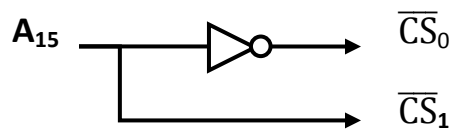


Example2: show how to implement $64K \times 8$ EPROM using two $32K \times 8$ EPROM?

Solution:

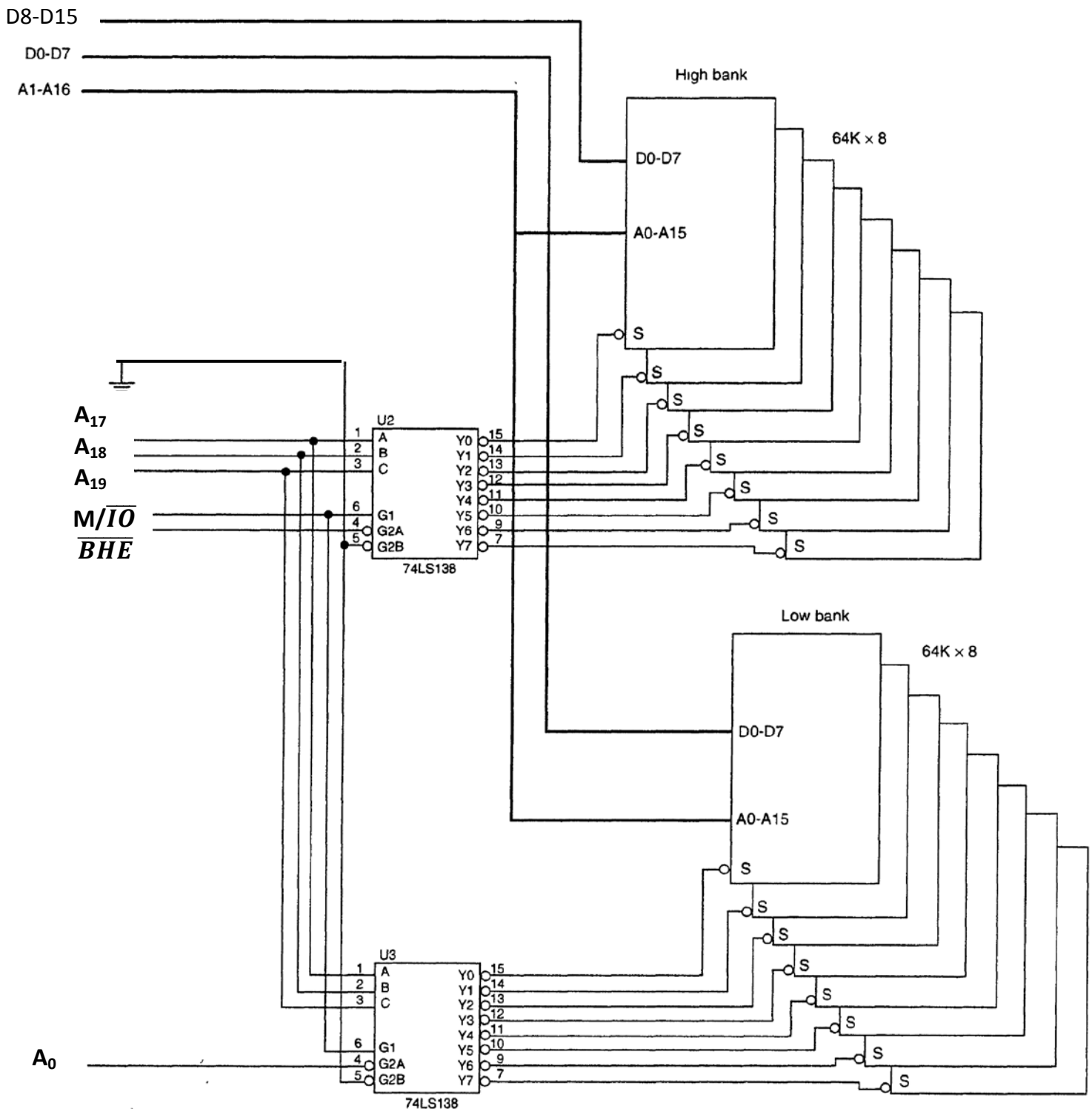


The $\overline{CS}_1$ and $\overline{CS}_0$ signals could be implemented as follow:



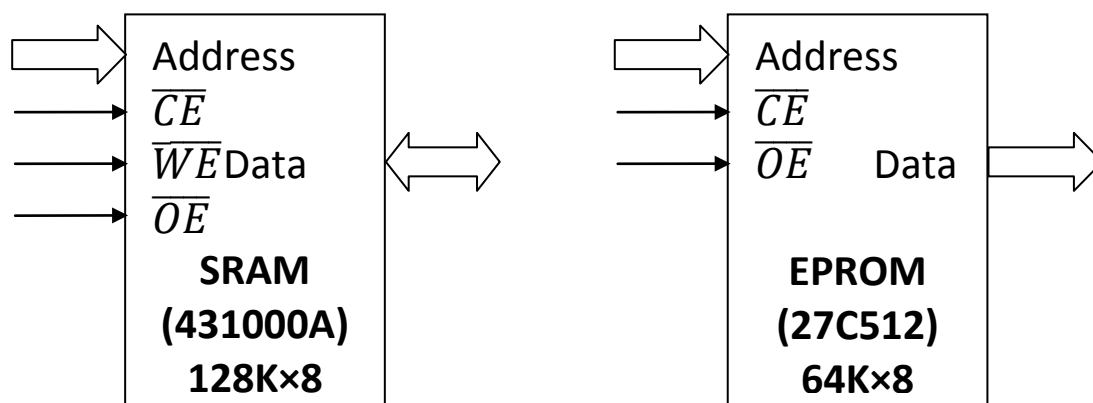


Example 3: Design a 8086 memory system consisting of 1Mbytes, Using $64K \times 8$ memory

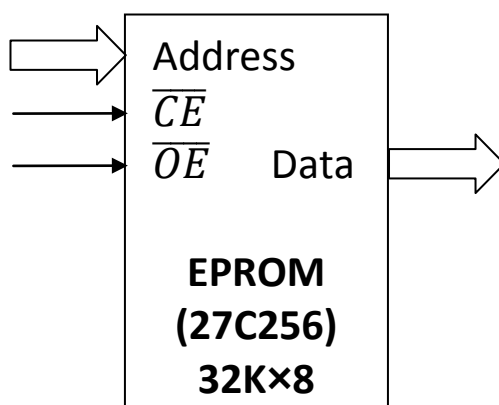




Example 4: Design 8086's memory system consisting of 512K bytes of RAM memory and 128K bytes of ROM use the devices in figure below. RAM memory is to reside over the address range 00000_H through 7FFFF_H and the address range of the ROM is to be A0000_H through BFFFF_H



Example 5: Design 8086's memory system consisting of 64K bytes of ROM memory, make use of the devices in figure below. The memory is to reside over the address range 60000_H through 6FFFF_H





lecture 12

I/O Interface Circuits

This lecture describes the IO interface circuits of an 8086-based microcomputer system.

The input/output system of the microprocessor allows peripherals to provide data or receive results of processing the data. This is done using I/O ports.

The 8086 microcomputers can employ two different types of input/output (I/O):

1. Isolated I/O.
2. Memory-mapped I/O.

1. Isolated input/output

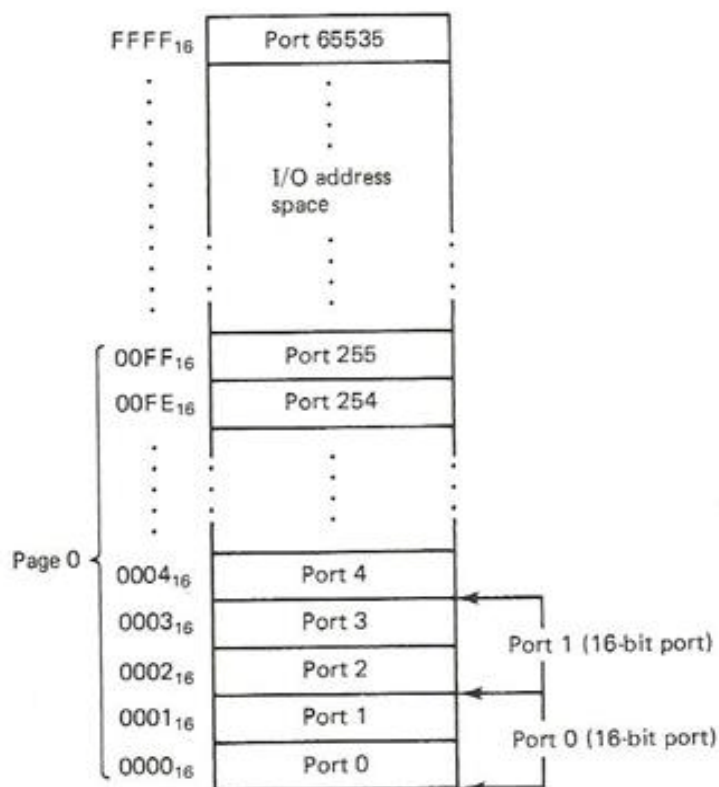
When using isolated I/O in a microcomputer system, the I/O devices are treated separate from memory.

As explained in lecture 2, the address space from a software point of view for the I/O ports is organized as bytes of data in the range 0000_{16} through $FFFF_{16}$.

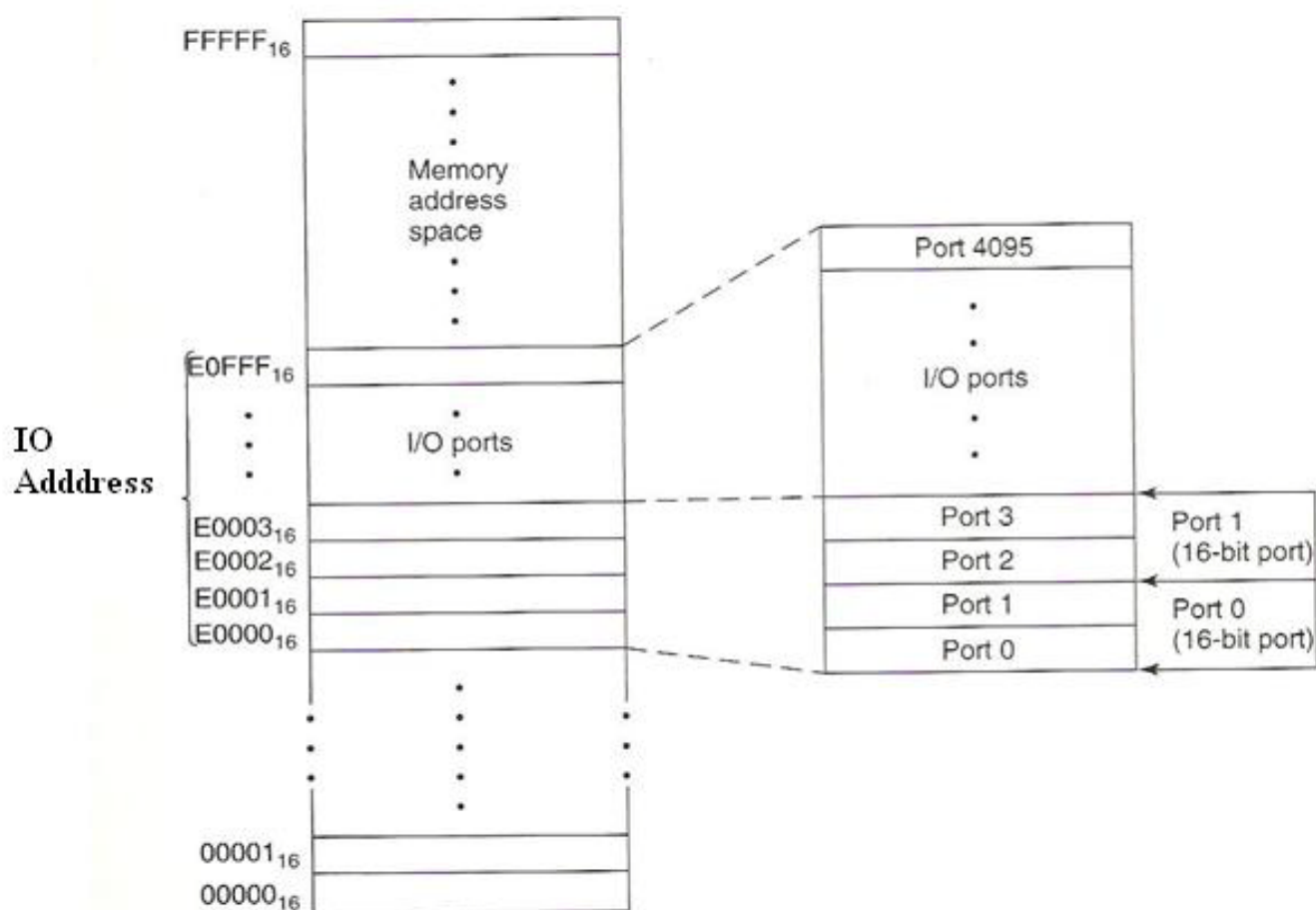
The part of the I/O address space from address 0000_{16} through $00FF_{16}$ is referred to as **Page 0** as shown in figure 12-1(a).

The way in which the MPU deals with input/output circuitry is similar to the way in which it interfaces with memory circuitry.

- There is an I/O interface circuitry for minimum mode. This interface also use the signals ALE , $\overline{BHE}$, $M/\overline{IO}$, $\overline{RD}$, $\overline{WR}$, $DT/\overline{R}$, and $\overline{DEN}$.
- There is an I/O interface circuitry for maximum mode. This interface uses the 8288 bus controller.
- Through this I/O interface, the MPU can input or output data in **bit**, **byte** or **word** (for the 8086).
- Unlike the memory interface, just the 16 least significant lines of the address bus (A_{15} through A_0) are used.
- The logic levels of signals A_0 and $\overline{BHE}$ determine whether data are input/output for an odd-addressed byte-wide port, even-addressed byte-wide port, or a word-wide port.
- Input/output operations are performed using special input and output instructions (shown in figure 12-2).
- There are two different forms of IN and OUT instructions:



(a)



(b)

Figure 12-1 (a) Isolated I/O ports (b) Memory-mapped I/O ports.



1. Direct I/O instructions:

- The address of the I/O port is specified as part of the instruction.
- Eight bits are provided for this direct address. For this reason, its value is limited to the address range form 00_{16} to FF_{16}
- This range is referred to as **Page 0** in the I/O address space.

2. Variable I/O instructions:

- The instructions use a 16-bit address that resides in the DX register within the MPU.
- The value in DX is not an offset.

Mnemonic	Meaning	Format	Operation
IN	Input direct Input indirect (variable)	IN Acc, Port IN Acc, DX	$(Acc) \leftarrow (Port)$ $(Acc) \leftarrow ((DX))$
OUT	Output direct Input indirect (variable)	OUT Port, Acc OUT DX, Acc	$(Port) \leftarrow (Acc)$ $((DX)) \leftarrow (Acc)$
where Acc = AL or AX			

Figure 12-2 Input/output instructions

Example 12-1: write a series of instructions that will output FF_{16} to an output port located at address $B000_{16}$ of the I/O address space.

Solution:

```

MOV DX, B000H
MOV AL, FF
OUT DX, AL

```

Example 12-2: Data are to be read from two byte-wide input ports at addresses AA_{16} and $A9_{16}$ and then output as a word to a word-wide output port at address $B000_{16}$. Write a series of instructions to perform this input/output operation.

Solution:

```

IN AL, AAH
MOV AH, AL
IN AL, A9H
MOV DX, B000H
OUT DX, AX

```



Input/Output Bus Cycles

The input/output bus cycles are essentially the same as those involved in the memory interface. Figure 12-3 show the output bus cycle of the 8086. It's similar to the write cycle except for the signal $M/\overline{IO}$.

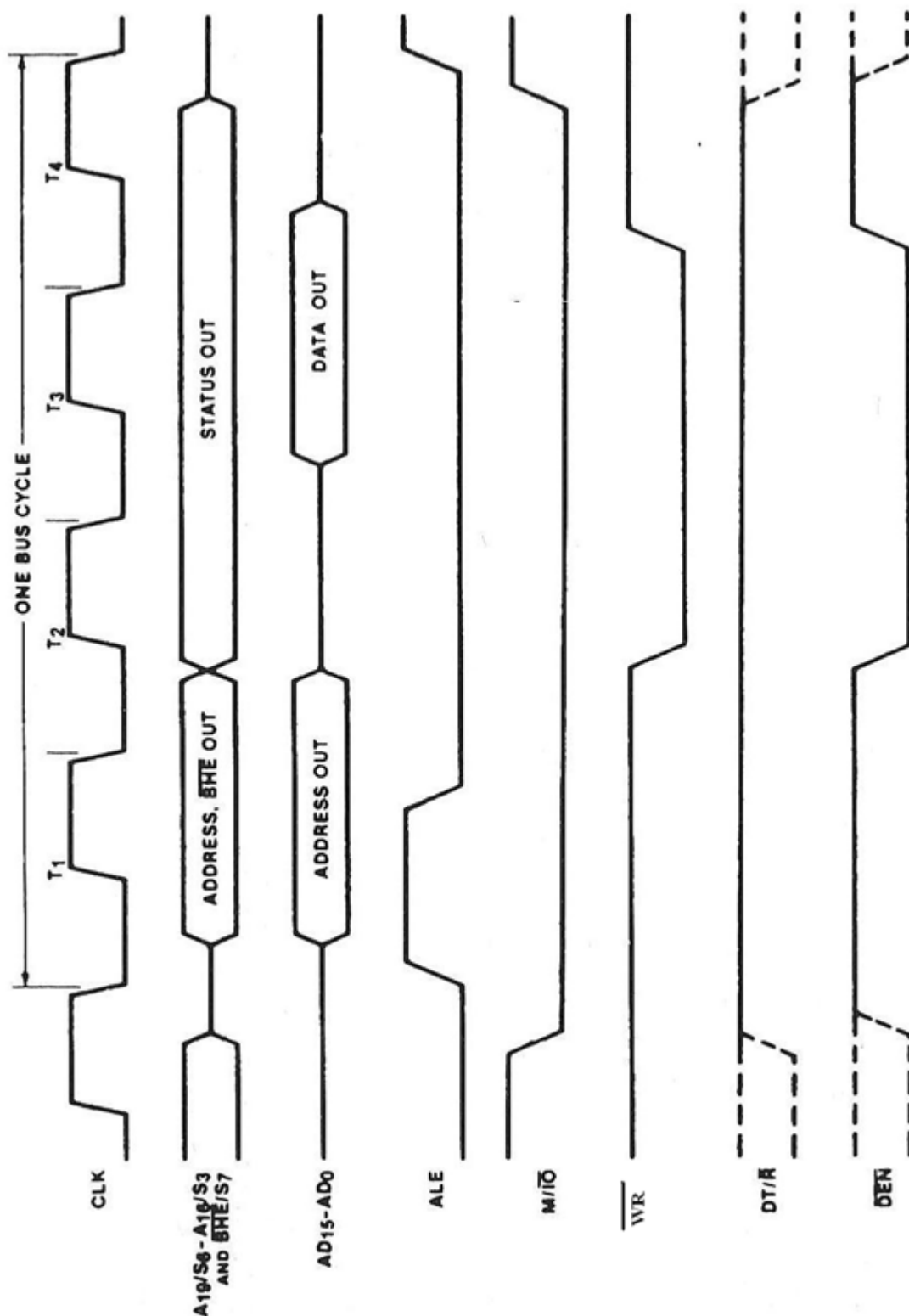


Figure 12-3 Output Cycle



Byte-Wide output ports using Isolated I/O

Here we explain the circuits that can be used to implement parallel output ports (8 bit) in microcomputer system employing isolated I/O.

Time Delay Loop and Blinking an LED at an Output Port

The circuit in Figure 12-4 show how to attach a LED to output port **O₇** of parallel **port 0**. The port address is 8000H, and the LED corresponds to **bit 7** of the byte of data that is written to port 0. The circuit use 74LS374 (edge clocked octal latch).

For the LED to turn on, **O₇** must be switched to logic 0, and it will remain on until this output is switched back to 1. The 74LS374 is not an inverting latch, therefore, to make **O₇** logic 0, simply write 0 to that bit of the octal latch.

Example 12-3: Write instruction sequence to make the LED (in Figure 12-4) blink.

Solution: we must write a program that first makes **O₇** logic 0 to turn on the LED, delays for a short period of time, and then switches **O₇** back to 1 to turn off the LED. This piece of program can run as a loop to make the LED continuously blink. This is done as follows:

```

                MOV DX, 8000H      ; Initialize address of port0
                MOV AL, 00H        ; Load data with bit 7 as logic 0
ON_OFF: OUT DX, AL                ; Output the data to port 0

                MOV CX, FFFFH      ; Load delay count of FFFFH
HERE:  LOOP HERE                  ; Time delay loop

                XOR AL, 80H        ; Complement bit 7 of AL
                JMP ON_OFF         ; Repeat to Output the new bit 7

```

Example 12-4: For figure 12-4, what is the I/O address of port 7 on the circuit? Assume all unused address bit are at logic 0.

Solution:

A15	A14	A13	A12		A4	A3	A2	A1	A0
1	0	0	0		0	1	1	1	0

the address is **800E₁₆**

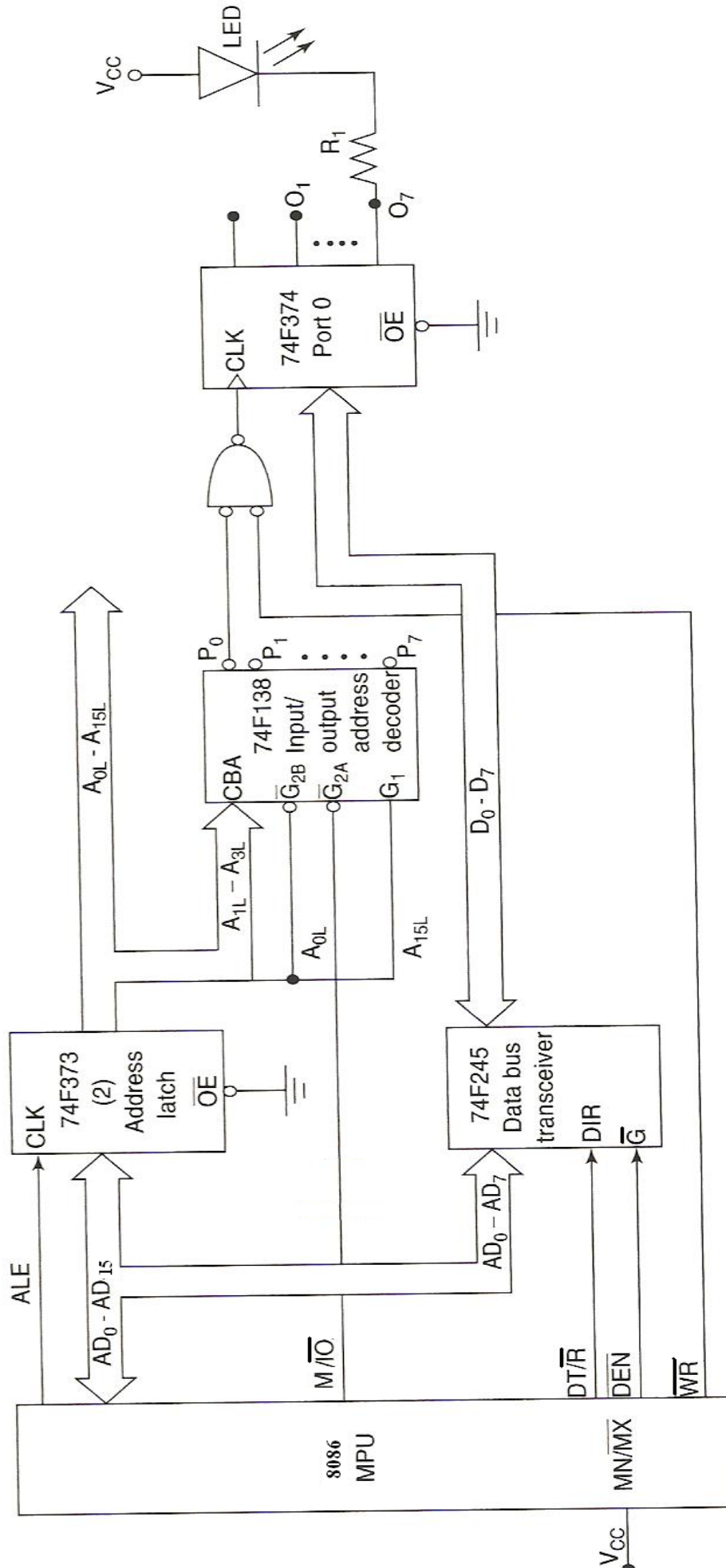


Figure 12-4: Driving an LED connected to an output port.



Byte-Wide output ports using Isolated I/O

Here we explain the circuits that can be used to implement parallel input ports (8 bit) in microcomputer system employing isolated I/O.

Polling the Setting of a Switch

The circuit in Figure 12-5 show how to attach a switch to input port **I₂** of parallel **port 0**. The port address is **8000H**, and the switch corresponds to **bit2** of the byte of data that is read from port 0. The circuit use 74LS244 (unidirectional octal buffer).

It is common practice to poll a switch like this with software waiting for it to close. The instruction sequence that follows will poll the switch at **I₂**:

```

                MOV CL, 03H
                MOV DX, 8000H
POLL_I2:       IN AL, DX
                SHR AL, CL
                JC POLL_I2
CONTINUE:      ...
                ...

```

If the switch is open, then bit 2 in AL is 1 and this value is shifted into CF. The program will still loop until the switch is closed.

If the switch closed, then the polling operation is complete and the instruction following the **JC** is executed.

Example 12-5: Write a sequence of instructions to read in the contents of ports 1 and 2 in the circuit shown in figure 12-5, and save them at consecutive memory addresses $A0000_{16}$ and $A0001_{16}$ in memory.

Solution:

```

MOV AX, A000H      ; set up the segment to start at A000H
MOV DS, AX
MOV DX, 8002H
IN AL, DX          ; input from port 1
MOV [0000H], AL    ; save the input at A0000H
MOV DX, 8004
IN AL, DX          ; input form port 2
MOV [0001H], AL    ; save the input at A0001H

```

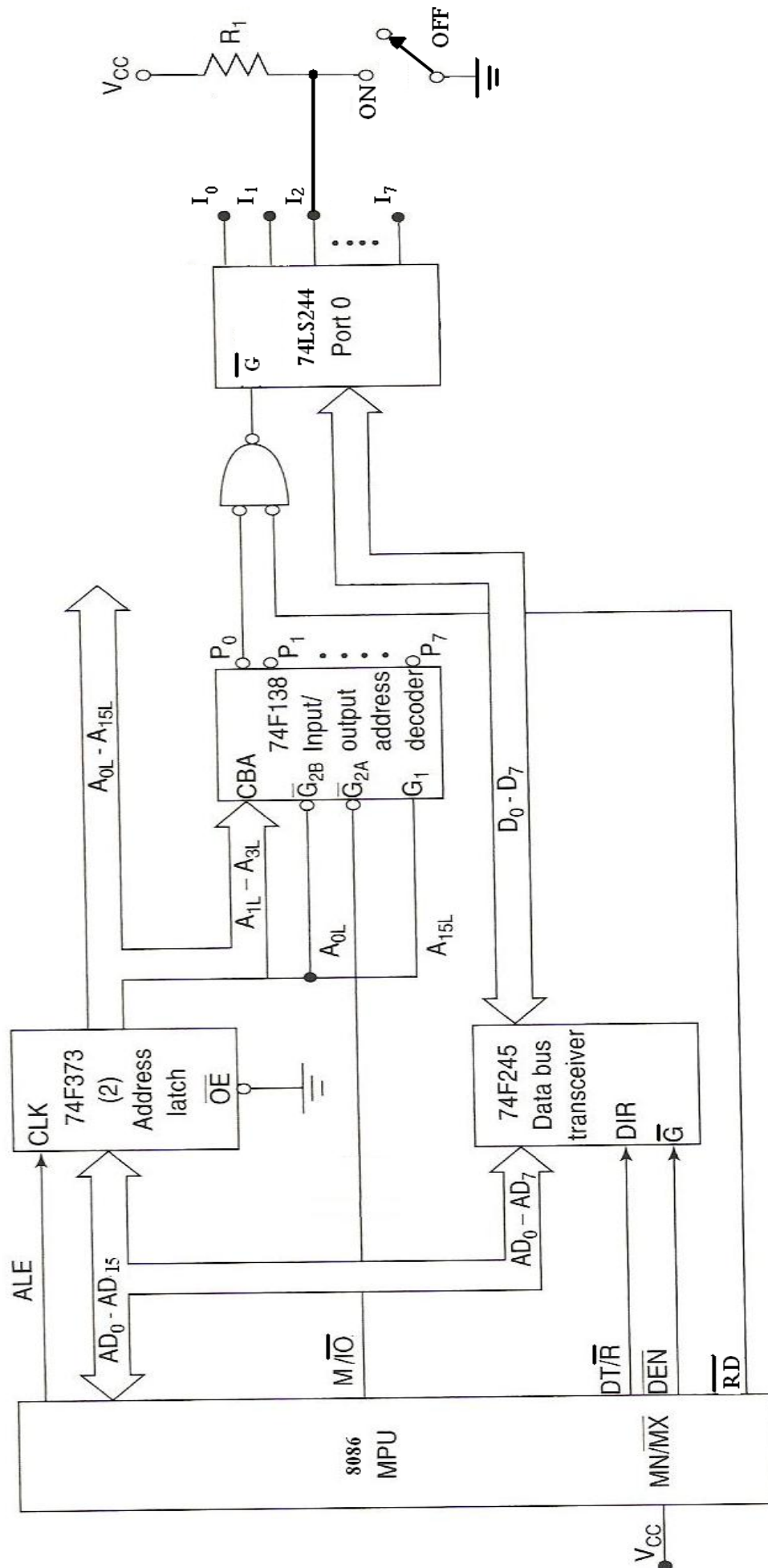


Figure 12-5: reading the setting of switch connected to an input port.



2. Memory-Mapped Input/Output

I/O devices can be placed in the memory address space of the microcomputer as well as in the independent I/O address space. In this case, the MPU looks at the I/O port as though it is a storage location in memory.

For example, in Figure 12-1(b) the 4096 memory addresses in the range from $E0000_{16}$ through $E0FFF_{16}$ are assigned to I/O devices.

Example 12-6: Compare between isolated I/O and memory-mapped I/O.

Solution:

Isolated I/O	Memory-mapped I/O
Use only the special input/output instructions. (IN and OUT)	All memory instructions and addressing modes are available to perform I/O operation. (MOV, AND XCHG, SUB)
Faster because I/O instructions is specifically designed to run faster than memory instructions	Slower because memory instructions execute slower than the special I/O instructions
The memory address space is not affected	Part of the memory address space is lost



Problems

1. Write a program to input data from input port at address **1AH** using direct I/O instruction. Write another program to perform the same operation using indirect I/O instruction.
2. If an 8086 running at **10MHz** inserts two wait states into all I/O bus cycles, what is the duration of the bus cycle to output a word of data to a word-wide port at I/O address **1A1₁₆**?
3. Write sequence of instructions that inputs the byte of data from input ports at I/O addresses **A000₁₆** and **B000₁₆**, adds these values together, and saves the sum in memory location **IO_SUM**.
4. Write a sequence of instructions that will input the contents of the input port at I/O address **B0₁₆** and jump to the beginning of a service routine identified by the label **ACTIVE_IN** if the least significant bit of the data is **1**.
5. Draw 8086 microprocessor **Input Cycle**.
6. What is the address of output port 3 in the circuit shown in figure 12-4?
7. What is the address of input port 6 in the circuit shown in figure 12-5?
- ★ 8. Draw an 8086 isolated I/O interface circuit that interfaces **three** switches at **bits 0, 2, 6** of input port at address **2002₁₆** and **one** LED at **bit 5** of output port at address **2004₁₆**. Then write an instruction sequence will continuously check the value of the three switches. The output LED will **turn on** only if the three switches are closed (logic 0).
- ★ 9. Draw an 8086 isolated I/O interface circuit that interfaces **two** switches at **bit 0** of input port at address **0002₁₆** and **bit 0** of input port at address **0004₁₆**, and **one** LED at output port of address **0004₁₆**. Then write an instruction sequence will continuously check the values of the two switches. The output LED will **turn on** only if the two switches are different.
- ★ 10. Repeat problem 8 using memory-mapped I/O.
- ★ 11. Repeat problem 9 using memory-mapped I/O.

-----End-----



lecture 13

82C55A Programmable Peripheral Interface

The 82C55A is LSI peripheral designed to permit easy implementation of parallel I/O in the 8086 microcomputer systems. It provides a flexible parallel interface, which includes features such as single-bit, 4-bit, and byte-wide input and output ports; Level-sensitive inputs; latched outputs; strobed inputs or outputs; and strobed bidirectional input/output these features are selected under software control. The 82C55 is shown in figure below:

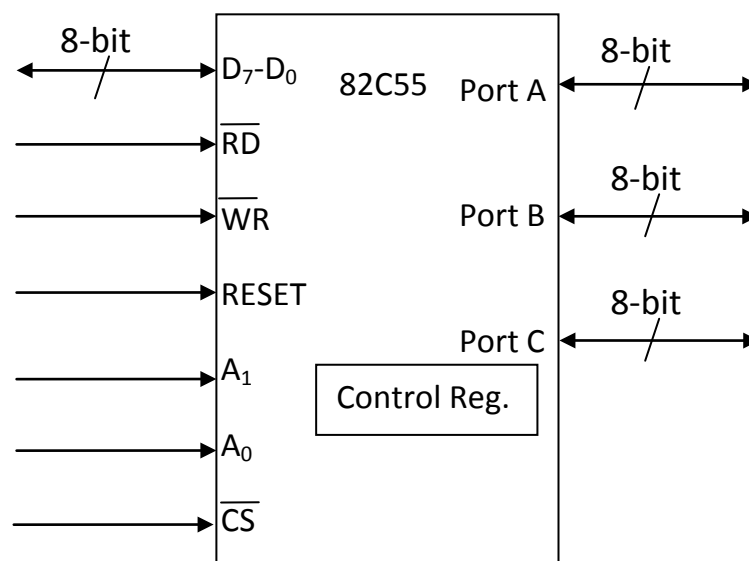


Figure 13-1 the 82C55

The source or destination register within the 82C55A is selected by a 2-bit register select code. The MPU must apply this code to the register-select inputs A_0 and A_1 in the 82C55A as shown in table below:

A_1A_0	82C55 register
00	A
01	B
10	C
11	Control register

Two other signals are shown on the microprocessor interface side of the block diagram. They are the reset (RESET) and chip-select ($\overline{CS}$) inputs. $\overline{CS}$ must be logic 0



during all read or write operations to the 82C55A. It enables the 82C55A's microprocessor interface circuitry for an input or output operation.

The bits of control register and their control functions are shown in figure 13-2.

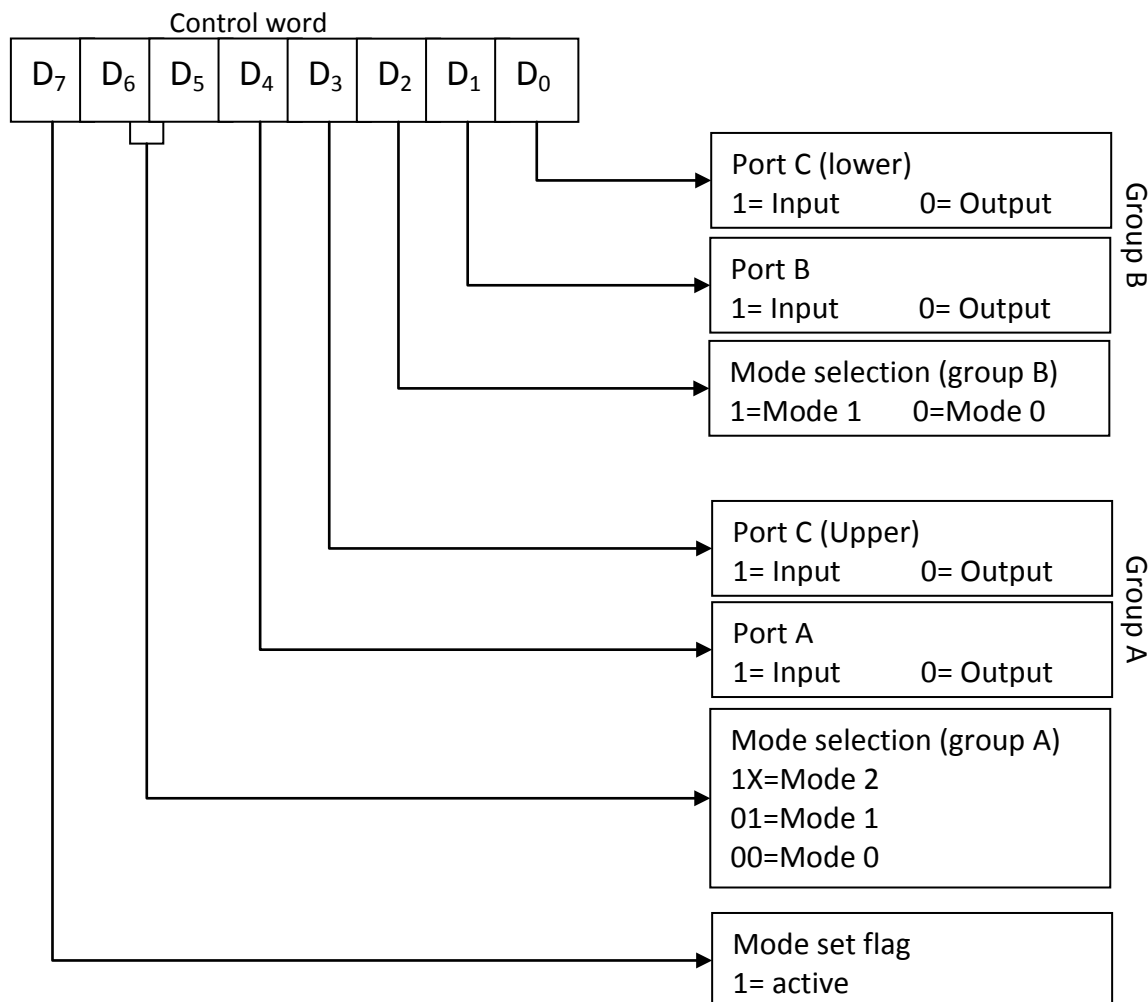


Figure 13-2 control word for the 82C55 PPI

Example 13-1: What is the mode and I/O configuration for ports A, B, C of an 82C55 after its control register is loaded with 82_{16} ?

Solution:

82_{16}	=	D7	D6	D5	D4	D3	D2	D1	D0
		1	0	0	0	0	0	1	0

$$D_7 = 1$$

then mode set flag is active

$$D_6 D_5 = 00$$

then Group A (A and C upper) is in mode 0

$$D_4 = 0$$

A is an output

$$D_3 = 0$$

C upper is an output

$$D_2 = 0$$

then Group B (B and C lower) is in mode 0

$$D_1 = 1$$

B is an input

$$D_0 = 0$$

C lower is an output



Example 13-2: Write down 82C55 control word that set Port A, Port B and Port C lower as input in mode 0, and set Port C upper as output in mode 0.

Solution:

Mode set flag is active	then $D_7 = 1$
then Group A (A and C upper) is in mode 0	$D_6 D_5 = 00$
A is aninput	$D_4 = 1$
C upper is an output	$D_3 = 0$
then Group B (B and C lower) is in mode 0	$D_2 = 0$
B is an input	$D_1 = 1$
C lower is aninput	$D_0 = 1$

Control word = 93H

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	1	0	0	1	1

Example 13-3: In 8086's8-bit isolated I/O system, an 82C55 PPI is connected so that the address of A, B, C ports, and Control register are $4D08_{16}$, $4D09_{16}$, $4D0A_{16}$ and $4D0B_{16}$ respectively.

- Draw the circuit diagram.
- Write program to set Register A, B as input and Register C as output (all in mode 0). Then continuously receive two unsigned number from Registers A and B, compare them and output the larger to Register C.

Example 13-4:Use the 8086 microprocessor as a level controller for the ON-OFF process shown in figure 13-3. The microprocessor monitors the level by checking status of level sensors (LS1 and LS2) and controls valve actuators (V1 andV2) to sustain the level between LS1 and LS2.

- Draw the hardware interface circuit.
- Write the control algorithm for the closed loop control in assembly language.

Example 13-5: Repeat **Example 13-3** using memory-mapped IO.

Example 13-6: Repeat **Example 13-4** using 82C55 PPI.

Example 13-7: Repeat **Example 13-4** using 82C55 PPI and memory-mapped IO.

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	1	0	0	1	0

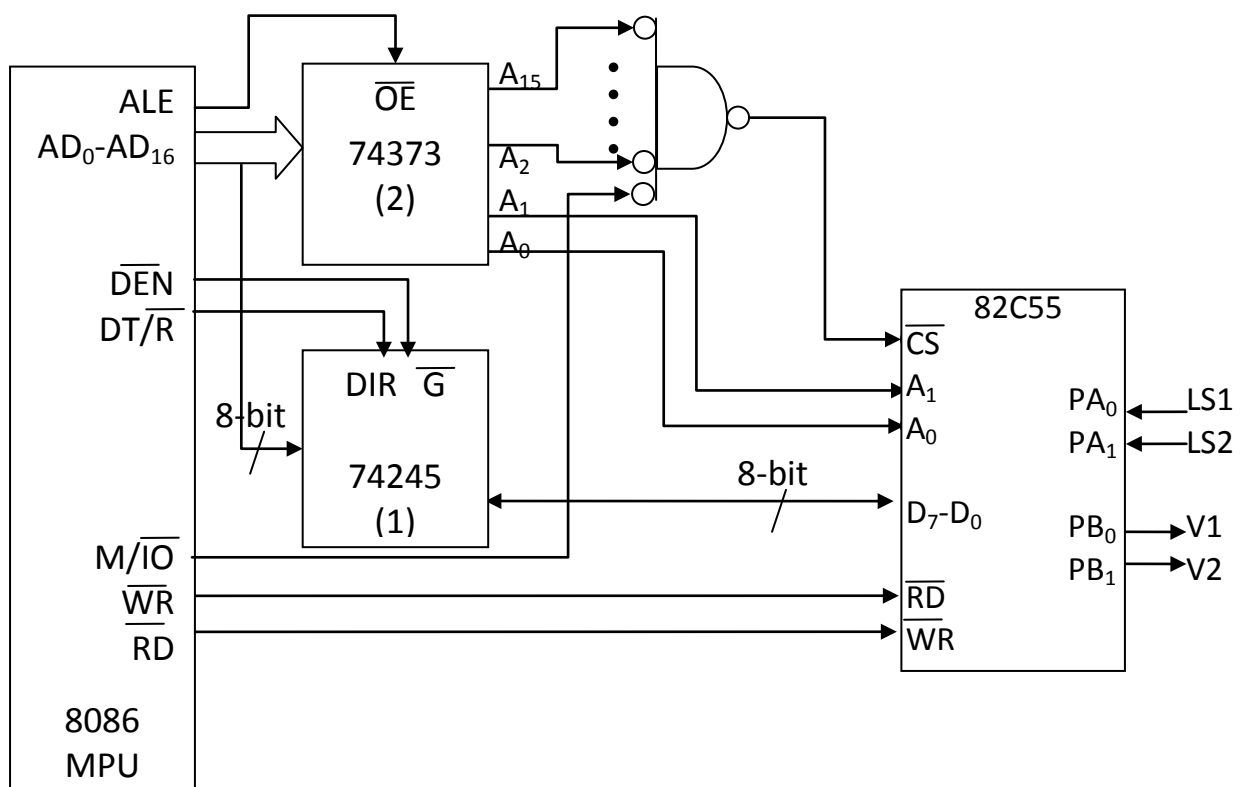


البرنامج

```

MOV AL, 92H
MOV DX, 4D0BH
OUT DX, AL
again: MOV DL, 08H      (because DH is the same)
      IN AL, DX
      MOV BL, AL
      INC DL
      IN AL, DX
      CMP AL, BL
      JNC no_exchange
      MOV AL, BL
no_exchange: INC DL
            OUT DX, AL
            MOV CX, FFFFH
delayloop: DEC CX
            JNZ delayloop
            JMP again
    
```

Solution of Example 13-6:



Let address of Port A =0000H
 address of Port B =0001H
 address of Port C =0002H
 address of control register =0003H



Let LS1 connected to PA₀
LS2 connected to PA₁
V1 connected to bit PB₀
V2 connected to bit PB₁

Then Port A is input , Port B is output and Port C not used.
Control word = 90H

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	1	x	0	0	x

Values of V1 and V2 is determined as shown in table below:

LS2	LS1	V2	V1	notes
0	0	1	0	Fluid is above LS2 and above LS1
0	1	0	0	Fluid is above LS2 and Below LS1
1	0	X	X	Fluid is Below LS2 and above LS1 (impossible)
1	1	0	1	Fluid is Below LS2 and Below LS1

البرنامج (الطريقة الاولى)

```

again:    MOV AL, 90H          (prepare control word)
          OUT 03H, AL         (set the control register)
          IN AL, 00H          (read sensors)
          MOV BL, AL
          NOT AL
          AND AL, 01H
          SHL AL, 01H
          AND BL, 02H
          SHR BL, 01H
          OR AL, BL
          OUT 01H, AL         (set valves)
          MOV CX, FFFFH
delayloop: DEC CX             (use delay counter)
          JNZ delayloop
          JMP again            (repeat forever)

```

البرنامج (طريقة اخرى)

```

again:    MOV AL, 90H          (prepare control word)
          OUT 03H, AL         (set the control register)
          IN AL, 00H          (read sensors)
          CMP AL, 00H
          JZ case0
          CMP AL, 01H
          JZ case1
          CMP AL, 03H(binary =0000 0011)
          JZ case3

```



```

continue:      OUT 01H, AL          (set valves)
delayloop:    DEC CX              (use delay counter)
                  JNZ delayloop
                  JMP again         (repeat forever)

case0:         MOV AL, 02H
                  JMP continue
case1:         MOV AL, 00H
                  JMP continue
case3:         MOV AL, 01H
                  JMP continue

```

Seven-segment display Interface

Seven-segment display used to display number (and letters). Seven segment display labeling is shown in figure 13-4

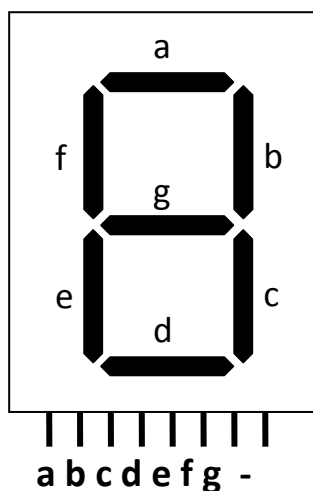


Figure 13-4 Seven-segment display

For example to display number (**9**) the bit must set as follow:

a	b	c	d	e	f	g	
1	1	1	1	0	1	1	x

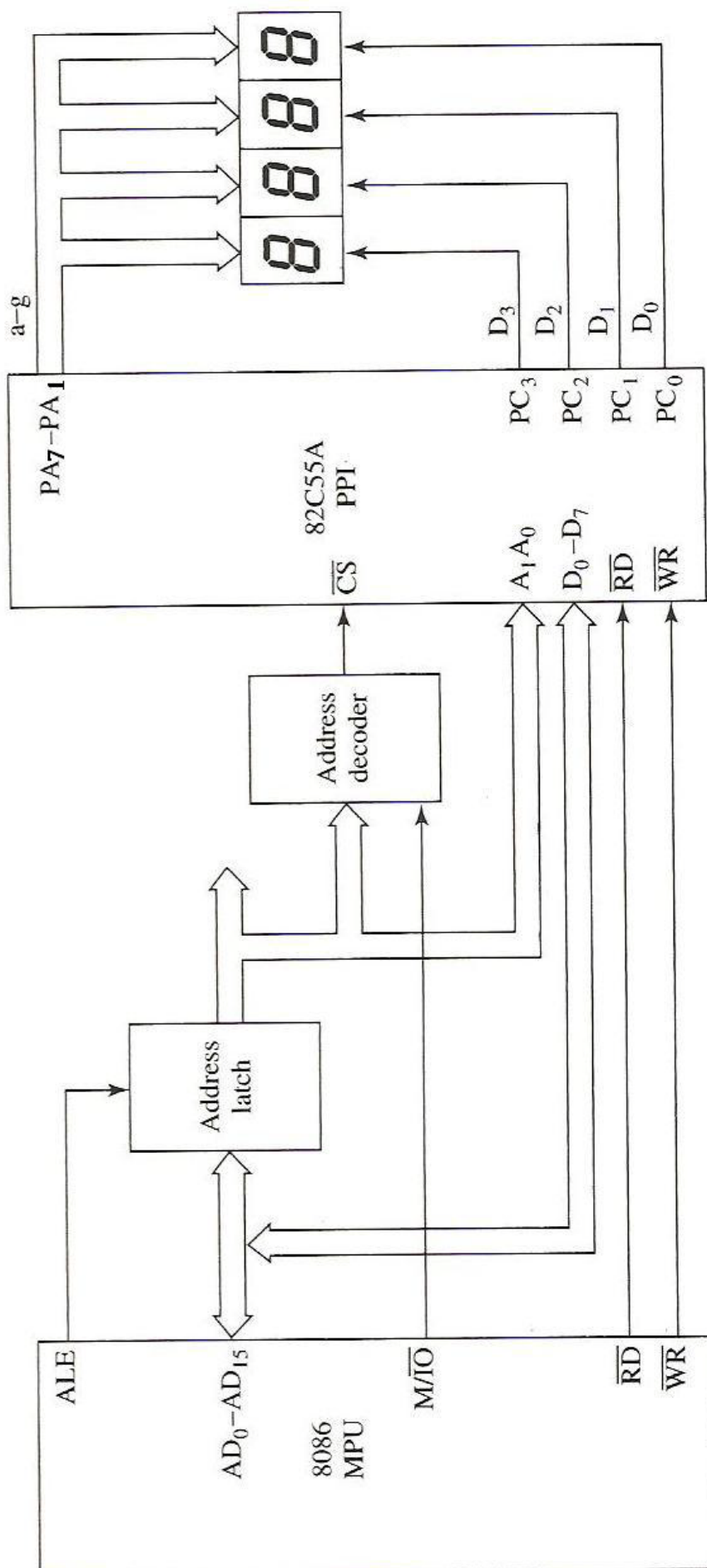


Figure 13-5 Four -digit Seven-segment display interface to 8086 microprocessor using 82C55 PPI

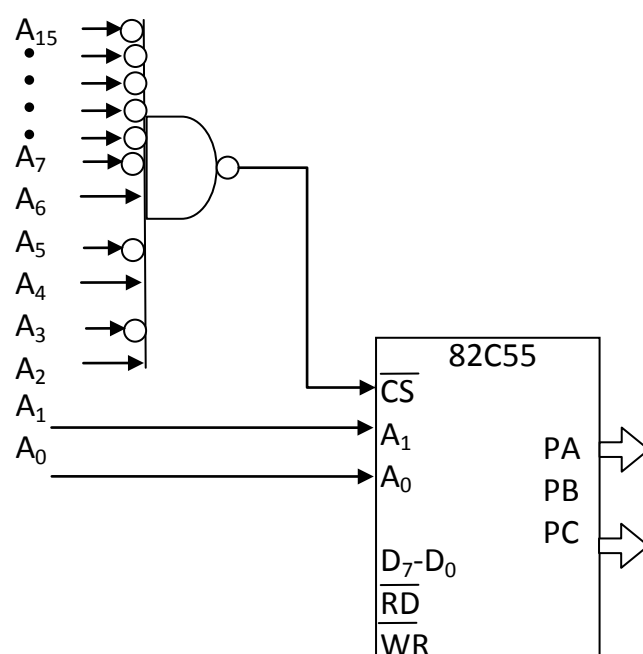


If the byte **F6**₁₆ (or **F7**₁₆ because bit D₀ is neglected) is written to address of the seven-segment display then it will display number nine.

Example 13-8:Figure 13-5 shows an interface of **four-digitseven-segment** numeric display. The circuit use Port A to output value to the display, and Port C lower to select which digit is active.

- Draw the detail of the **address decode** circuit that make the address of port A is 0054₁₆, address of port C is 0056₁₆ and address of control register is 0057₁₆.
- Write program to display the word **PASS**.Note that the program must continuously set each one of the four digits because they are multiplexed.

Solution



Control word =80H

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	X	0	X	0

Pcharacter : CE H

a	b	c	d	e	f	g	
1	1	0	0	1	1	1	x

Acharacter : EE H

a	b	c	d	e	f	g	
1	1	1	0	1	1	1	x

Scharacter :B6 H

a	b	c	d	e	f	g	
1	0	1	1	0	1	1	x



البرنامج

```
MOV AL, 80H
OUT 57H, AL          (set the control register)
```

```
again: MOV AL, 08H
        OUT 56H, AL   (enable fourth digit)
        MOV AL, CEH
        OUT 54H, AL   (display character P)
```

```
        MOV AL, 04H
        OUT 56H, AL   (enable third digit)
        MOV AL, EEH
        OUT 54H, AL   (display character A)
```

```
        MOV AL, 02H
        OUT 56H, AL   (enable second digit)
        MOV AL, B6H
        OUT 54H, AL   (display character S)
```

```
        MOV AL, 01H
        OUT 56H, AL   (enable first digit)
        MOV AL, B6H
        OUT 54H, AL   (display character S)
```

```
JMP again
```

No delay in the program

Example 13-9: repeat example 13-8 using memory-mapped IO.

Example 13-10: Depending on circuit and addresses of example13-8:

- Write program that display the **16-bit** number at memory location $0450C_{16}$.
- Write program that display a sequence counter that count from 0 to 9 on the first digit only. (Note: turn off the other unused digits and use delay between counts).

Note: you may need to store the display code for each number in a **lookup table** (use the XLAT instruction and set table start at address 00080H).

Solution: (Home Work)!!!!



lecture 14

8086 Interrupt Types and Interface

14.1 Interrupt Mechanism Types, and Priority

Interrupts provide a mechanism for quickly changing program environment. Transfer of program control is initiated by the occurrence of either an event to the microprocessor or an event in its external hardware.

The 8088 and 8086 microcomputers are capable of implementing any combination of up to 256 interrupts. As Fig. 11-1 shows, they are divided into five groups.

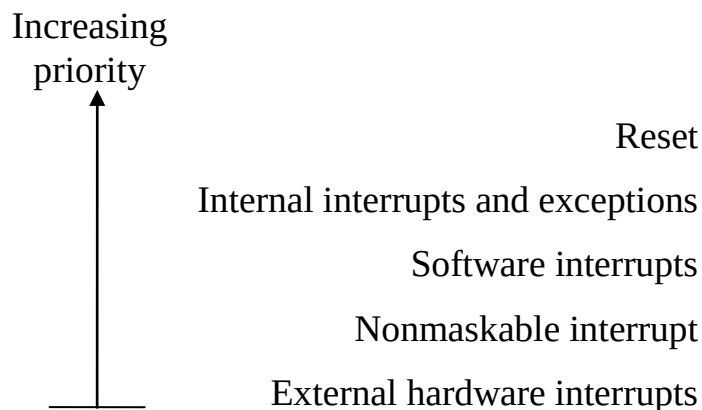


Figure 14.1 Types of interrupts and their priority

The user defines the function of the external hardware, software, and nonmaskable interrupt. For instance, hardware interrupts are often assigned to devices such as the keyboard, printer, and timers. On the other hand, the functions of the internal interrupts and reset are not user defined. They perform dedicated system functions.

An example of a high-priority service routine that should not be interrupted is that for a power failure. Once initiated, this routine should be quickly run to completion to assure that the microcomputer goes through an orderly power-down. A keyboard should also be assigned to a high-priority interrupt. This will assure that the keyboard buffer does not get full and lock out additional entries. On the other hand, devices such as the floppy disk or hard disk controller are typically assigned to a lower priority level.

Figure 14.2 Interrupt vector table of the 8086.



Looking at this table, we see that it contains 256 address pointers (vectors). Which are identified as vector 0 through vector 255. That is, one pointer corresponds to each of the interrupt types 0 through 255. These address pointers identify the starting location of their service routines in program memory.

Note in Fig. 11-2 that the pointer table is located at the low-address end of the memory address space. It starts at address 00000_{16} and ends at $003FE_{16}$. This represents the first 1Kbytes of the memory.

Each of the 256 pointers requires two words (4 bytes) of memory and is always stored at an even-address boundary.

For example, the offset and base address for type number 255, IP_{255} and CS_{255} , are stored at word addresses $003FC_{16}$ and $003FE_{16}$, respectively. When loaded into the MPU, it points to the instruction at $CS_{255}:IP_{255}$.

Looking more closely at the table in Fig. 11-2, we find that the first 31 pointers either have dedicated functions or are reserved. The next 27 pointers, 5 through 31, represent a reserved portion of the pointer table and should not be used. The remainder of the table, the 224 pointers in the address range 00080_{16} through $003FF_{16}$, is available to the user for storage of software or hardware interrupt vectors.

Example 14-1: At what address are CS_{50} and IP_{50} stored in memory?

Solution:

$$\text{Address} = 4 \times 50 = 200$$

and expressing it as a hexadecimal number results in

$$\text{Address} = C8_{16}$$

Therefore, IP_{50} is stored at $000C8_{16}$ and CS_{50} at $000CA_{16}$.

14.3 Interrupt Instructions

A number of instructions are provided in the instruction set of the 8086 microprocessors for use with interrupt processing. Figure 11.3 lists these instructions.



Mnemonic	Meaning	Format	Operation	Flags affected
CLI	Clear interrupt flag	CLI	$0 \rightarrow (IF)$	IF
STI	Set interrupt flag	STI	$1 \rightarrow (IF)$	IF
INT n	Type n software interrupt	INT n	$(Flags) \rightarrow ((SP)-2)$ $0 \rightarrow TF, IF$ $(CS) \rightarrow ((SP)-4)$ $(2+4*n) \rightarrow CS$ $(IP) \rightarrow ((SP)-6)$ $(4*n) \rightarrow (IP)$	TF, IF
IRET	Interrupt return	IRET	$((SP)) \rightarrow (IP)$ $((SP)+2) \rightarrow (CS)$ $((SP)+4) \rightarrow (Flags)$ $(SP)+6 \rightarrow (SP)$	All
INTO	Interrupt on overflow	INTO	INT 4 steps	TF , IF
HLT	Halt	HLT	Wait for an external interrupt or reset to occur	None
WAIT	Wait	Wait	Wait for $\overline{TEST}$ input to go active	None

Figure 14-3 Interrupt instructions.

-

- **STI** enables the external interrupt request (INTR) input for operation by setting IF, while **CLI** disable the external interrupt input by resetting IF.
- **INT n** instruction is used to initiate a vectored call of a subroutine.

For example: **INT 50** initiates execution of a subroutine whose starting point is identified by vector 50 in the pointer table (in Figure 11.2).

It also :

1. saves the flag register on the stack,
2. saves the old program context on the stack,
3. and clears TF and IF.

- **IRET** instruction must be included at the end of each interrupt service routine.
- **INTO** is the *interrupt-on-overflow* instruction. This instruction must be included after arithmetic instructions that can result in an overflow condition, such as divide. It tests the overflow flag, and if the flag is found to be set, a **type 4** internal interrupt is initiated.

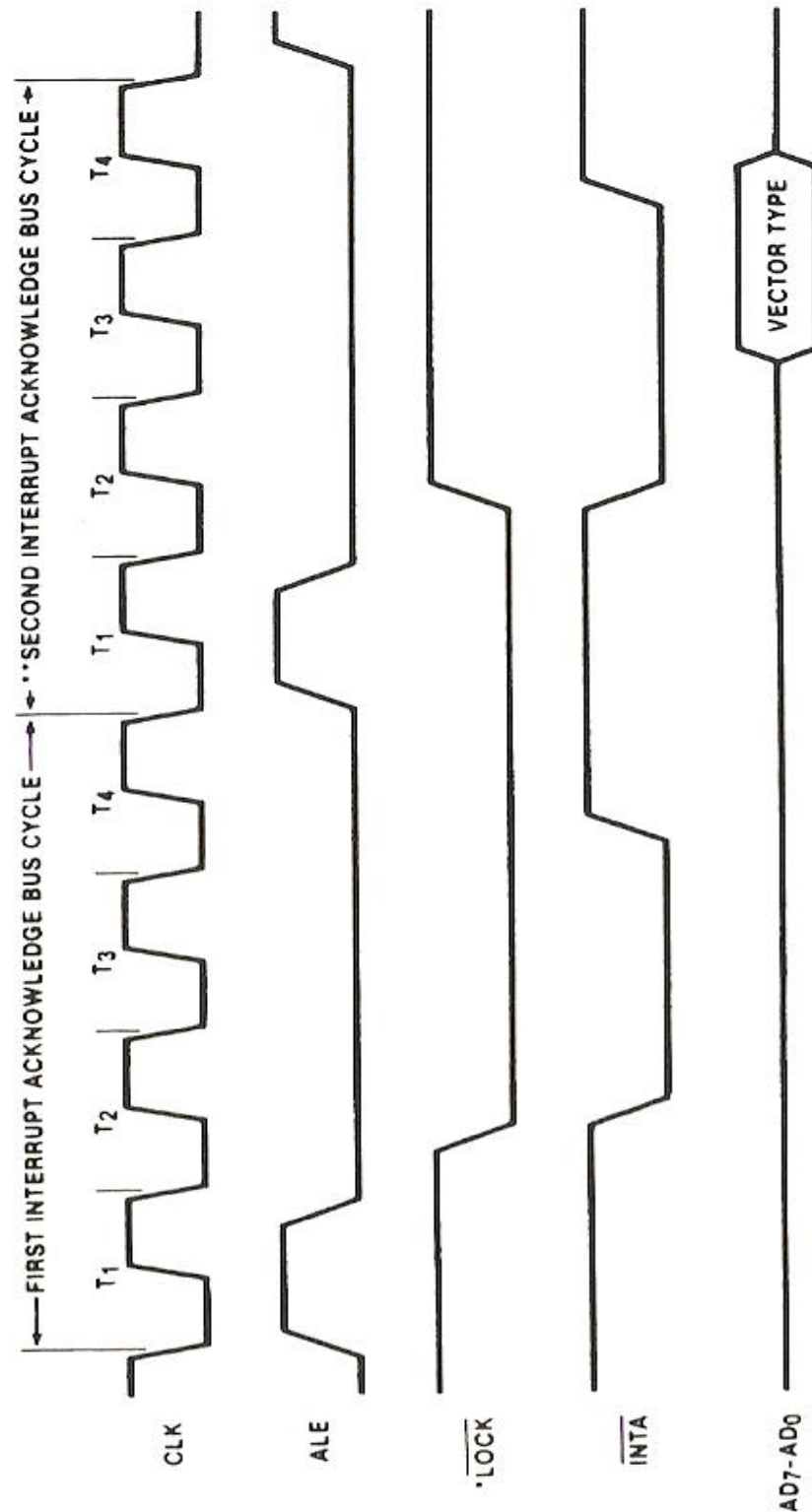


Figure 14-4 Interrupt acknowledgment cycle

11.4 External Hardware Interrupt Interface Signals

- When an interrupt request has been recognized on the NMI pin, the 8086 initiate **type 2** interrupt (CS₂:IP₂).
 - It cannot be masked by IF.
 - The NMI input is positive edge triggered. Therefore, a request for service is automatically latched internal to the MPU.



- When an interrupt request has been recognized on the **INTR** pin, then
 - If **IF**= 0 then the interrupt request is ignored.
 - If **IF**= 1 then 8086
 1. saves the flag register on the stack,
 2. saves the old program context on the stack,
 3. and clears TF and IF.
 4. respond with two pulses at $\overline{\text{INTA}}$ during interrupt acknowledge bus cycle (see figure 11-4).
 - The first pulse signals the external circuitry that the interrupt request has been acknowledged and to prepare to sent the number to the 8086.
 - The second pulse tells the external circuitry to put the type number on the data bus.
- **RESET :**
 - The reset input of the 8086 MPU provides a hardware means for initializing the microcomputer.
 - After reset the MPU start execution at address:
 - CS : IP = FFFFH : 0000H
 - This mean the physical address is FFFF0₁₆
 - What instructions should be written in this address?

Example 14-2: Develop a circuit that places interrupt type number 60H on the data bus in response to the INTR.

(a) Draw the interrupt circuit interface.

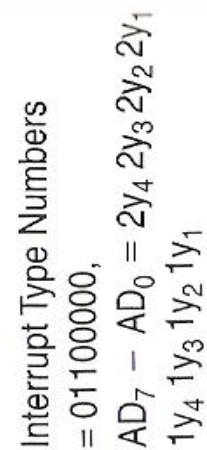
(b) Then write program at address 2000 :1000H that

- Increment the content of memory location 0100H by 1.
- Output the new content at output port 5000H.

(c) Make this program an ISR for the type 60H interrupt.

Solution:

(a) circuit diagram:



```
2000:1000    PUSH AX
              PUSH DX
              MOV  DX, 5000H
              MOV  AX, [0100H]
              INC  AX
              MOV  [0100H], AX
```



OUT DX, AX
POP DX
POP AX
IRET

(c) Type 60h interrupt vector is stored at address:

$$\text{Address} = 60\text{H} \times 4 = 180\text{H}$$

Then $\text{IP}_{60\text{H}}$ is stored at addresses 00180H and 00181H and $\text{CS}_{60\text{H}}$ is stored at addresses 00182H and 00183H.

14.5 Internal interrupt function

It is involve four types: divide error, overflow error, single step, and breakpoint.

Single Step

The *single-step* function relates to an operation option of the 8086. If the trap flag (TF) is set, the *single-step* mode of operation is enabled.

When TF is set, the MPU initiates a type 1 interrupt to the service routine defined by IP_1 and CS_1 at addresses 00004_{16} and 00006_{16} , respectively, at the completion of every instruction of the user program.

Problems:

1. List in order the interrupt groups; start with the lowest priority and end with the highest priority.
2. What is the range of type numbers assigned to the interrupts in the 8086 microcomputer system?
3. How many bytes of memory does an interrupt vector take up?
4. How many bytes of memory does an interrupt vector table take up?
5. Which interrupt function's service routine is specified by $\text{CS}_4:\text{IP}_4$?
6. The breakpoint routine in an 8086 microcomputer system starts at address $\text{AA}000_{16}$ in the code segment located at address $\text{A}0000_{16}$. Specify how the breakpoint vector will be stored in the interrupt-vector table.
7. What type number and interrupt vector table addresses are assigned to NMI?
8. List the internal interrupts serviced by the 8086.
9. Draw the interrupt acknowledgment cycle.
10. Develop a circuit that places interrupt type number CCH on the data bus in response to the INTR.
11. Explain briefly INTO instruction.





lecture 15

8254 programmable interval timer

14.1 The 8254 programmable interval timer

The 8254 programmable interval timer consists of three 16-bit programmable counters (timers). Each counter is capable of counting in binary or binary coded decimal (BCD). The maximum allowable input frequency to any counter is (10 MHz). This device is useful whenever the microprocessor must control real-time events. Some examples of usage include real-time clock, events counter, and motor speed and direction control. Figure (15-1) shows the pin out configuration of 8254 and Figure (15-2) shows its block diagram.

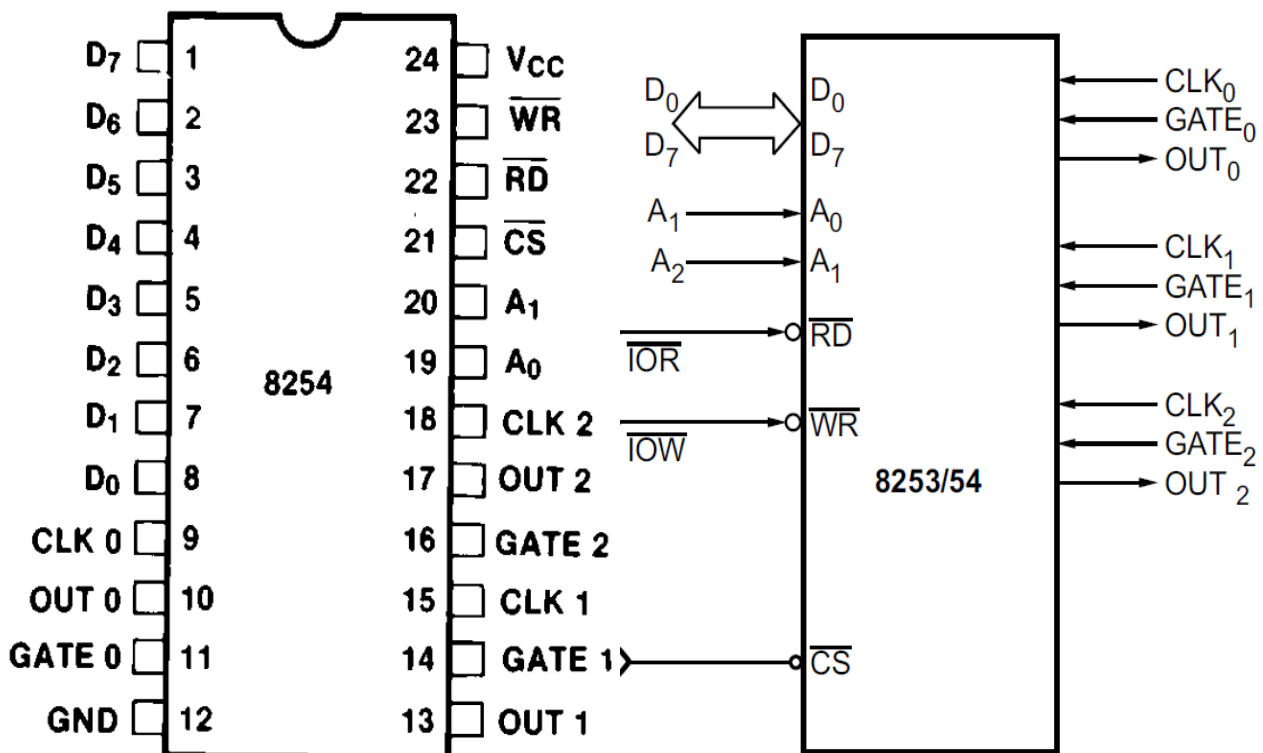


Figure (15-1) 8254 pin out configuration

Figure (15-2) 8254 block diagram.

Each timer contains a **CLK** input, a gate input (**GATE**) and an output (**OUT**). The **CLK** inputs provides the basic operating frequency to the timer, the **GATE** pin controls the timer in some modes, and the **OUT** pin is where we obtain the output of the timer.

Note: The three counters in 8254 operate as **Down Counters**.



The signal that connected to the microprocessor are: the data bus pins (D7-D0), $\overline{RD}$, $\overline{WR}$, $\overline{CS}$ and address input A1 and A0. The address inputs are present to select any of the four internal registers used for programming, reading or writing to a counter. The table below show the address selection inputs to the 8254:

A_1A_0	Function
00	Counter 0
01	Counter 1
10	Counter 2
11	Control word register

Programming the 8254

The control word register section actually contains three 8-bit registers used to configure the operation of counters 0, 1, and 2. The format of a control word is shown in Figure (15-3).

$A_1, A_0 = 11$ $\overline{CS} = 0$ $\overline{RD} = 1$ $\overline{WR} = 0$

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
SC1	SC0	RW1	RW0	M2	M1	M0	BCD

SC—Select Counter

SC1	SC0	
0	0	Select Counter 0
0	1	Select Counter 1
1	0	Select Counter 2
1	1	Read-Back Command (see Read Operations)

M—Mode

M2	M1	M0	
0	0	0	Mode 0
0	0	1	Mode 1
X	1	0	Mode 2
X	1	1	Mode 3
1	0	0	Mode 4
1	0	1	Mode 5

RW—Read/Write

RW1	RW0	
0	0	Counter Latch Command (see Read Operations)
0	1	Read/Write least significant byte only
1	0	Read/Write most significant byte only
1	1	Read/Write least significant byte first, then most significant byte

BCD

0	Binary Counter 16-bits
1	Binary Coded Decimal (BCD) Counter (4 Decades)

NOTE:

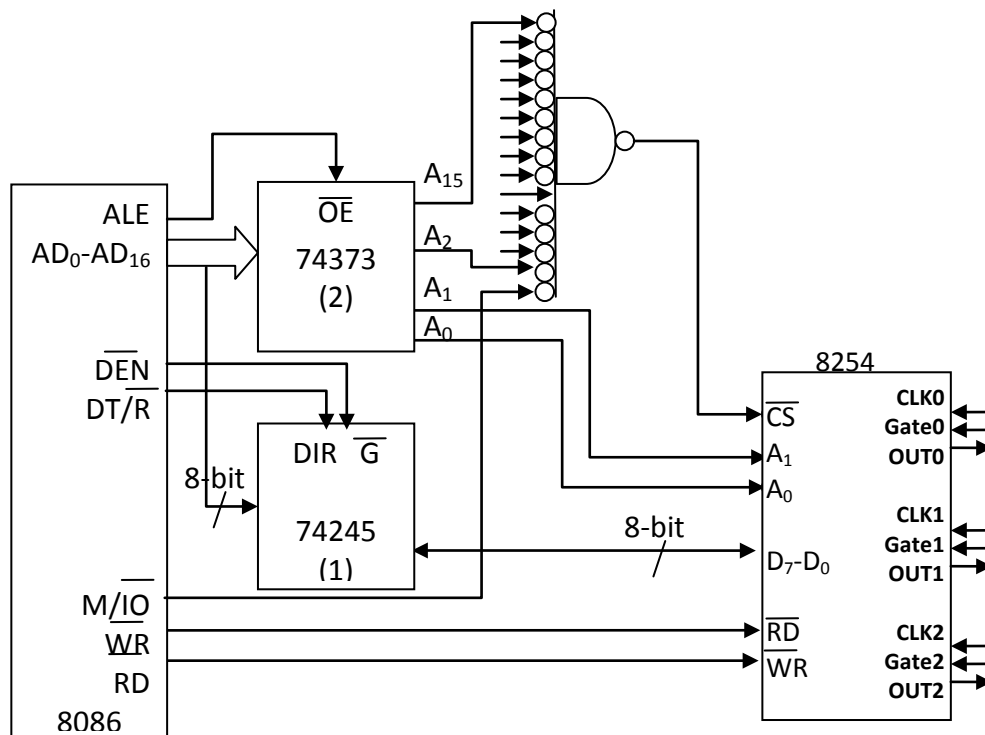
Don't care bits (X) should be 0 to insure compatibility with future Intel products.

Figure 15-3. Control Word Format



Here we find that the two most significant bits are a code that assigns the control word to a counter. For instance, making these bits 01 select **counter 1**. Bits **D₁** through **D₃** are a three bit **mode select** code, **M₂M₁M₀**, which selects one of six modes of counter operation. The least significant bit **D₀** is labelled BCD and selects either binary or BCD mode of counting. For instance, if this bit is set to logic 0, the counter acts as a 16-bit binary counter. Finally the 2-bit code **RW1, RW0** is used to set the sequence in which bytes are read from or loaded into the 16-bit count register.

Example 1: for the figure shown below, write a program sequence to set up the three counters of the 8254 programmable interval timer as follows:



COUNTER 0: binary counter operating in mode 0 with initial value of 1234H

COUNTER 1: BCD counter operating in mode 2 with initial value of 99H

COUNTER 2: binary counter operating in mode 4 with initial value of 1FFFH

Solution

From figure above:

- Counter 0 require address 40H
- Counter 1 require address 41H
- Counter 2 require address 42H
- Control register require address 43H

Following the bit definitions of control word format we get



Mode word for counter 0 = $00110000_2 = 30_{16}$

Mode word for counter 1 = $01010101_2 = 55_{16}$

Mode word for counter 2 = $10111000_2 = B8_{16}$

The following program sequence can be used to set up the 8254 with the desired mode words and counts:

```
MOV AL, 30H ; set up counter 0 mode
OUT 43H, AL
MOV AL, 55H ; set up counter 1 mode
OUT 43H, AL
MOV AL, B8H ; set up counter 2 mode
OUT 43H, AL
MOV AL, 34H ; initialize counter 0 with 1234H
OUT 40H, AL
MOV AL, 12H
OUT 40H, AL
MOV AL, 99H ; initialize counter 1 with 99H
OUT 41H, AL
MOV AL, FFH ; initialize counter 2 with 1FFFH
OUT 42H, AL
MOV AL, 1FH
OUT 42H, AL
```

Reading the content of the counter on the fly

The contents of the count registers can be read without inhibiting the counter. That is, the count can be read on the fly. To do this in software, a command must be first issued to the mode register to capture the current value of the counter into a temporary internal storage register. Setting bits D5 and D4 (RW1/RW0) of the mode byte to 00 specifies the **latch mode** of operation. Once this mode byte has been written to 8254, the contents of the temporary storage register for the counter can be read.

The format of latching command is as follow (must written to control register):

$A_1, A_0 = 11$; $CS = 0$; $RD = 1$; $WR = 0$

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
SC1	SC0	0	0	X	X	X	X



Example2: Write a program sequence to read the contents of counter 2 on the fly,in the previous example.

Solution:

The mode word of counter 2 will be:

$$10000000_2 = 80H$$

Program

```
MOV AL, 80H ; latch counter 2
OUT 43H, AL
IN AL, 42H ; read the low byte
MOV BL, AL
IN AL, 42H ; read the high byte
MOV AH, AL
MOV AL, BL ; AX= counter 2 value
```

Modes of Operation

Six modes (mode0 – mode5) of operation are available to each of 8254 counters. We will discuss two modes only (mode 2 and mode 3).

MODE 2

Mode 2 allows the counter to generate a series of continuous pulses that are one clock pulse wide. The separation between pulses is determined by the count. For example, for a count of 5, the output is a logic 1 for four clock periods and low for one clock period. The cycle is repeated until the counter is programmed with a new count of until the G pin is placed at a logic 0 level. The G input must be a logic 1 for this mode to generate a continuous series of pulses. Examples of **Mode2** behaviour is shown in Figure 15-4.

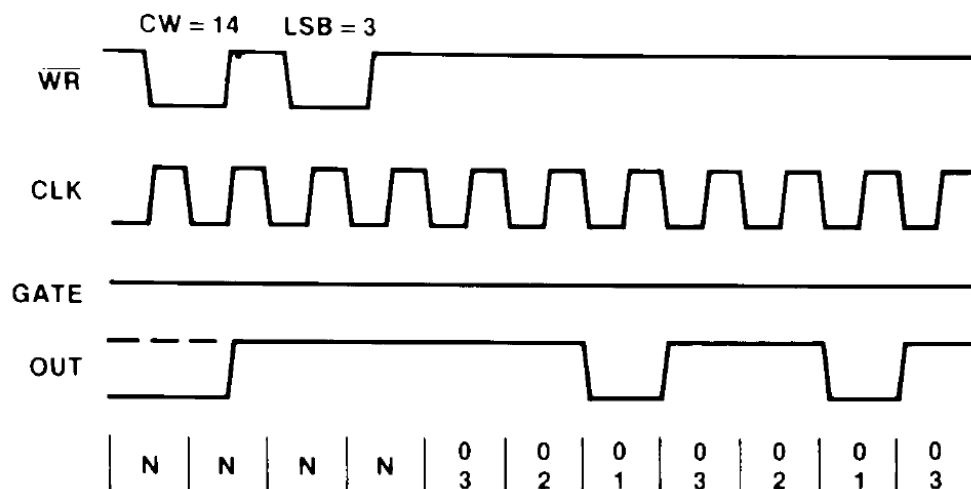


Figure 15-4 Mode 2 of 8254 (a)

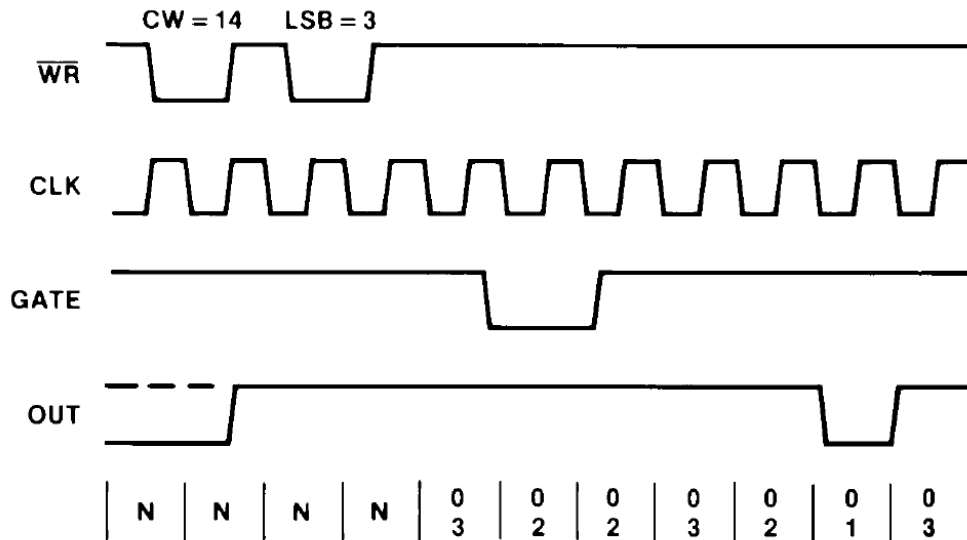


Figure 15-4 Mode 2 of 8254 (b)

Example 3

The counter 1 in Figure 15-5 is programmed to operate in mode 2 and is loaded with number 10_{16} . Describe the signal produced at OUT1. Assume that the counter is configured for BCD counting.

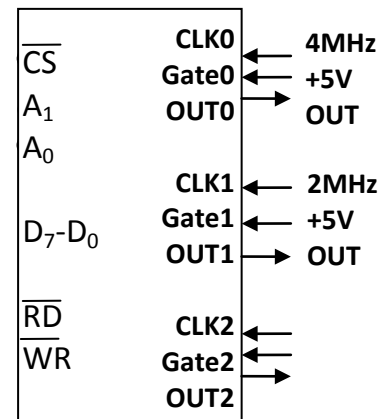


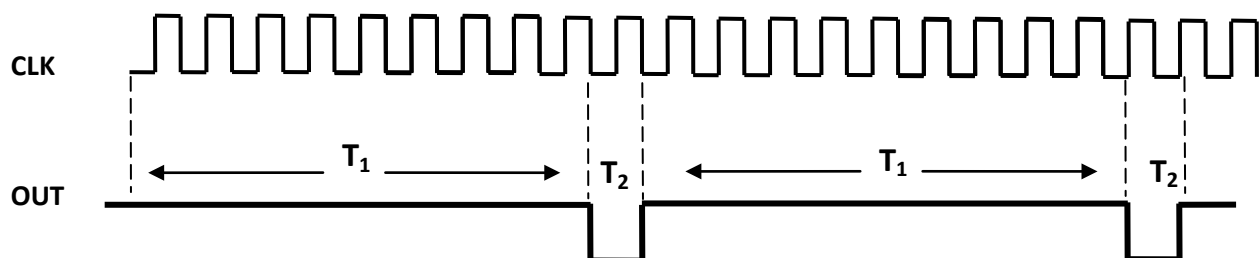
Figure 15-5

Solution: in mode 2 the output goes low for one period of the input clock after counter content decrement to zero, therefore

$$T_2 = 1/(2\text{MHz}) = 0.5 \times 10^{-6}\text{sec} = 0.5 \mu\text{sec}$$

$$T = 10 * 0.5 \mu\text{sec} = 5\mu\text{sec}$$

$$\text{Output frequency } F = 1/T = 1/(5\mu\text{sec}) = 200\text{KHz}$$



MODE 3

Mode 3 generates a continuous square-wave at the OUT connection, provided that the G pin is logic 1. If the count is even, the output is high for on-half of the count and low for one-half of the count. If the count is odd, the output is high for one clocking



period longer than it is low. For example, if the counter is programmed for a count 5, the output is high for three clocks and low for two clocks.

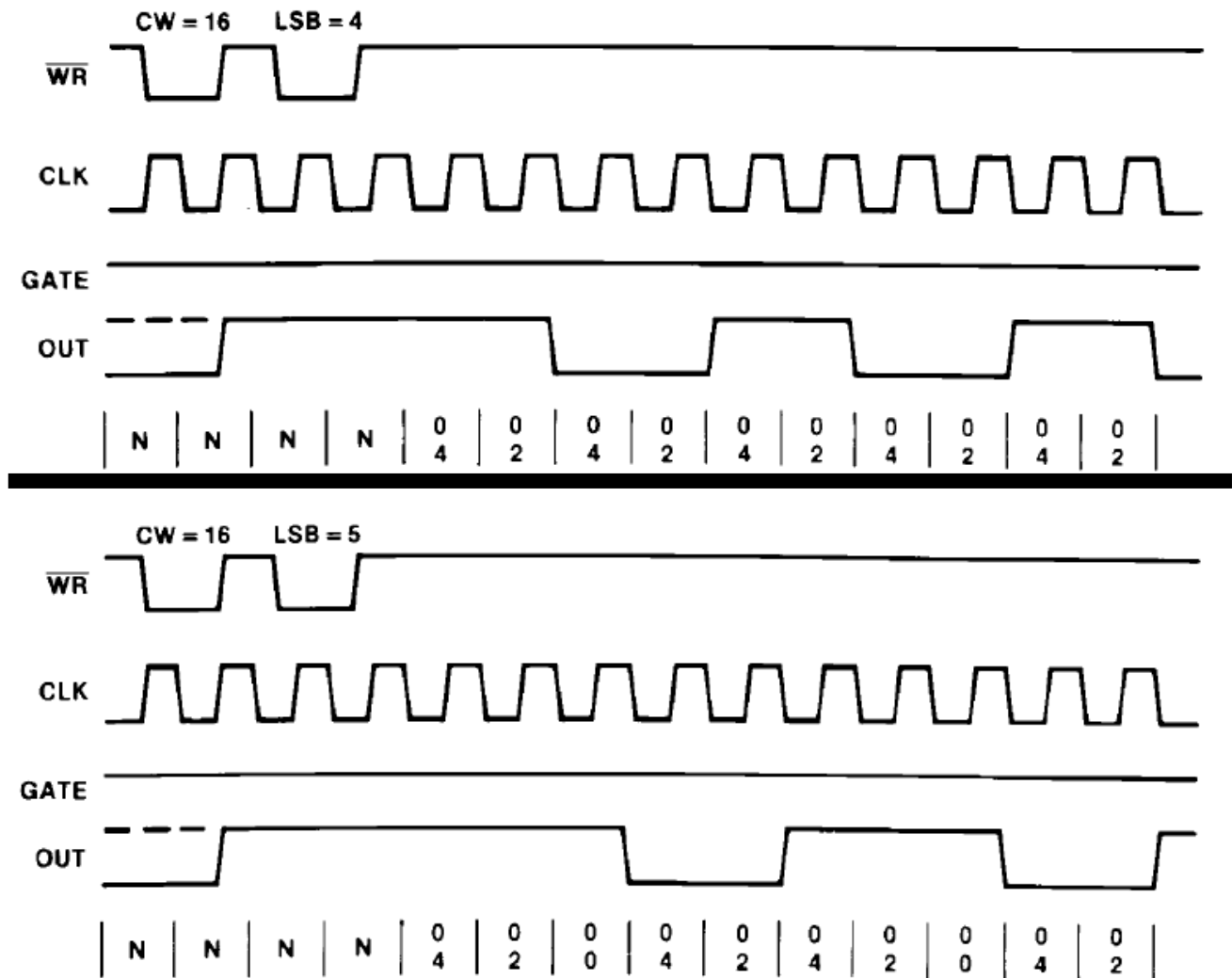


Figure 15-6 Mode 3 of 8254

Example 4

The counter 0 in Figure 15-5 is programmed to operate in mode 3 and is loaded with the value 15_{16} . Determine the characteristics of the square wave at the OUT1. Assume that the counter is configured for BCD counting.

Solution:

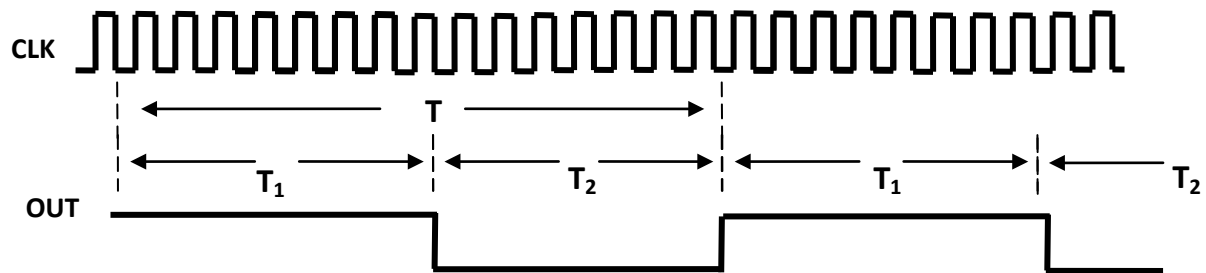
$$T_{clk1} = 1/(4\text{MHz}) = 0.25 \mu\text{sec}$$

$$T_1 = T_{clk1} * (N+1)/2 = 0.25 \mu\text{sec} [(15+1)/2] \\ = 2 \mu\text{sec}$$

$$T_2 = T_{clk1} * (N-1)/2 = 0.25 \mu\text{sec} [(15-1)/2] \\ = 1.75 \mu\text{sec}$$

$$T = T_1 + T_2 = 2 \mu\text{sec} + 1.75 \mu\text{sec} = 3.75 \mu\text{sec}$$

$$F_{out} = 1/(T_{out}) = 1/(3.75 \mu\text{sec}) = 0.2666 * 10^6 \text{ Hz} = 266.666 \text{ KHz}$$



Example 4: A 2-MHz clock is available for timing in a system that needs to be interrupted once every 4 second. How can two counters be cascaded to obtain this interrupt rate? (assume addresses are:

Counter0 address=4000H

Counter1 address=4001H

Counter2 address=4002H

Control register address=4003H

Solution

Figure 15-7 shows how counters 0 and 1 are connected to implement the 0.25-Hz interrupt clock. The 2-MHz clock is connected to CLK0 will create output pulses on OUT0 when counter 0 is programmed in mode2. These output pulses serve as the clock for counter 1 (also programmed in mode 2). Dividing 2MHz by 0.25 Hz gives 8,000,000!.

This is the count that must be simulated by both counters. Many different counting schemes are possible. Once scheme requires that counter 0 be loaded with 50,000 and counter 1 with 160. Note that the product of these two numbers is 8,000,000. Counter 0 will output one pulse for every 50,000 CLK0 pulses. Counter 1 will output one pulse for every 160 CLK1 pulse.

The instructions needed for this interrupt timing circuit are:

```
MOV DX, 4003H
MOV AL, 34H
OUT DX, AL
MOV AL, 54H
OUT DX, AL
MOV DX, 4000H
MOV AX, C350H
OUT DX, AL
MOV AL, AH
OUT DX, AL
INC DX
MOV AL, A0H
OUT DX, AL
HLT
```

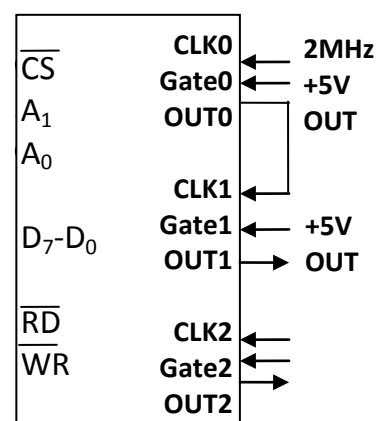


Figure 15-7



lecture 16

Intel processors

The 8286 microprocessor

The 80286 microprocessor (also a 16-bit architecture microprocessor) was almost identical to the 8086, except it addressed a 16M byte memory system instead of a 1M byte system. The instruction set of the 80286 was almost identical to the 8086, except for a few additional instructions that managed the extra 15M bytes of memory. The clock speed of the 80286 was increased, so it executed some instructions in as little as 250ns with the original release 8.0 MHz (see figure 1).

The 80386 microprocessor

Applications began to demand faster microprocessor speed, more memory, and wider data paths. This led to the arrival of the 80386 in 1986, by Intel Corporation. The 80386 represent a major overhaul of the 16-bit 8086-80286 architecture. The 80386 was Intel's first practical 32-bit microprocessor that contained a 32-bit data bus and a 32-bit memory address, through these 32-bit buses, the 80386 addresses up to 4G bytes of memory.

Besides providing higher clocking speeds, the 80386 included a memory management unit that allowed memory resources to be allocated and managed by the operation system.

The instruction set of the 80386 was upward-compatible with the earlier 8086, 80286 microprocessors.

The 80386 was available in a few modified versions such as :

- The 80386SX: address 16M bytes of memory.
- The 80386SL: address 32M bytes of memory.
- The 80386SLC: address 32M bytes of memory and contain an internal cache memory that allowed it to process data at higher rates.

The 80486 microprocessor

In 1989, Intel released the 80486 microprocessor, which incorporate an 80386-like microprocessor, and 80387-like numeric coprocessor, and an 8K byte cache memory system into one integrated package.

The internal architecture of the 80486 was modified so that about half of its instructions executed in one clock instead of two clocks. Also the 80486 was available in different versions.



The Pentium microprocessor

The Pentium introduced in 1993, able to address 4G bytes and has 16K bytes cache.

The most important feature of the Pentium is its **dual integer processors**. The Pentium executes two instructions, which are not dependent on each other, simultaneously because it contains two independent internal integer processors called super scaled technology.

Another feature that enhances performance is a **jump prediction** technology that speeds the execution of programs that include jump.

Real and protected mode

The 80286 and above operate in either the **real** or **protected** mode. Only the 8086 operate exclusively in the real mode. **Real mode operation** allows the microprocessor to address only the first 1M byte of the memory space (even if it is the Pentium 4 microprocessor). The DOS operating system requires the microprocessor to operate in the real mode. Real mode operation allows application software written for the 8086, which contain only 1M byte of memory, to function in the 80286 and above without changing the software. In all cases, each of these microprocessors begins operation in the real mode by default whenever power is applied or microprocessor is reset.

When configure for protected-mode the microprocessor provides more instruction and advanced software architecture (like memory management, paging and multitasking for 80386).

Virtual 8086 mode

This special mode is designed so that multiple 8086 real-mode software applications can execute at one time. The virtual 8086 mode can be used to share one microprocessor with many users by portioning the memory so that each user has its own DOS partition (see figure 2). When in this mode, the 80386DX (and above) support an 8086 microprocessor programming model and can directly run programs writhe for the 8086. That is, it creates a virtual 8086machine for executing the program.

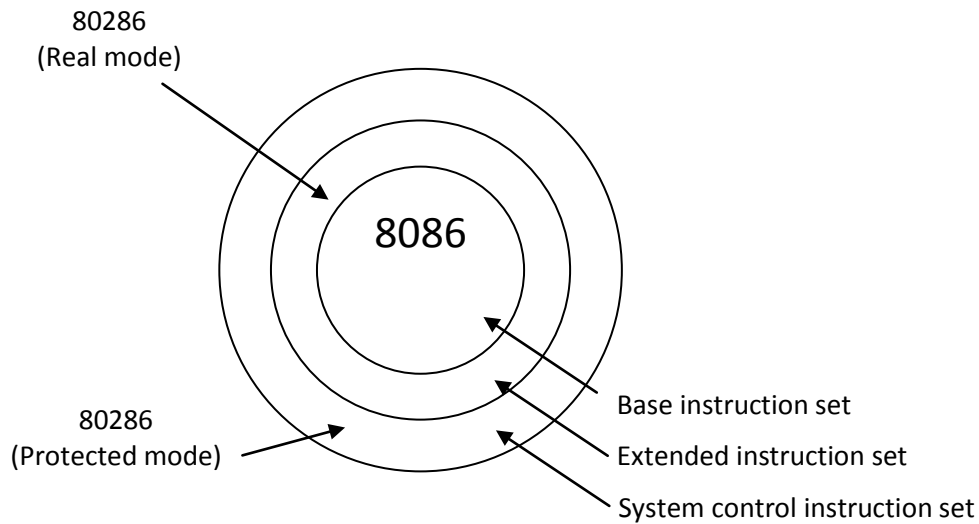


Figure 1 evolution of the instruction set of the 8086 microprocessor family

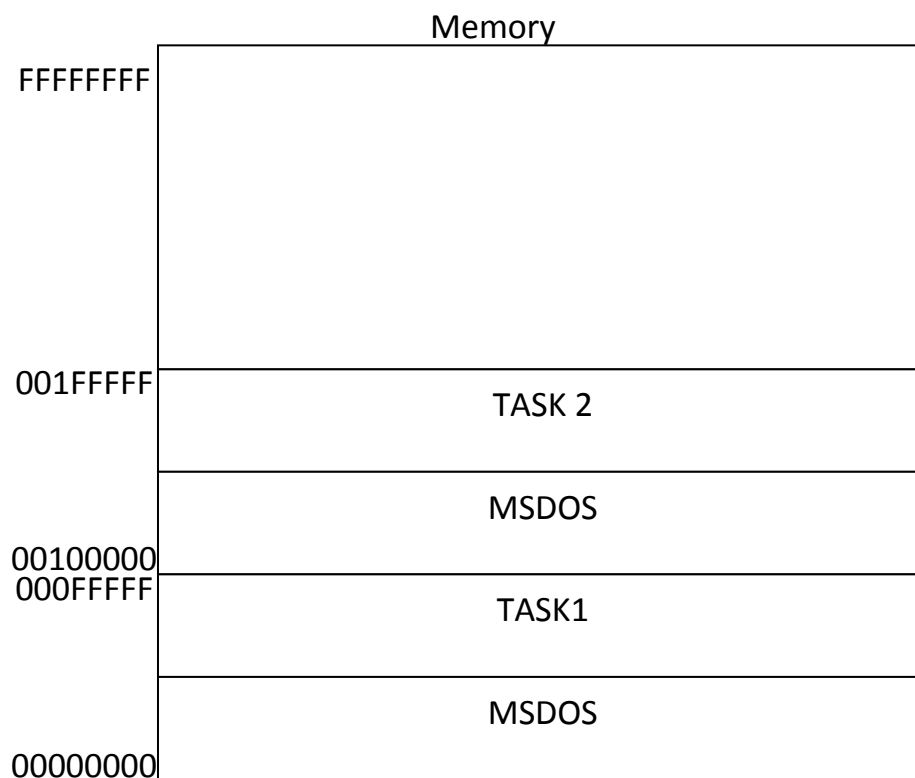


Figure 2 Two tasks resident to an 80386 operated in the virtual 8086 mode



Microprocessors vs. Microcontrollers

	Microprocessor	Microcontroller
Hardware architecture	Microprocessor is a single-chip CPU	Microcontrollers contains, in a singles IC, a CPU and much of remaining circuitry of a complete microcomputer system, like RAM, ROM,a serial interface, a parallel interface, timer, and interrupt scheduling circuitry all within in the same IC.
Applications	Microprocessors are most commonly used as the CPU in microcomputer systems. They are suited to processing information in computer systems.	Microcontrollers are found in small, minimum-component designs performing control-oriented activities. They are suited to control of I/O devices in designs requiring minimum component count.
Instruction set features	Instruction sets are processing intensive implying they have powerful addressing modes with instructions catering to operations on large volumes of data. Their instructions operate on bytes, words and double words. Addressing modes provide access to large arrays of data, using address pointers and offset.	Microcontrollers have instruction sets catering to the control of inputs and outputs. Microcontrollers have instructions to set and clear individual bits and perform other bit-oriented operations such as logically ANDing, ORing or XORing bits. The instructions are highly compact. The majority of instructions are implemented in a single byte.
Example	8086, 80286 . . .	8048, 8051 . . .